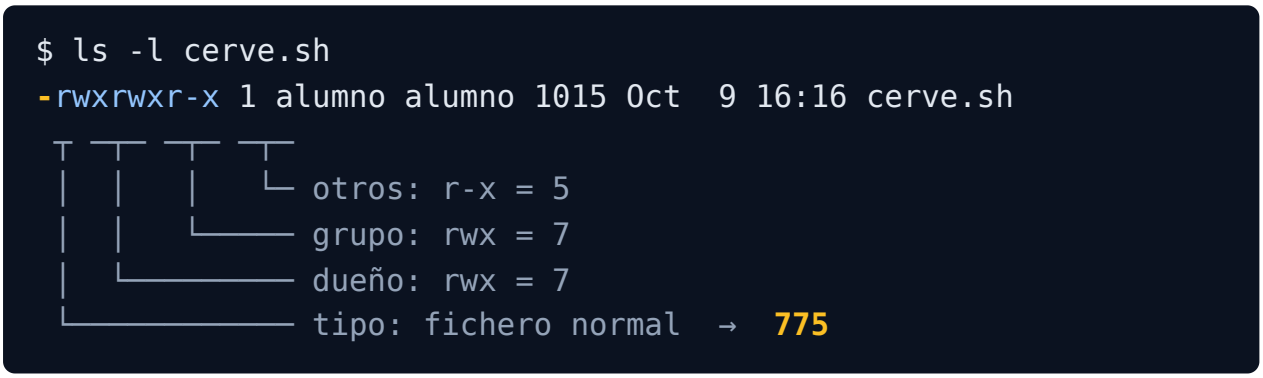
LPIC-1 · OBJETIVO 104.5 Y 110.1

Permisos en Linux

De **cero a master**: entender `-rwxrwxr-x`, `755` y `4755` nada más verlos.



- ✔ 15 capítulos paso a paso
- ✔ 188 ejercicios, series y retos
- ✔ Salida real de cada comando
- ✔ Laboratorio con usuarios ana · beto · carla
- ✔ chmod, umask, chown, SUID/SGID/sticky
- ✔ Octal ⇄ binario ⇄ decimal ⇄ letras
- ✔ Diagnóstico de “Permission denied”
- ✔ Mini-examen tipo LPIC-1 (30 preguntas)

Edición de 16/10/2026 · Código, laboratorio y herramientas: **github.com/TheBillBull/Guias-y-Herramientas**

Índice

	Presentación, instalación y mapa LPIC-1	**3**
0	Antes de empezar: quién eres, quién es cada uno y el laboratorio	**8**
1	Leer ls -l de un vistazo	**15**
2	r, w y x en los ficheros: leer, modificar, ejecutar	**27**
3	r, w y x en los directorios: el capítulo que casi todos entienden mal	**34**
4	¿Quién decide qué? El algoritmo de los permisos	**43**
5	Octal, binario, decimal y letras: entender 755, 644 o 4755 de un vistazo	**51**
6	chmod en modo simbólico: u+x, go-w, a=r...	**66**
7	chmod en modo octal: 755, 644, 4755... y cómo aplicarlos a un árbol	**76**
8	umask: por qué tus ficheros nuevos nacen con esos permisos	**85**
9	chown y chgrp: cambiar el dueño y el grupo	**95**
10	Permisos especiales: SUID, SGID y sticky (s, S, t, T, 4755, 2775, 1777)	**103**
11	Los permisos en la vida real: el sistema, los comandos de cada día y los enlaces	**115**
12	Más allá de rwx: atributos, ACL, capabilities y montajes	**123**
13	Diagnóstico: "¡Permission denied!" y qué hacer · Recetas	**130**
14	🏆 El jefe final: misiones, auditoría y mini-examen LPIC-1	**139**
★	Chuleta de una página	**149**

Las **soluciones** de los ejercicios aparecen desplegadas en cajas verdes: **inténtalo primero** y compara después. Los marcadores del lector de PDF permiten saltar a cada apartado.

🔐 Permisos en Linux — de cero a *master* (LPIC-1)

> Una guía **paso a paso, como para un niño que ve `ls -l` por primera vez**, que termina con ejercicios de nivel experto. **188 ejercicios, series y retos** (más un mini-examen de **30 preguntas** tipo LPIC-1), con la **salida real** de cada comando, un **laboratorio** que se monta con un script y **cuatro herramientas** para convertir y diagnosticar permisos.

Nació de una captura de pantalla como esta (`ls -l saludos.sh`, `ls -l /etc/shadow`, `ls -ld /tmp/`, `ls -ld /var/spool/cron/crontabs/`…) y de una pregunta: *«quiero entender `-rwxrwxr-x` nada más verlo, y entender qué es `777` o `4755`»*. Al terminar, lo harás.

```
-rwxrwxr-x 1 alumno alumno 1015 Oct  9 16:16 cerve.sh
│└┘└┘└┘                                       └─ …y el 775 que significa
│ │  │  └─ otros: pueden leer y ejecutar (r-x = 5)
│ │  └─ grupo:   pueden leer, escribir y ejecutar (rwx = 7)
│ └─ dueño:      puede leer, escribir y ejecutar (rwx = 7)
└─ es un fichero normal
```

Los comandos de este libro **se ejecutaron de verdad** (Ubuntu 24.04, GNU coreutils 9.4, bash 5.2) con un usuario normal llamado `alumno`, y lo que ves como *Resultado* es lo que salió. Cuando algo falla o sorprende (y pasa a menudo: el SGID que *desaparece en silencio*, el `chmod -w` que quita de menos, el `chown` que borra el SUID…), el libro lo cuenta. Solo hay unas pocas partes **no ejecutadas**, y están marcadas como tales (ver más abajo).

🚀 Empieza aquí

Necesitas un **Ubuntu** (o cualquier Linux con GNU coreutils), una terminal (`Ctrl` + `Alt` + `T`) y un usuario que pueda usar `sudo`.

```
# 1. Descarga la guía
cd ~
git clone https://github.com/TheBillBull/Guias-y-Herramientas.git

# 2. Monta el laboratorio (crea ~/lab-permisos y los usuarios de práctica ana, beto y carla;
#    te pedirá tu contraseña de sudo y que confirmes)
bash ~/Guias-y-Herramientas/guias/permisos-linux-lpic1/laboratorio/preparar-laboratorio.sh

# 3. Instala las herramientas (opcional pero muy recomendado)
sudo install -m 755 ~/Guias-y-Herramientas/guias/permisos-linux-lpic1/herramientas/permisos
/usr/local/bin/
sudo install -m 755 ~/Guias-y-Herramientas/guias/permisos-linux-lpic1/herramientas/diagnostica
/usr/local/bin/

# 4. Entra y empieza por el capítulo 0
cd ~/lab-permisos
```

> 🔁 Si estropeas el laboratorio, **vuelve a ejecutar el paso 2**: lo deja como nuevo. Y cuando termines: `bash …/preparar-laboratorio.sh --limpiar` lo **borra todo** (la carpeta, los usuarios `ana` `beto` `carla`, los grupos y el permiso de tu carpeta personal). Solo borra lo que él mismo creó.

Luego sigue los capítulos **en orden**: cada uno se apoya en el anterior.

📚 Los capítulos

#	Capítulo	Ejercicios	Qué aprenderás
00	Antes de empezar: quién eres, quién es cada uno y el laboratorio	8	`whoami` `id` `groups`, `/etc/passwd`, `root`, `sudo -u`, el elenco ana·beto·carla, cómo probar "como otra persona"
01	Leer `ls -l` de un vistazo	15	Los 10 caracteres, tipos de fichero, columnas, `ls -ld`, `stat`. ⭐ **Tu captura, línea a línea** (incluido el `°`)
02	`r`, `w`, `x` en los ficheros	13	Leer, modificar, ejecutar; `./script` frente a `bash script`; `x` sin `r`; binarios; `w` no es borrar
03	`r`, `w`, `x` en los directorios	11	⭐ **El que casi todos entienden mal**: las 8 combinaciones medidas, buzón `-wx`, `namei -l`, borrar manda la carpeta
04	¿Quién decide qué?	13	El algoritmo (root → dueño → grupo → otros), paradojas, grupos principal/secundarios, `usermod -aG` y `newgrp`
05	Octal, binario, decimal y letras	16	⭐ **755, 644, 4755 de un vistazo**: 4-2-1, conversiones rápidas, trampa del decimal, herramienta `permisos`, series de velocidad
06	`chmod` simbólico	15	`u+x` `go-w` `a=r`, `g=u`, `X`, `-R`, `--reference`, y la trampa del "sin quién" (`chmod -w` + umask)
07	`chmod` octal	14	Equivalencias, octal frente a simbólico, `chmod -R` sin romper nada (`find`, `X`), bits como `|` y `& ~`
08	`umask`	16	`666 & ~umask`, **no es una resta**, cómo cambiarla, dónde se configura (Ubuntu `0002`), qué comandos la obedecen
09	`chown` y `chgrp`	14	Quién puede cambiar qué, todas las sintaxis, `-R`, `--from`, huérfanos, casos reales (`sudo` deja ficheros de root)
10	SUID, SGID y *sticky*	16	⭐ `s` `S` `t` `T`, `4755` `2775` `1777`; demostraciones reales; carpeta de equipo; auditoría con `find -perm`
11	Los permisos en la vida real	11	Mapa del sistema (`/etc/shadow`, `/home`, `/tmp`, `/var/mail`…), qué pide cada comando, `sed -i`, enlaces duros y simbólicos, SSH
12	Más allá de `rwx`	10	`chattr +i/+a`, opciones de montaje, *capabilities*, ACL, AppArmor/SELinux, y el mapa de todas las "puertas"
13	Diagnóstico y recetas	7	El método de 7 pasos, `namei`, `diagnostica`, `strace`, casos reales, recetario y **retos de reparación**
14	El jefe final	9	Misión intranet con **autocomprobación**, auditoría de seguridad, reconstruir listados, velocidad y **mini-examen de 30 preguntas**
—	📄 Chuleta de una página		Todo resumido para repasar

⭐ = lo que salía en tu captura de pantalla, explicado y reproducido:

Lo que enseñaste	Dónde está
`ls -l cerve.sh` → `-rwxrwxr-x 1 alumno alumno 1015 …`	Cap. 1 · 1.1
`ls -l saludos.sh` → `-rwxr-x--x` (751) y por qué "otros" no pueden ejecutarlo	Cap. 1 · 1.10 · Cap. 2
`ls -l salario.sh °` → el error por el `°`	Cap. 1 · 1.10 ②
`ls -l /etc/shadow` (`-rw-r-----` root:shadow) y `/etc/passwd` (`r--` de otros)	Cap. 1 · 1.10 · Cap. 11
`ls -ld /tmp/` → `drwxrwxrwt` (1777)	Cap. 1 · Cap. 10 · sticky
`sudo ls -ld /var/spool/cron/crontabs/` → `drwx-wx--T` (1730)	Cap. 1 · Cap. 3 · el buzón · Cap. 10 · recreado
"¿qué es 777, 4755…?" y "convertir rápido"	Cap. 5 · Cap. 10 · entrenador

🛠️ Las herramientas

En `herramientas/` y `laboratorio/`. Todas están **verificadas contra el comportamiento real del sistema** (miles de comparaciones automáticas).

Herramienta	Para qué	Cómo se usa
`permisos` (script de Bash)	**Traduce** permisos: octal ⇄ letras ⇄ binario ⇄ decimal ⇄ hex; calcula `umask`; aplica expresiones de `chmod`	`permisos 755` · `permisos -rwsr-xr-x` · `permisos 644 g+w,o-r` · `permisos -u 027` · `permisos /etc/shadow`
`diagnostica` (script de Bash)	**¿Por qué `carla` no puede leer esto?** Recorre la ruta tramo a tramo con el algoritmo real y señala el bloqueo	`sudo diagnostica -u carla /ruta/al/fichero`
`calculadora-chmod.html`	Calculadora **visual**: casillas `r/w/x` + especiales, conversión en todas direcciones y calculadora de `umask`	ábrela en el navegador
`entrenador-conversion.html`	**Entrenador de velocidad**: preguntas aleatorias, pasos de cálculo, racha, tiempo medio y *sprint* de 60 s	ábrelo en el navegador
`preparar-laboratorio.sh`	Crea `~/lab-permisos`, los usuarios `ana` `beto` `carla` y los grupos `equipo` `soporte`	`bash preparar-laboratorio.sh [--sin-usuarios | --solo-usuarios | --limpiar]`

🎚️ Cómo se hacen los ejercicios

Icono	Nivel
🟢	Fácil: aplicar lo que acabas de aprender
🟡	Medio: combinar dos o tres ideas
🔴	Difícil: hay que pensar un rato
•	Master: preguntas de examen, trampas y retos

La **solución está escondida** en un desplegable (`💡 Ver solución`). La técnica que funciona: **(1)** lee, **(2)** escribe tu intento, **(3)** ejecútalo en tu laboratorio, **(4)** si no sale, piensa por qué, **(5)** solo entonces mira. Los comandos se muestran de dos maneras: **bloque + *Resultado*** y **transcripción** (`$ comando` y debajo su salida).

🧪 La filosofía: probar como otra persona

Lo que distingue a esta guía: para entender un permiso **hay que ver cómo falla**. Por eso el laboratorio tiene **cuatro personas** y todo se prueba con `sudo -u ana …`, `sudo -u carla …`:

Quién	Qué es	Grupos
🧑‍🎓 **alumno**	**tú**	`alumno`
👩 **ana**	compañera de equipo	`ana`, `equipo`
👨 **beto**	de equipo y de soporte	`beto`, `equipo`, `soporte`
👩‍🔧 **carla**	solo de soporte	`carla`, `soporte`

🔬 Sobre los datos del libro (y qué no se ejecutó)

- **Entorno de las salidas:** Ubuntu 24.04, GNU coreutils 9.4, bash 5.2, sistema de ficheros ext4. Cada comando se tecleó en una **terminal real** del usuario `alumno` con `umask 0002` (la de Ubuntu); los `sudo -u usuario …` se ejecutaron **como ese usuario** (con el `umask 0022` que impone `sudo`).
- **Idioma:** las salidas están en inglés (`LANG=C.UTF-8`): `Permission denied`. En un Ubuntu en español verás *Permiso denegado*, y los meses de `ls -l` en minúscula (`oct`). La tabla de equivalencias está en el

capítulo 0.

- **Tus números serán otros:** los UID/GID (`1001`, `1005`…), los tamaños, las fechas y los inodos cambian de una máquina a otra. Lo que importa son los **nombres** y los **permisos**.
- **Tu `/etc/shadow`, tus SUID…** no serán idénticos a los del libro (los ejemplos de ficheros del sistema se tomaron de la máquina de generación).
- **Los tres scripts de la captura** (`cerve.sh`, `saludos.sh`, `salario.sh`) se recrean con el **mismo tamaño en bytes** (1015, 93 y 419), las mismas fechas y los mismos permisos que en tus `ls -l`.
- **Lo que NO se ejecutó** (está marcado en el texto como "no ejecutado"): las **ACL** (`setfacl`/`getfacl`, capítulo 12: faltaban las herramientas en el entorno de generación y **no entran en LPIC-1**), el mensaje de **SSH** sobre claves demasiado abiertas (capítulo 11: no había cliente `ssh`), y los comandos **interactivos** (`newgrp` a pantalla completa, `vi`…).
- **Una advertencia sincera sobre `sudo`:** en el generador, `sudo -u ana …` lo ejecuta un proceso con privilegios que cambia de usuario (`runuser`); el resultado de los **permisos** es el mismo que con `sudo` de verdad, pero el *aviso de contraseña* de `sudo` no aparece en las salidas.

🧭 Ruta de estudio sugerida

Si tienes…	Haz esto
Una semana	Un capítulo y medio al día (0-10), con la chuleta al final. Practica el entrenador 5 min cada día.
Un mes	Semana 1: caps. 0-4 (lo conceptual). Semana 2: 5-8 (números, `chmod`, `umask`). Semana 3: 9-12. Semana 4: 13-14 y repetir los ●.
Prisa por el examen LPIC-1	Capítulos **1, 3, 4, 5, 6, 7, 8, 9, 10** y el **mini-examen del 14**. Los capítulos 11-13 son "vida real" (y lo preguntan en forma de casos).

Trucos para aprender: haz los ejercicios **sin mirar**; vuelve a hacerlos días después; cuando una solución te sorprenda, **cámbiala** y mira qué se rompe; usa `man chmod`, `man 7 credentials` y `info coreutils 'File permissions'` como libros de consulta; y **comprueba siempre con otro usuario** (`sudo -u carla …`): *"que no pueda"* también se prueba.

🗺️ Mapa LPIC-1 (101-500 / 102-500)

Objetivo	Capítulos
104.5 Gestionar permisos y propiedad de ficheros	**1-10** (el núcleo), 13, 14
104.6 Crear y cambiar enlaces duros y simbólicos	11 (permisos de los enlaces)
107.1 Gestionar cuentas de usuario y grupo y ficheros del sistema relacionados	0, 4, 9
110.1 Tareas de administración de seguridad (auditar SUID/SGID, `sudo`, `/etc/shadow`)	**10**, 11, 14
103.1 / 103.3 Línea de comandos y gestión básica de ficheros	0, 1, 2, 3
Extras de la vida real (LPIC-2 y más allá)	12 (`chattr`, ACL, *capabilities*, MAC)

📂 Contenido del repositorio

```
guias/permisos-linux-lpic1/
├── README.md                    ← este índice
├── 00-…14-*.md                  ← los 15 capítulos
├── chuleta.md                   ← resumen de una página
├── laboratorio/
│   └── preparar-laboratorio.sh  ← crea ~/lab-permisos, usuarios y grupos (y --limpiar)
└── herramientas/
    ├── permisos                 ← traductor de permisos (Bash)
    ├── diagnostica              ← ¿por qué este usuario no puede? (Bash)
    ├── calculadora-chmod.html   ← calculadora visual
    └── entrenador-conversion.html ← entrenador de velocidad
```

Capítulo 0 · Antes de empezar: quién eres, quién es cada uno y el laboratorio

> 🎯 **Objetivo:** entender *a quién* se le aplican los permisos (usuarios, grupos, `root`) y montar un laboratorio con varias "personas" para comprobar los permisos **de verdad**.
>
> 📘 **LPIC-1:** 104.5 (permisos y propiedad) · 107.1 (usuarios y grupos) · 103.1 (línea de comandos).
>
> 🧪 **Antes de empezar:** una terminal en un Ubuntu (`Ctrl` + `Alt` + `T`) y un usuario que pueda usar `sudo`.

0.1 · ¿Para qué sirven los permisos?

Imagina un **edificio de oficinas** con mucha gente dentro: tú, tus compañeros, el jefe, la limpiadora, un cartero... Si todas las puertas estuvieran abiertas para todos, cualquiera podría leer tu correo, tirar a la basura tus papeles o fotocopiar la nómina del jefe.

Linux es **multiusuario** desde el día uno: en el mismo ordenador conviven personas **y programas** (el servidor web, la base de datos, el reloj...), y cada uno es "alguien" distinto. Los **permisos** son el sistema de **llaves y cerraduras** que dice, para *cada* fichero y *cada* carpeta:

Pregunta	La respuesta vive en…
🧑 ¿**Quién es el dueño**?	el **propietario** (*owner*)
👪 ¿**Qué grupo** tiene derechos especiales?	el **grupo** (*group*)
🔑 ¿**Qué puede hacer** cada uno?	los **permisos** (*mode*): leer, escribir, ejecutar

Toda la guía es una forma larga de contestar a esas tres preguntas. Y todo se ve de un vistazo con `ls -l`, que aprenderás a leer **como si fuera tu idioma** en el capítulo 1.

0.2 · ¿Quién soy? `whoami`, `id` y `groups`

Antes de hablar de permisos hay que saber **quién eres tú para el sistema**. Tres comandos:

```
whoami
id
groups
```

Resultado:

```
alumno
uid=1001(alumno) gid=1001(alumno) groups=1001(alumno)
alumno
```

Cómo se lee `id`:

```
uid=1001(alumno) gid=1001(alumno) groups=1001(alumno)
 |        |          |       |           └─ TODOS tus grupos (números y nombres)
 |        |          |       └──────────── el nombre de tu grupo principal
 |        |          └──────────────────── el número de tu grupo principal (GID)
 |        └─────────────────────────────── el nombre de tu usuario
 └──────────────────────────────────────── el número de tu usuario (UID)
```

- El sistema **no entiende de nombres**, solo de **números**: el `uid` y el `gid`. Los nombres (`alumno`) son una comodidad para los humanos.
- Tu **UID** puede ser distinto al de este libro (en una Ubuntu recién instalada, el primer usuario suele ser el **1000**). Da igual: los **nombres** son lo que importa.
- Todo usuario tiene **un grupo principal** (el que se le pone a los ficheros que crea) y puede tener **otros grupos secundarios**.

> 💡 En Ubuntu cada usuario nuevo recibe un grupo **propio** con su mismo nombre (aquí `alumno:alumno`). Se llama *User Private Group* y es la razón de que veas `alumno alumno` (usuario y grupo iguales) en casi todos tus `ls -l`.

0.3 · Usuarios y grupos: `/etc/passwd` y `/etc/group`

Linux guarda la lista de personas y grupos en dos ficheros de texto **que ya hemos visto en tu captura**:

```
getent passwd alumno
getent group alumno
```

Resultado:

```
alumno:x:1001:1001:alumno LPIC:/home/alumno:/bin/bash
alumno:x:1001:
```

`getent` pregunta al sistema "dime la ficha de..." (y funciona aunque los usuarios vengan de LDAP, etc.). Es equivalente a `grep alumno /etc/passwd`. Las fichas:

```
alumno:x:1001:1001:alumno LPIC:/home/alumno:/bin/bash
  |    |   |    |      |           |            └─ la shell (programa que te recibe al entrar)
  |    |   |    |      |           └──────────────── carpeta personal (HOME)
  |    |   |    |      └──────────────────────────── comentario (nombre completo, etc.)
  |    |   |    └─────────────────────────────────── GID del grupo principal
  |    |   └──────────────────────────────────────── UID
  |    └──────────────────────────────────────────── "x" = la contraseña está en /etc/shadow
  └───────────────────────────────────────────────── nombre de usuario

alumno:x:1001:
  |    |   |   └─ miembros SECUNDARIOS del grupo (vacío: nadie más)
  |    |   └──── GID
  |    └──────── (contraseña de grupo, casi nunca se usa)
  └───────────── nombre del grupo
```

Fichero	Contiene	Permisos típicos
`/etc/passwd`	usuarios (nombre, UID, GID principal, HOME, shell)	`-rw-r--r--` (**todos** pueden leerlo)
`/etc/shadow`	las **contraseñas cifradas**	`-rw-r-----` root:shadow (**nadie** que no sea root o `shadow`)
`/etc/group`	grupos y sus miembros	`-rw-r--r--`
`/etc/gshadow`	contraseñas de grupo	`-rw-r-----`

> 🧠 **Por qué `passwd` es legible por todos pero `shadow` no:** `passwd` tiene que poder leerse para que `ls -l` traduzca "1001" a "alumno". Las contraseñas cifradas, en cambio, estuvieron en `passwd` hasta los años 90... y se descubrió que cualquiera podía copiarlas y adivinarlas. Por eso se movieron a `shadow`, con permisos mucho más cerrados. Verás **las dos** líneas (`passwd` y `shadow`) de tu captura dentro de un rato.

0.4 · `root`, el que lo puede todo

Hay un usuario especial: `root`, con **UID 0**. Para el sistema, "ser administrador" significa exactamente eso: tener el UID 0. `root` **se salta casi todos los permisos** (puede leer, escribir y borrar casi cualquier cosa), así que:

> ⚠️ **Trabaja siempre como usuario normal y pide ser `root` solo para el comando que lo necesite.** Un `rm -rf` equivocado como `root` puede destrozar el sistema; como usuario normal, los permisos te protegen.

La forma de pedirlo es `sudo` (*"superuser do"*): ejecuta **un solo comando** como otro usuario (por defecto `root`), pidiéndote **tu** contraseña.

```
sudo  [-u usuario]  comando
  │       └─ "como este usuario" (si no lo pones: root)
  └──────────── "ejecuta esto con otra identidad"
```

```
$ id root
uid=0(root) gid=0(root) groups=0(root)
$ sudo id
uid=0(root) gid=0(root) groups=0(root)
$ sudo -u ana id
uid=1002(ana) gid=1002(ana) groups=1002(ana),1005(equipo)
```

La primera línea usa `id root` (preguntar por `root`, sin `sudo`). La segunda ejecuta `id` **siendo** `root`. La tercera, **siendo** `ana`. Esta última es la herramienta estrella de toda la guía: **probar un permiso "disfrazado" de otra persona**.

> 💬 En tu Ubuntu, `sudo` te pedirá la contraseña la primera vez (`[sudo] contraseña para alumno:`) y la recordará unos minutos. En el libro esa línea no sale porque se ejecutó sin ella; en tu terminal sí.

0.5 · Monta el laboratorio 🧪

Para entender permisos hay que **ver cómo falla** algo cuando no se tiene permiso. Y para eso hacen falta **otras personas**. Vamos a crear un pequeño elenco:

Quién	Qué es	Grupos
🧑‍🎓 **alumno**	**tú**	`alumno`
👩 **ana**	compañera de equipo	`ana` , `equipo`
👨 **beto**	compañero de equipo y de soporte	`beto` , `equipo` , `soporte`
👩‍🔧 **carla**	solo de soporte	`carla` , `soporte`

(`equipo` y `soporte` son dos grupos de práctica.)

```
# 1. Descarga la guía (si no lo has hecho ya)
cd ~
git clone https://github.com/TheBillBull/Guias-y-Herramientas.git

# 2. Monta el laboratorio (te pedirá tu contraseña de sudo y confirmación)
bash ~/Guias-y-Herramientas/guias/permisos-linux-lpic1/laboratorio/preparar-laboratorio.sh

# 3. Entra
cd ~/lab-permisos
ls -l
```

Qué hace el script (y **solo** esto):

1. Crea la carpeta `~/lab-permisos` con tres scripts y unos ficheros de datos.
2. Crea con `sudo` los usuarios **ana, beto, carla** (con la marca `lab-permisos-lpic1` en su comentario) y los grupos **equipo, soporte**.
3. Ajusta tu carpeta personal para que esos usuarios puedan *atravesarla* (esto lo entenderás en el capítulo 3).

Quiero…	Comando
Dejarlo **como nuevo** (por si lo estropeas)	`bash …/preparar-laboratorio.sh` (se puede repetir)

Quiero…	Comando
Solo la carpeta, sin crear usuarios	`bash …/preparar-laboratorio.sh --sin-usuarios`
Borrarlo todo al terminar la guía	`bash …/preparar-laboratorio.sh --limpiar`

> 🛡️ `--limpiar` **solo** borra usuarios que lleven la marca `lab-permisos-lpic1`: nunca toca los tuyos. Y si ya existiera un usuario `ana` que no sea de la guía, el script **se niega** a tocarlo.

Comprobemos que el elenco existe (todas las salidas de este libro se generaron con este mismo laboratorio):

```
$ id ana
uid=1002(ana) gid=1002(ana) groups=1002(ana),1005(equipo)
$ id beto
uid=1003(beto) gid=1003(beto) groups=1003(beto),1005(equipo),1006(soporte)
$ id carla
uid=1004(carla) gid=1004(carla) groups=1004(carla),1006(soporte)
$ getent group equipo soporte
equipo:x:1005:ana,beto
soporte:x:1006:beto,carla
$ cd ~/lab-permisos
$ ls -l
total 16
-rwxrwxr-x 1 alumno alumno 1015 Oct  9 16:16 cerve.sh
drwxr-xr-x 2 alumno alumno 4096 Oct 16 17:08 datos
-rwxrwxr-x 1 alumno alumno  419 Oct 14 08:43 salario.sh
-rwxr-x--x 1 alumno alumno   93 Oct 13 08:07 saludos.sh
```

Fíjate en que `getent group equipo` lista los **miembros**: `ana,beto`. `alumno` **no** es de `equipo`. Eso nos dará mucho juego.

> 🔁 **Regla de oro:** si una práctica te deja el laboratorio hecho un lío, **vuelve a ejecutar el script** y empieza limpio. Cada capítulo del libro empieza con el laboratorio recién montado.

0.6 · Probar permisos "como otra persona": `sudo -u`

Esta es la técnica que usaremos **cientos de veces**:

```
sudo -u ana  COMANDO      →  ejecuta COMANDO siendo ana
```

Por ejemplo, ¿puede **carla** leer el fichero `secreto.txt` que está en tu laboratorio?

```
$ ls -l datos/secreto.txt
-rw------- 1 alumno alumno 44 Oct 12 09:30 datos/secreto.txt
$ cat datos/secreto.txt
La contraseña del wifi es: patata-frita-42
$ sudo -u carla cat datos/secreto.txt
cat: datos/secreto.txt: Permission denied
```

- Tú (`alumno`) **sí** puedes leerlo: eres su dueño y el fichero tiene permiso `rw-` para el dueño.
- `carla` **no**: `Permission denied`. (¡Aún no sabes leer ese `-rw-------`! Paciencia: es el capítulo 1.)

Cuatro **trucos de prueba** para esta guía:

Quiero saber…	Comando
¿Puede `ana` **leer** `f`?	`sudo -u ana cat f`
¿Puede `ana` **escribir** en `f`?	`sudo -u ana sh -c 'echo x >> f'`
¿Puede `ana` **ejecutar** `f`?	`sudo -u ana ./f`
¿Quién soy ahora?	`sudo -u ana id`

> ⚠️ **Ojo con el redirector `>` y `sudo`:** en `sudo echo hola > fichero` el `>` lo hace **tu** shell (sin privilegios) *antes* de que `sudo` entre en juego. Por eso hay que escribir `sudo sh -c 'echo hola > fichero'`. Lo verás en acción en el capítulo 9.

0.7 · Mensajes de error: inglés y español

Los comandos te hablan en el idioma de tu sistema. **Este libro se generó en inglés** (`LANG=C.UTF-8`); en un Ubuntu en español verás lo mismo, traducido:

En el libro (inglés)	En tu Ubuntu en español	Significa
`Permission denied`	`Permiso denegado`	No tienes permiso (el error de esta guía)
`Operation not permitted`	`Operación no permitida`	Ni siquiera siendo dueño/pidiéndolo así: acción reservada (p. ej. `chown` a otro)
`No such file or directory`	`No existe el archivo o el directorio`	La ruta no existe
`Not a directory`	`No es un directorio`	Usaste como carpeta algo que no lo es
`cannot access 'x'`	`no se puede acceder a 'x'`	Falló el acceso (a menudo, por permisos)
`command not found`	`orden no encontrada`	El comando no existe (o está fuera del `PATH`)

Y **los meses** de `ls -l` saldrán en minúscula (`oct` en vez de `Oct`), como en tu captura. Es solo cosmética.

0.8 · Cómo se hacen los ejercicios

Igual que en la guía de *regex y grep*:

Icono	Nivel
🟢	Fácil: aplicar lo que acabas de aprender
🟡	Medio: combinar dos o tres ideas
🔴	Difícil: hay que pensar un rato
•	Master: preguntas de examen y trampas

La **solución está escondida** en un desplegable. Técnica: **(1)** lee, **(2)** escribe tu intento, **(3)** ejecútalo en tu laboratorio, **(4)** si no sale, piensa por qué, **(5)** solo entonces mira.

Dos formatos de salida:

- **Bloque + *Resultado*:** un comando (o varios) y debajo lo que salió.
- **Transcripción** (`$ comando` y debajo su salida): cuando hay que ver *qué comando produjo qué*.

0.9 · 🏋 Ejercicios

🟢 Ejercicio 0.1 · ¿Quién eres?

Averigua **tu nombre de usuario**, **tu UID** y **tu GID** con un solo comando cada uno (pista: `id` tiene opciones `-u`, `-g`, `-un`, `-gn`).

> 💡 **Solución**

```
$ id -u
1001
$ id -g
1001
$ id -un
alumno
$ id -gn
alumno
```

`-u` = UID, `-g` = GID del grupo principal; con `-n` pide el **nombre** en vez del número.

🟢 Ejercicio 0.2 · Los grupos de beto

¿A qué grupos pertenece `beto`? Hazlo de dos maneras: con `id` y con `groups`.

💡 Solución

```
$ id beto
uid=1003(beto) gid=1003(beto) groups=1003(beto),1005(equipo),1006(soporte)
$ groups beto
beto : beto equipo soporte
$ id -Gn beto
beto equipo soporte
```

Las tres dicen lo mismo. `id -Gn` es la más cómoda para *scripts*: solo los **nombres** de **G**rupos, separados por espacios.

🟢 Ejercicio 0.3 · ¿Quién está en `equipo`?

Saca los **miembros** del grupo `equipo` y su GID.

💡 Solución

```
$ getent group equipo
equipo:x:1005:ana,beto
```

El GID es el tercer campo (1005 en este laboratorio) y los miembros, el cuarto (`ana,beto`).

🟡 Ejercicio 0.4 · ¿Es `alumno` de `equipo`?

Comprueba que **tú no** perteneces a `equipo`, pero `ana` sí. Hazlo con `id -Gn` para los dos.

💡 Solución

```
$ id -Gn alumno
alumno
$ id -Gn ana
ana equipo
```

`alumno` solo está en su grupo privado; `ana` está en `ana` y en `equipo`. Esto será decisivo en el capítulo 4.

🟡 Ejercicio 0.5 · La carpeta personal de ana

¿Cuál es el HOME de `ana` y qué shell usa? (Pista: el campo 6 y 7 de su línea en `/etc/passwd`.)

💡 Solución

```
$ getent passwd ana
ana:x:1002:1002:lab-permisos-lpic1:/home/ana:/bin/bash
$ getent passwd ana | cut -d: -f6,7
/home/ana:/bin/bash
```

`cut -d: -f6,7` corta por `:` y se queda con los campos 6 y 7.

🟡 Ejercicio 0.6 · Tú, "disfrazado"

Usando `sudo -u`, ejecuta `whoami` como `ana`, como `beto` y como `root`.

> 💡 **Solución**

```
$ sudo -u ana whoami
ana
$ sudo -u beto whoami
beto
$ sudo whoami
root
```

Sin `-u`, `sudo` usa `root`.

🔴 Ejercicio 0.7 · ¿Quién es el UID 0?

El sistema no mira nombres, mira números. Busca **qué usuario(s)** tienen UID 0 en `/etc/passwd` con un solo comando `awk` o `grep`.

> 💡 **Solución**

```
$ awk -F: '$3 == 0 {print $1}' /etc/passwd
root
$ grep -E '^[^:]*:[^:]*:0:' /etc/passwd
root:x:0:0:root:/root:/bin/bash
```

Debería salir **solo** `root`. Si en un sistema real ves **otro** usuario con UID 0, ¡alarma! Tiene los mismos poderes que `root` aunque se llame distinto. (Auditoría clásica de LPIC 110.1.)

• Ejercicio 0.8 · Pregunta de examen

¿Qué fichero guarda las contraseñas cifradas de los usuarios en un Linux moderno? a) `/etc/passwd` b) `/etc/shadow` c) `/etc/group` d) `/etc/login.defs`

> 💡 **Solución**

b) `/etc/shadow`**.** Y se puede comprobar con sus permisos: `-rw-r-----` para `root:shadow`, es decir, no legible por "otros".

```
$ ls -l /etc/passwd /etc/shadow
-rw-r--r-- 1 root root   3558 Oct 16 17:08 /etc/passwd
-rw-r----- 1 root shadow 1485 Oct 16 17:08 /etc/shadow
```

Capítulo 1 · Leer `ls -l` de un vistazo

> 🎯 **Objetivo:** que, en cuanto veas `-rwxrwxr-x 1 alumno alumno 1015 Oct 9 16:16 cerve.sh`, **entiendas cada carácter sin pensar**: qué tipo de cosa es, quién manda, quién puede qué, cuánto pesa y cuándo se tocó.
>
> 📘 **LPIC-1:** 104.5 (permisos y propiedad) · 103.3 (gestión de ficheros).
>
> 🧪 **Antes de empezar:** `cd ~/lab-permisos` (monta el laboratorio con el Capítulo 0 si no lo tienes).

1.1 · La línea completa, trozo a trozo

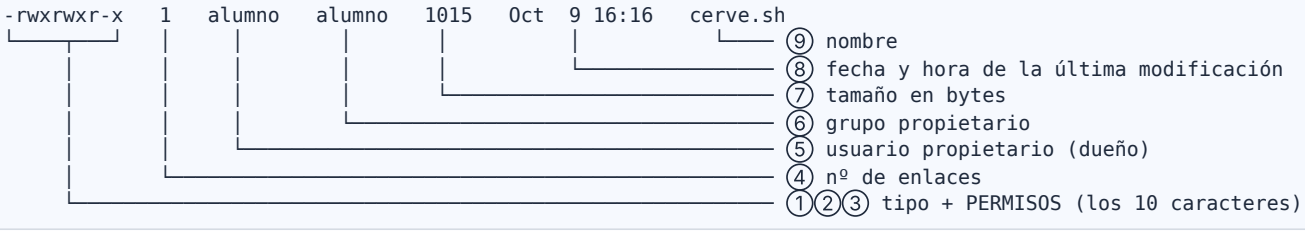
Este es el comando que más vas a escribir en esta guía: `ls -l` (*list, long*: "lista en formato largo").

```
ls -l cerve.sh
```

Resultado:

```
-rwxrwxr-x 1 alumno alumno 1015 Oct  9 16:16 cerve.sh
```

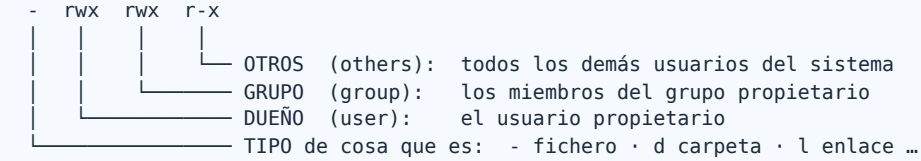
Una **sola línea**, con **9 datos** separados por espacios. Vamos a ponerles nombre, de izquierda a derecha:



#	Dato	En `cerve.sh`	Qué significa
①②③	**tipo + permisos**	`-rwxrwxr-x`	el corazón de esta guía (ahora mismo lo desmontamos)
④	enlaces	`1`	cuántos nombres tiene este mismo fichero (casi siempre 1)
⑤	dueño	`alumno`	el **usuario** propietario
⑥	grupo	`alumno`	el **grupo** propietario
⑦	tamaño	`1015`	bytes (≈ 1 KB de texto)
⑧	fecha	`Oct 9 16:16`	cuándo se **modificó el contenido** por última vez
⑨	nombre	`cerve.sh`	cómo se llama

> 💡 `total 16` es la primera línea cuando listas **varios** ficheros o una carpeta: son los bloques de disco que ocupan (en KB). Ignórala al leer permisos.

1.2 · Los 10 caracteres de los permisos



Primer carácter: el tipo. Los 9 siguientes son **tres trozos de tres**, y **siempre en el mismo orden**:

Trozo	Para quién	Letra en las órdenes
1º (caracteres 2-4)	el **dueño** (*user*)	`u`

Trozo	Para quién	Letra en las órdenes
2º (caracteres 5-7)	el **grupo** (*group*)	`g`
3º (caracteres 8-10)	**otros** (*others*)	`o`
los tres a la vez	todos (*all*)	`a`

Dentro de cada trozo, siempre `r w x` en ese orden, y una raya `-` significa "este permiso **NO** lo tiene":

Posición	Letra	Significa (en un fichero)
1ª	`r`	**r**ead: leer el contenido
2ª	`w`	**w**rite: modificar el contenido
3ª	`x`	e**x**ecute: ejecutarlo como programa

> 🧠 **Se lee la POSICIÓN, no la letra.** El trozo `r-x` es "puede leer, **no** puede escribir, puede ejecutar". La raya del medio no es un adorno: es la `w` que falta. Por eso `r-x`, `rw-`, `-wx`... son **todos distintos** y **todos de 3 caracteres**.

🔬 Tu línea, leída en voz alta

```
- rwx rwx r-x     1  alumno  alumno ...  cerve.sh

-    → es un fichero normal
rwx  → el DUEÑO (alumno) puede leer, escribir y ejecutar
rwx  → el GRUPO (alumno) puede leer, escribir y ejecutar
r-x  → LOS DEMÁS pueden leer y ejecutar, pero NO escribir
```

Con eso ya sabes leer **cualquier** línea. El resto es práctica.

📋 La receta de 5 segundos

1. **Primer carácter** → ¿qué es? (`-` fichero, `d` carpeta...)
2. **Tres trozos de tres** → dueño | grupo | otros.
3. En cada trozo, ¿`r`? ¿`w`? ¿`x`?
4. **Mira las columnas ⑤ y ⑥** (dueño y grupo): *¿soy yo? ¿es mi grupo?* → de ahí sale **qué trozo me toca** (el capítulo 4 lo explica con todo detalle).
5. Si hay `s`, `S`, `t` o `T` en vez de `x`, hay **permisos especiales** (capítulo 10).

1.3 · El primer carácter: ¿qué *cosa* es?

En Linux **casi todo es un fichero** (hasta los discos y las impresoras). El primer carácter dice de qué clase:

Carácter	Tipo	Ejemplo típico
`-`	**fichero normal** (texto, programa, foto...)	`cerve.sh`
`d`	**directorio** (carpeta)	`datos`
`l`	**enlace simbólico** (un "acceso directo")	`/bin -> usr/bin`
`c`	dispositivo de **caracteres** (teclado, `/dev/null`...)	`/dev/null`
`b`	dispositivo de **bloques** (discos)	`/dev/sda`
`p`	**tubería con nombre** (*pipe*, FIFO)	las crea `mkfifo`
`s`	**socket** (comunicación entre programas)	`/run/…/*.sock`

Los vemos uno a uno, **de verdad**:

```
$ ls -ld cerve.sh datos
-rwxrwxr-x 1 alumno alumno 1015 Oct  9 16:16 cerve.sh
drwxr-xr-x 2 alumno alumno 4096 Oct 16 17:08 datos
$ ln -s datos/notas.txt enlace.txt
$ ls -l enlace.txt
lrwxrwxrwx 1 alumno alumno 15 Oct 16 17:08 enlace.txt -> datos/notas.txt
$ ls -ld /bin
lrwxrwxrwx 1 root root 7 Apr 22  2024 /bin -> usr/bin
$ ls -l /dev/null
crw-rw-rw- 1 root root 1, 3 Oct 16 09:16 /dev/null
$ mkfifo mi-tuberia
$ ls -l mi-tuberia
prw-rw-r-- 1 alumno alumno 0 Oct 16 17:08 mi-tuberia
$ python3 -c "import socket; s = socket.socket(socket.AF_UNIX); s.bind('mi.sock')"
$ ls -l mi.sock
srwxrwxr-x 1 alumno alumno 0 Oct 16 17:08 mi.sock
```

Cosas que se ven en ese resultado:

- 🔗 `enlace.txt -> datos/notas.txt`: un enlace simbólico muestra **a dónde apunta**. Y fíjate: sus permisos son **siempre** `lrwxrwxrwx`. Es normal: *los que cuentan son los del fichero al que apunta*, no los del "acceso directo" (lo veremos en el capítulo 11).
- 📟 `/dev/null` no muestra un tamaño sino `1, 3`: es el número **mayor y menor** (*major, minor*) del dispositivo. Los ficheros `c` y `b` no tienen "tamaño".
- 💿 Un disco real sería así (el nombre varía según tu equipo: `/dev/sda`, `/dev/nvme0n1`, `/dev/vda`...):

```
brw-rw---- 1 root disk 8, 0 oct 16 09:16 /dev/sda
```

`b` de bloques, dueño `root`, grupo `disk`: los miembros de `disk` pueden leer y escribir el disco entero. ¡Por eso no te metas en ese grupo a la ligera!

> 🧰 `file cerve.sh` y `stat -c %F cerve.sh` te dicen el tipo con palabras: `Bourne-Again shell script` / `regular file`.

1.4 · La columna ④: el número de enlaces

Para un **fichero**, es cuántos *nombres* (enlaces duros) tiene. Casi siempre `1`.

Para un **directorio**, el número es **2 + el número de subdirectorios** que contiene (el `2` sale del propio nombre y de su `.`). Comprobémoslo:

```
$ mkdir carpeta
$ ls -ld carpeta
drwxrwxr-x 2 alumno alumno 4096 Oct 16 17:08 carpeta
$ mkdir carpeta/uno carpeta/dos
$ ls -ld carpeta
drwxrwxr-x 4 alumno alumno 4096 Oct 16 17:08 carpeta
```

Pasó de `2` a `4`: `2 + 2 subcarpetas`. Es un truco para saber cuántas subcarpetas tiene una carpeta sin entrar en ella.

1.5 · Columnas ⑤ y ⑥: dueño y grupo

Siempre dos nombres: **el usuario propietario** y **el grupo propietario**. Ambos se eligen *por separado*:

```
$ ls -l datos
total 16
-rw-r--r-- 1 alumno alumno 56 Oct 12 09:30 agenda.txt
-rw-r--r-- 1 alumno alumno 39 Oct 12 09:30 notas.txt
-rw------- 1 alumno alumno 44 Oct 12 09:30 secreto.txt
-rw-r--r-- 1 alumno alumno 37 Oct 12 09:30 ventas.csv
```

Aquí todo es de `alumno alumno`. Más adelante veremos ficheros `root shadow`, `root crontab`, `alumno equipo`...

Recuerda que Linux solo usa **números** y `ls` los traduce a nombres. Con `-n` verás los números reales:

```
$ ls -l cerve.sh
-rwxrwxr-x 1 alumno alumno 1015 Oct  9 16:16 cerve.sh
$ ls -n cerve.sh
-rwxrwxr-x 1 1001 1001 1015 Oct  9 16:16 cerve.sh
```

Si borras un usuario y sus ficheros siguen ahí, `ls -l` mostrará el **número** (ya no sabe traducirlo): son los llamados ficheros **huérfanos** (`find / -nouser` los busca).

1.6 · Columna ⑦: el tamaño

En **bytes**. Con `-h` (*human*) lo ves en K, M, G:

```
$ truncate -s 3M grande.bin
$ ls -l grande.bin
-rw-rw-r-- 1 alumno alumno 3145728 Oct 16 17:08 grande.bin
$ ls -lh grande.bin
-rw-rw-r-- 1 alumno alumno 3.0M Oct 16 17:08 grande.bin
$ ls -lS
total 20
-rw-rw-r-- 1 alumno alumno 3145728 Oct 16 17:08 grande.bin
drwxrwxr-x 4 alumno alumno    4096 Oct 16 17:08 carpeta
drwxr-xr-x 2 alumno alumno    4096 Oct 16 17:08 datos
-rwxrwxr-x 1 alumno alumno    1015 Oct  9 16:16 cerve.sh
-rwxrwxr-x 1 alumno alumno     419 Oct 14 08:43 salario.sh
-rwxr-x--x 1 alumno alumno      93 Oct 13 08:07 saludos.sh
lrwxrwxrwx 1 alumno alumno      15 Oct 16 17:08 enlace.txt -> datos/notas.txt
prw-rw-r-- 1 alumno alumno       0 Oct 16 17:08 mi-tuberia
srwxrwxr-x 1 alumno alumno       0 Oct 16 17:08 mi.sock
```

`ls -lS` ordena por tamaño (el mayor primero). Y ojo con los **directorios**: su "tamaño" (`4096`) es lo que ocupa la **lista de nombres**, no la suma de lo que contienen.

1.7 · Columna ⑧: la fecha (¿hora o año?)

`ls -l` enseña la fecha de la **última modificación** del contenido (*mtime*). Y tiene una regla curiosa:

Si el fichero es…	`ls -l` muestra…	Ejemplo
de hace **menos de 6 meses**	**mes día hora:minuto**	`Oct 9 16:16`
de hace **más de 6 meses** (o del futuro)	**mes día año**	`Mar 1 2024`

```
$ touch -d '2024-03-01 10:00' viejo.txt
$ ls -l viejo.txt cerve.sh
-rwxrwxr-x 1 alumno alumno 1015 Oct  9 16:16 cerve.sh
-rw-rw-r-- 1 alumno alumno    0 Mar  1  2024 viejo.txt
$ ls --full-time cerve.sh
-rwxrwxr-x 1 alumno alumno 1015 2026-10-09 16:16:00.000000000 +0000 cerve.sh
```

`--full-time` enseña la fecha **exacta** (con segundos y zona horaria). Y hay **tres** fechas por fichero:

Fecha	Se actualiza cuando…	Opción de `ls`
mtime (*modification*)	cambia el **contenido**	`ls -l` (por defecto)
atime (*access*)	se **lee** el fichero	`ls -lu`
ctime (*change*)	cambian los **metadatos** (¡**permisos**, dueño, nombre!)	`ls -lc`

> 🕵️ El **ctime** es una pista forense: `chmod` y `chown` **no** cambian el `mtime`, pero **sí** el `ctime`. Quien cambia permisos a escondidas no puede falsear el ctime fácilmente.

1.8 · `ls -d`: ver la ficha de la CARPETA, no su contenido

Cuando le das una **carpeta** a `ls -l`, te lista **lo de dentro**. Pero si lo que quieres son los **permisos de la carpeta misma**, necesitas `-d`:

```
$ ls -l datos
total 16
-rw-r--r-- 1 alumno alumno 56 Oct 12 09:30 agenda.txt
-rw-r--r-- 1 alumno alumno 39 Oct 12 09:30 notas.txt
-rw------- 1 alumno alumno 44 Oct 12 09:30 secreto.txt
-rw-r--r-- 1 alumno alumno 37 Oct 12 09:30 ventas.csv
$ ls -ld datos
drwxr-xr-x 2 alumno alumno 4096 Oct 16 17:08 datos
$ ls -ld .
drwxr-xr-x 4 alumno alumno 4096 Oct 16 17:08 .
$ ls -ld /tmp
drwxrwxrwt 14 root root 4096 Oct 16 17:08 /tmp
```

En tu captura tienes justo `ls -ld /tmp/`. Regla: `ls -ld` = **"la ficha de ESTA cosa"**. Si te olvidas de la `-d`, preguntas por los vecinos en vez de por la casa.

1.9 · `stat`: la ficha completa

`ls -l` es un resumen. `stat` te da **todo** de un fichero, incluidos los permisos **en octal y en letras a la vez**:

```
$ stat cerve.sh
  File: cerve.sh
  Size: 1015        Blocks: 8          IO Block: 4096   regular file
Device: 254,0   Inode: 599409      Links: 1
Access: (0775/-rwxrwxr-x)  Uid: ( 1001/  alumno)   Gid: ( 1001/  alumno)
Access: 2026-10-09 16:16:00.000000000 +0000
Modify: 2026-10-09 16:16:00.000000000 +0000
Change: 2026-10-16 17:08:01.950034144 +0000
 Birth: 2026-10-16 17:08:01.939853445 +0000
```

Lo que nos interesa es esta línea:

```
Access: (0775/-rwxrwxr-x)  Uid: ( 1001/  alumno)   Gid: ( 1001/  alumno)
         │    └── en letras      └── dueño: número/nombre      └── grupo: número/nombre
         └── en OCTAL (el 0 inicial son los bits especiales)
```

Y con `-c` (*custom format*) construyes tu propia ficha. Los códigos más útiles:

Código	Muestra	Ejemplo
`%a`	permisos en **octal**	`775`
`%A`	permisos en **letras**	`-rwxrwxr-x`
`%U`	**nombre** del dueño	`alumno`
`%G`	**nombre** del grupo	`alumno`
`%u` `%g`	**número** de dueño y de grupo	`1001`
`%s`	tamaño en bytes	`1015`
`%F`	tipo en palabras	`regular file`
`%n`	nombre	`cerve.sh`

```
$ stat -c '%a  %A  %U:%G  %s  %F  %n' cerve.sh saludos.sh salario.sh datos
775  -rwxrwxr-x  alumno:alumno  1015  regular file  cerve.sh
751  -rwxr-x--x  alumno:alumno  93  regular file  saludos.sh
775  -rwxrwxr-x  alumno:alumno  419  regular file  salario.sh
755  drwxr-xr-x  alumno:alumno  4096  directory  datos
```

> 🌟 `stat -c '%a %n' *` **es la forma más rápida de ver los permisos en octal de todos los ficheros de una carpeta.** Guárdalo.

1.10 · 🔍 Desmontando TU captura, línea a línea

Ahora ya sabes lo suficiente para leer **la captura que me enseñaste** (la de `~/bin`). Aquí va, **comando a comando**:

① `ls -l saludos.sh`

```
-rwxr-x--x 1 alumno alumno 93 oct 13 08:07 saludos.sh
```

- **Tipo:** `-` → fichero normal
- **Dueño (`alumno`):** `rwx` → leer, escribir, ejecutar
- **Grupo (`alumno`):** `r-x` → leer, ejecutar
- **Otros (el resto del mundo):** `--x` → ejecutar
- **En octal:** `751`

🧐 **Detalle interesante:** los *otros* tienen `--x` (solo ejecutar, sin leer). Para un **script** eso **no sirve de nada**: para lanzarlo, `bash` tiene que **leerlo**. Compruébalo con `carla` (que es "otros" para este fichero):

```
$ sudo -u carla ./saludos.sh
/bin/bash: /home/alumno/lab-permisos/saludos.sh: Permission denied
$ sudo -u ana ./saludos.sh
/bin/bash: /home/alumno/lab-permisos/saludos.sh: Permission denied
```

(`ana` tampoco puede: es "otros" para este fichero, porque `alumno` no es de `equipo` ni al revés.) Un binario compilado (no un script) sí se podría ejecutar con `--x`; lo comprobaremos en el capítulo 2.

② `ls -l salario.sh °` — el error de la captura

```
alumno@lpic1:~/bin$ ls -l salario.sh °
ls: no se puede acceder a '°': No existe el archivo o el directorio
-rwxrwxr-x 1 alumno alumno 419 oct 14 08:43 salario.sh
```

Se coló un `°` (el símbolo de grado) detrás del nombre. Para la terminal, cada palabra separada por un espacio es **otro argumento**, así que `ls` entendió: *"lista* `salario.sh` ***y también*** *un fichero llamado* `°` *"*. Como `°` no existe → error; y luego, aun así, lista `salario.sh`, que sí existe. Lo reproducimos en tu laboratorio:

```
$ ls -l salario.sh °
ls: cannot access '°': No such file or directory
-rwxrwxr-x 1 alumno alumno 419 Oct 14 08:43 salario.sh
```

> 🎓 Moraleja de LPIC: **un error de `ls` no siempre es un problema de permisos**. `No existe el archivo o el directorio` = la ruta no existe. Un problema de permisos diría `Permiso denegado`. Lee siempre **la frase** del error.

③ `ls -l salario.sh`

```
-rwxrwxr-x 1 alumno alumno 419 oct 14 08:43 salario.sh
```

- **Tipo:** `-` → fichero normal
- **Dueño (`alumno`):** `rwx` → leer, escribir, ejecutar
- **Grupo (`alumno`):** `rwx` → leer, escribir, ejecutar
- **Otros (el resto del mundo):** `r-x` → leer, ejecutar
- **En octal:** `775`

④ `ls -l /etc/shadow`

```
-rw-r----- 1 root shadow 2409 oct 16 11:44 /etc/shadow
```

- **Tipo:** `-` → fichero normal
- **Dueño (`root`):** `rw-` → leer, escribir
- **Grupo (`shadow`):** `r--` → leer
- **Otros (el resto del mundo):** `---` → **nada**
- **En octal:** `640`

👉 **Las contraseñas cifradas**: solo `root` las puede cambiar y solo `root` y los miembros del grupo `shadow` las pueden leer. **Cualquier otro usuario: nada.** (Lo vimos en el capítulo 0.)

⑤ `ls -l /etc/passwd` (con `r--` resaltado en tu captura)

```
-rw-r--r-- 1 root root 4187 oct 16 11:44 /etc/passwd
```

- **Tipo:** `-` → fichero normal
- **Dueño (`root`):** `rw-` → leer, escribir
- **Grupo (`root`):** `r--` → leer
- **Otros (el resto del mundo):** `r--` → leer
- **En octal:** `644`

El trozo que resaltaste, `r--`, es el de **otros**: *todo el mundo puede leer, nadie más que root puede modificar*. Y lo puedes probar:

```
$ ls -l /etc/passwd
-rw-r--r-- 1 root root 3558 Oct 16 17:08 /etc/passwd
$ sudo -u carla head -n 1 /etc/passwd
root:x:0:0:root:/root:/bin/bash
$ sudo -u carla sh -c 'echo prueba >> /etc/passwd'
sh: 1: cannot create /etc/passwd: Permission denied
```

`carla` lo lee sin problemas (`root:x:0:0…`), pero **no** puede escribir (`Permission denied`). (Intentar escribir en `/etc/passwd` sin ser `root` es inofensivo: justo para eso están los permisos.)

⑥ `ls -ld /tmp/`

```
drwxrwxrwt 18 root root 4096 oct 16 11:28 /tmp/
```

- **Tipo:** `d` → directorio (carpeta)
- **Dueño (`root`):** `rwx` → listar, crear/borrar/renombrar dentro, entrar
- **Grupo (`root`):** `rwx` → listar, crear/borrar/renombrar dentro, entrar
- **Otros (el resto del mundo):** `rwt` → listar, crear/borrar/renombrar dentro, entrar
- **En octal:** `1777`
- **Bits especiales: sticky** (bit pegajoso) (se explican en el capítulo 10)

🧐 **Dos cosas raras:**

- Lleva `-d` porque quieres la ficha de la carpeta, no su contenido.
- Termina en `t` en vez de `x`: es el **bit *sticky*** (capítulo 10). Hace que, aunque **todos** puedan escribir en `/tmp`, **cada uno solo pueda borrar sus propios ficheros**. Si no, cualquiera borraría los temporales de los demás.

⑦ `sudo ls -ld /var/spool/cron/crontabs/`

```
drwx-wx--T 2 root crontab 4096 oct  7 20:42 /var/spool/cron/crontabs/
```

- **Tipo:** `d` → directorio (carpeta)
- **Dueño (`root`):** `rwx` → listar, crear/borrar/renombrar dentro, entrar
- **Grupo (`crontab`):** `-wx` → crear/borrar/renombrar dentro, entrar

- **Otros (el resto del mundo):** `--T` → **nada**
- **En octal:** `1730`
- **Bits especiales: sticky** (bit pegajoso) (se explican en el capítulo 10)

Esta es **la joya** de tu captura. Es la carpeta donde `cron` guarda las **tareas programadas de cada usuario**. Descifremos *por qué* está así de rara:

Trozo	Valor	Por qué
dueño `root`	`rwx`	root administra la carpeta
grupo `crontab`	`-wx`	el programa `crontab` (que tiene el bit **SGID** `crontab`, mira `ls -l /usr/bin/crontab` → `-rwxr-sr-x root crontab`) puede **entrar y crear/modificar** ficheros, pero **no listar** los de los demás
otros	`---`	ningún usuario normal puede ni mirar
`T`	sticky sin `x`	los ficheros de un usuario **no los puede borrar otro**

Es un ejemplo perfecto de que `w` **y** `x` **sin** `r` en una carpeta tiene un uso real: un **buzón** (se pueden *echar* cosas, pero no *ver* qué hay). Lo entenderás del todo en el capítulo 3.

> 💬 Una nota sobre tu `sudo`: para ver **la ficha** de esa carpeta con `ls -ld` normalmente **no hace falta** `sudo` (solo hay que poder *atravesar* `/var/spool/cron`, que suele ser `drwxr-xr-x`). Sí haría falta `sudo` para **listar lo que hay dentro** (`ls /var/spool/cron/crontabs/`), porque ahí te falta el `r`. ¿Por qué? Porque `ls -ld` solo consulta la ficha de la carpeta; no la abre. ¡Capítulo 3!

1.11 · Otras formas útiles de `ls`

Comando	Para qué
`ls -l`	formato largo
`ls -la`	incluye ocultos (`.algo`)
`ls -ld carpeta`	la **ficha de la carpeta** misma
`ls -lh`	tamaños legibles (K, M, G)
`ls -ln`	dueño y grupo como **números**
`ls -lt`	ordena por fecha (el más reciente primero)
`ls -lS`	ordena por tamaño
`ls -lR`	recursivo (entra en subcarpetas)
`ls -li`	muestra el **número de inodo** (identifica el fichero de verdad)
`ls --full-time`	fecha exacta
`stat -c '%a %n' *`	**octal + nombre**

> 🔎 **¿Y ese `+` o `.` al final de los permisos** (`-rw-r--r--+` o `-rw-r--r--.`)? Un `+` avisa de que el fichero tiene **ACL** (permisos extra más allá de `rwx`: capítulo 12). Un `.` indica un contexto **SELinux** (Fedora/RHEL; en Ubuntu casi nunca lo verás).

1.12 · 🏋️ Ejercicios

Para cada ejercicio de "descifrar", **tapa la solución y contesta en voz alta**: tipo, permisos del dueño, del grupo y de otros.

🟢 Ejercicio 1.1 · Un clásico

```
-rw-r--r-- 1 alumno alumno 220 oct  9 16:16 .bash_logout
```

💡 Solución

- **Tipo:** `-` → fichero normal
- **Dueño (`alumno`):** `rw-` → leer, escribir
- **Grupo (`alumno`):** `r--` → leer
- **Otros (el resto del mundo):** `r--` → leer
- **En octal:** `644`

El permiso por defecto de un fichero de texto normal: **el dueño lo edita, los demás lo leen.**

🟢 Ejercicio 1.2 · Una carpeta

```
drwxr-xr-x 2 alumno alumno 4096 oct  9 16:16 Documentos
```

💡 Solución

- **Tipo:** `d` → directorio (carpeta)
- **Dueño (`alumno`):** `rwx` → listar, crear/borrar/renombrar dentro, entrar
- **Grupo (`alumno`):** `r-x` → listar, entrar
- **Otros (el resto del mundo):** `r-x` → listar, entrar
- **En octal:** `755`

Dueño: todo. Los demás pueden **ver qué hay y entrar**, pero no crear ni borrar dentro. (En el capítulo 3 verás qué significan *listar* y *entrar*.)

🟢 Ejercicio 1.3 · Solo para mí

```
-rwx------ 1 alumno alumno 512 oct 10 12:00 backup.sh
```

💡 Solución

- **Tipo:** `-` → fichero normal
- **Dueño (`alumno`):** `rwx` → leer, escribir, ejecutar
- **Grupo (`alumno`):** `---` → **nada**
- **Otros (el resto del mundo):** `---` → **nada**
- **En octal:** `700`

El típico script **privado**: nadie más puede ni leerlo.

🟢 Ejercicio 1.4 · Solo lectura para todos

```
-r--r--r-- 1 root root 2048 oct 10 12:00 leeme.txt
```

💡 Solución

- **Tipo:** `-` → fichero normal
- **Dueño (`root`):** `r--` → leer
- **Grupo (`root`):** `r--` → leer
- **Otros (el resto del mundo):** `r--` → leer
- **En octal:** `444`

Nadie puede escribirlo (ni siquiera el dueño, mientras no se haga `chmod u+w`). `root`, en cambio, se lo salta.

🟡 Ejercicio 1.5 · El fichero que no deja entrar a nadie

```
---------- 1 alumno alumno 0 oct 10 12:00 cerrado
```

¿Puede su dueño leerlo? ¿Puede cambiarle los permisos?

> 💡 **Solución**
>
> - **Tipo:** `-` → fichero normal
> - **Dueño (`alumno`):** `---` → **nada**
> - **Grupo (`alumno`):** `---` → **nada**
> - **Otros (el resto del mundo):** `---` → **nada**
> - **En octal:** `000`
>
> Su dueño **no puede leerlo** (`---`), pero **sí puede hacerle** `chmod`: cambiar los permisos es cosa del **dueño**, no depende de los permisos del fichero. Comprobémoslo:
>
> ```
> $ touch cerrado
> $ chmod 000 cerrado
> $ cat cerrado
> cat: cerrado: Permission denied
> $ chmod 600 cerrado
> $ cat cerrado
> ```
>
> (Lo explicamos en el capítulo 6.)

🟡 Ejercicio 1.6 · Grupo con superpoderes

```
-rw-rw-r-- 1 alumno equipo 340 oct 10 12:00 informe.odt
```

¿Quiénes pueden **modificarlo** en este laboratorio?

> 💡 **Solución**
>
> - **Tipo:** `-` → fichero normal
> - **Dueño (`alumno`):** `rw-` → leer, escribir
> - **Grupo (`equipo`):** `rw-` → leer, escribir
> - **Otros (el resto del mundo):** `r--` → leer
> - **En octal:** `664`
>
> Pueden escribir: `alumno` (dueño) y los miembros del grupo `equipo`, es decir, `ana` **y** `beto`. `carla` solo puede leerlo (es "otros").

🟡 Ejercicio 1.7 · Una carpeta de trabajo

```
drwxrwx--- 3 root soporte 4096 oct 10 12:00 tickets
```

¿Puede `ana` entrar? ¿Y `carla`? ¿Y `beto`?

> 💡 **Solución**
>
> - **Tipo:** `d` → directorio (carpeta)
> - **Dueño (`root`):** `rwx` → listar, crear/borrar/renombrar dentro, entrar
> - **Grupo (`soporte`):** `rwx` → listar, crear/borrar/renombrar dentro, entrar
> - **Otros (el resto del mundo):** `---` → **nada**
> - **En octal:** `770`
>
> El grupo `soporte` tiene acceso total y los demás, nada. En este laboratorio son de `soporte` `beto` **y** `carla` → pueden entrar. `ana` **no** (no es de `soporte`) y entraría como "otros": `---`.

🟡 Ejercicio 1.8 · Un dispositivo especial (real)

```
crw-rw-rw- 1 root root 1, 3 oct 16 09:16 /dev/null
```

> 💡 **Solución**
>
> - **Tipo:** `c` → dispositivo de caracteres
> - **Dueño (`root`):** `rw-` → leer, escribir
> - **Grupo (`root`):** `rw-` → leer, escribir
> - **Otros (el resto del mundo):** `rw-` → leer, escribir
> - **En octal:** `666`
>
> Un dispositivo de caracteres que **todo el mundo** puede leer y escribir: el "agujero negro" `/dev/null` donde se tira lo que no se quiere ver (`comando 2>/dev/null`).

🟡 Ejercicio 1.9 · ¿Qué es esto?

```
lrwxrwxrwx 1 root root 7 abr 22  2024 /bin -> usr/bin
```

¿Pueden **todos escribir** en `/bin`, entonces?

> 💡 **Solución**
>
> - **Tipo:** `l` → enlace simbólico
> - **Dueño (`root`):** `rwx` → leer, escribir, ejecutar
> - **Grupo (`root`):** `rwx` → leer, escribir, ejecutar
> - **Otros (el resto del mundo):** `rwx` → leer, escribir, ejecutar
> - **En octal:** `777`
>
> **No.** En un enlace simbólico los permisos `rwxrwxrwx` **no cuentan**: Linux los ignora y usa los del **destino** (`/usr/bin`, que es `drwxr-xr-x root root`). Es la trampa clásica: *"¡777 en `/bin`!"* → mentira, es un enlace.

🔴 Ejercicio 1.10 · De más abierto a más cerrado

Ordena de **más abierto a más cerrado** estos cuatro ficheros: `-rw-------`, `-rw-r--r--`, `-rw-rw-rw-`, `-rw-r-----`.

> 💡 **Solución**
>
> 1. `-rw-rw-rw-` (666): **todo el mundo** puede leer y escribir. 😱 Casi nunca es buena idea.
> 2. `-rw-r--r--` (644): todos leen, solo el dueño escribe.
> 3. `-rw-r-----` (640): dueño lee/escribe; el grupo lee; los demás, nada.
> 4. `-rw-------` (600): solo el dueño.
>
> Truco: cuenta las **letras** (`r`, `w`, `x`) en total; más letras = más abierto (aquí 6, 4, 3 y 2).

🔴 Ejercicio 1.11 · Buscar con `grep` los scripts del dueño

Lista (con `ls -l` y `grep`) solo los ficheros **normales** de tu laboratorio cuyo **dueño tenga permiso de ejecución**. (Pista: regex sobre el inicio de la línea: `^-` y luego la `x` en la 4ª posición.)

> 💡 **Solución**
>
> ```
> $ ls -l | grep '^-..x'
> -rwxrwxr-x 1 alumno alumno    1015 Oct  9 16:16 cerve.sh
> -rwxrwxr-x 1 alumno alumno     419 Oct 14 08:43 salario.sh
> -rwxr-x--x 1 alumno alumno      93 Oct 13 08:07 saludos.sh
> ```

`^-` = primer carácter `-` (fichero normal); `..` = `r` y `w` del dueño, cualquier cosa; `x` = ejecución del dueño. (Si quieres más, mira la guía de regex.)

🔴 Ejercicio 1.12 · Cuántas subcarpetas

Sin entrar en `carpeta`, ¿cuántas subcarpetas directas tiene? (Usa el número de enlaces.)

💡 Solución

```
$ ls -ld carpeta
drwxrwxr-x 4 alumno alumno 4096 Oct 16 17:08 carpeta
$ stat -c '%h' carpeta
4
```

`%h` es el número de enlaces: **4 − 2 = 2 subcarpetas**.

🔴 Ejercicio 1.13 · La fecha del cambio de permisos

Cambia los permisos de `datos/notas.txt` y comprueba que **el `mtime` no cambia pero el `ctime` sí**. (Pista: `stat -c '%y %z'`, o `ls -l` frente a `ls -lc`.)

💡 Solución

```
$ stat -c 'mtime=%y  ctime=%z' datos/notas.txt
mtime=2026-10-12 09:30:00.000000000 +0000  ctime=2026-10-16 17:08:01.963853445 +0000
$ sleep 2
$ chmod 640 datos/notas.txt
$ stat -c 'mtime=%y  ctime=%z' datos/notas.txt
mtime=2026-10-12 09:30:00.000000000 +0000  ctime=2026-10-16 17:08:04.547853385 +0000
```

`%y` es el *mtime* y `%z` el *ctime*. Tras el `chmod`, el **mtime sigue siendo el del 12 de octubre** (el contenido no se tocó) y el **ctime pasa a ser "ahora"**. (`ls -l` enseña el mtime; `ls -lc`, el ctime.)

• Ejercicio 1.14 · Pregunta de examen

Dado `-rwxr-sr-x 1 root crontab 39568 ... /usr/bin/crontab`*, ¿qué significa la* `s` *del grupo?* a) El fichero es un enlace simbólico b) Se ejecuta con los permisos del **grupo** propietario c) Solo se puede leer d) Es un directorio

💡 Solución

b) La `s` en la posición de la `x` del **grupo** es el bit **SGID** con ejecución: el programa corre con los privilegios del **grupo** dueño (aquí, `crontab`), no con los de quien lo lanza. Capítulo 10.

• Ejercicio 1.15 · Pregunta de examen 2

*¿Cuál de estas líneas corresponde a una **carpeta** en la que el dueño puede todo, el grupo solo entrar y listar, y los demás nada?* a) `-rwxr-x---` b) `drwxr-x---` c) `drwx--x---` d) `d---r-x---`

💡 Solución

b) `d` (carpeta) + `rwx` (dueño todo) + `r-x` (grupo: listar `r` y entrar `x`) + `---` (otros nada).

a) empieza por `-` (es un fichero). c) el grupo solo tiene `--x` (entrar sin listar). d) el dueño no tiene nada.

Capítulo 2 · `r`, `w` y `x` en los ficheros: leer, modificar, ejecutar

> 🎯 **Objetivo:** saber, para un **fichero**, qué permite exactamente cada letra: **leer** el contenido, **escribirlo** (y qué *no* permite escribir: ¡no es borrar!) y **ejecutarlo** (con la sorpresa de los scripts).
>
> 📘 **LPIC-1:** 104.5 (permisos de ficheros normales) · 103.1.
>
> 🧪 **Antes de empezar:** `cd ~/lab-permisos`. Cada práctica crea sus propios ficheros.

2.1 · La regla de oro

Para un **fichero normal**:

Letra	Nombre	Permite…	Sin ella…
`r`	**read**	**ver el contenido**: `cat`, `less`, `grep`, `cp` (copiarlo), abrirlo en un editor	`Permission denied` al leerlo
`w`	**write**	**cambiar el contenido**: `echo > f`, `>>`, `nano`, `truncate`…	`Permission denied` al escribir
`x`	**execute**	**lanzarlo como programa**: `./f`	`Permission denied` al ejecutarlo

Tres comprobaciones rápidas con un fichero de prueba. Creamos uno con permisos `rw-` para el dueño (`600`):

```
$ echo "Hola mundo" > prueba.txt
$ chmod 600 prueba.txt
$ ls -l prueba.txt
-rw------- 1 alumno alumno 11 Oct 16 17:08 prueba.txt
$ cat prueba.txt
Hola mundo
```

Ahora vamos **quitándole** permisos uno a uno y viendo qué se rompe.

2.2 · Quitar la lectura (`r`)

```
$ chmod u-r prueba.txt
$ ls -l prueba.txt
--w------- 1 alumno alumno 11 Oct 16 17:08 prueba.txt
$ cat prueba.txt
cat: prueba.txt: Permission denied
```

`-w-------`: con solo `w`, **no puedes leer**, pero **sí puedes escribir** (¡qué rareza!). Como en un buzón: se echa, no se mira.

```
$ echo "Otra línea" >> prueba.txt
$ cat prueba.txt
cat: prueba.txt: Permission denied
```

Pudimos **añadir** (`>>`), pero no leer ni siquiera lo que acabamos de escribir. Para comprobar que se escribió, hay que devolverle la `r`:

```
$ chmod u+r prueba.txt
$ cat prueba.txt
Hola mundo
Otra línea
```

> 🧪 **Un caso real** de "solo escritura": ficheros donde muchos usuarios *depositan* información sin poder verse entre sí.

2.3 · Quitar la escritura (`w`)

```
$ chmod u-w prueba.txt
$ ls -l prueba.txt
-r-------- 1 alumno alumno 23 Oct 16 17:08 prueba.txt
$ echo "no me dejará" >> prueba.txt
bash: prueba.txt: Permission denied
$ echo "tampoco" > prueba.txt
bash: prueba.txt: Permission denied
$ cat prueba.txt
Hola mundo
Otra línea
```

Con `r--` **se puede leer, no escribir**: ni para **añadir** (`>>`) ni para **sobrescribir** (`>`). Y recuerda: el fichero queda **intacto** (la segunda orden no llegó a vaciarlo).

¿`w` permite borrar el fichero?

No. Esta es **la gran confusión**. Mira:

```
$ ls -l prueba.txt
-r-------- 1 alumno alumno 23 Oct 16 17:08 prueba.txt
$ rm prueba.txt < /dev/null
$ ls prueba.txt
ls: cannot access 'prueba.txt': No such file or directory
```

Borramos un fichero **sin** permiso de escritura y *se borró*. ¿Por qué? Porque **borrar un fichero es modificar la carpeta que lo contiene** (le quitas su nombre de la lista), no tocar el fichero. Lo que cuenta es el permiso `w` **de la carpeta** (capítulo 3).

> 🗨️ Si pruebas `rm` en un fichero de solo lectura **directamente en la terminal**, `rm` te preguntará por precaución: `rm: remove write-protected regular file 'prueba.txt'?`. Contesta `y` y se borra igual. Es **solo un aviso amable**, no una prohibición. (Aquí lo hicimos con `< /dev/null` para no tener que contestar.)

2.4 · Quitar la ejecución (`x`) y los scripts

Creemos un script de verdad (un programa en texto que lanza `bash`):

```
$ cat > hola.sh <<'EOF'
#!/bin/bash
echo "¡Hola desde un script! Soy $(whoami)"
EOF
$ ls -l hola.sh
-rw-rw-r-- 1 alumno alumno 57 Oct 16 17:08 hola.sh
$ ./hola.sh
bash: ./hola.sh: Permission denied
```

Falla. El fichero se acaba de crear **sin** `x` (los ficheros nuevos **nunca** nacen ejecutables, es una medida de seguridad; ya veremos por qué en el capítulo 8 con `umask`). Hay que dárselo:

```
$ chmod u+x hola.sh
$ ls -l hola.sh
-rwxrw-r-- 1 alumno alumno 57 Oct 16 17:08 hola.sh
$ ./hola.sh
¡Hola desde un script! Soy alumno
```

La línea `#!/bin/bash` del principio (llamada ***shebang***) le dice al sistema: *"para ejecutar este fichero, usa el programa* `/bin/bash` *"*.

🎭 `./script` frente a `bash script`

Hay **dos formas** de lanzar un script, y piden permisos **distintos**:

Forma	Quién lo lee	Necesita en el fichero…
`./hola.sh`	el **sistema** ejecuta el fichero	`r` + `x`
`bash hola.sh`	**tú** le pasas el texto a `bash`	solo `r`

```
$ chmod u-x hola.sh
$ ./hola.sh
bash: ./hola.sh: Permission denied
$ bash hola.sh
¡Hola desde un script! Soy alumno
```

Sin `x`, `./hola.sh` falla, pero `bash hola.sh` **funciona**: `bash` es el programa que se ejecuta (y él sí tiene `x`); el script es solo un **texto que lee**. Esto explica algo muy típico del examen: *"un script sin `x` se puede ejecutar pasándoselo al intérprete."*

🎭 Y al revés: `x` sin `r`

```
$ chmod u=x hola.sh
$ ls -l hola.sh
---xrw-r-- 1 alumno alumno 57 Oct 16 17:08 hola.sh
$ ./hola.sh
/bin/bash: ./hola.sh: Permission denied
$ bash hola.sh
bash: hola.sh: Permission denied
```

Con solo `x`, el dueño **no puede ejecutarlo**, porque para un **script** hace falta que el intérprete **lea** el texto. **Un script necesita `r` y `x`.**

🎭 Pero un programa compilado (binario) es distinto

Un binario no es "texto que se lee": el sistema lo carga y lo ejecuta directamente. Para eso **basta** `x`. Probémoslo con una copia del programa `ls` (es un binario de verdad):

```
$ cp /usr/bin/ls mi-ls
$ chmod 111 mi-ls
$ ls -l mi-ls
---x--x--x 1 alumno alumno 142312 Oct 16 17:08 mi-ls
$ ./mi-ls datos
agenda.txt  notas.txt  secreto.txt  ventas.csv
$ cat mi-ls | head -c 4
cat: mi-ls: Permission denied
```

`---x--x--x` (`111`): **ejecutable por todos, ilegible para todos**, y funciona. Es la técnica que usan programas que se quieren *dejar usar* pero no *copiar ni inspeccionar*.

	¿Script?	¿Binario?
Ejecutar con `./f`	necesita `r` + `x`	basta `x`
Ejecutar con `intérprete f`	basta `r`	—

> 💡 **Por eso `saludos.sh` de tu captura** (`-rwxr-x--x`) no podían usarlo "otros": tenían `--x` pero es un **script** y les faltaba la `r` (capítulo 1).

2.5 · Permisos que se aplican *al abrir*, no *en cada instante*

Un detalle sutil: Linux comprueba los permisos **cuando abres el fichero**. Si un programa ya lo tiene abierto y alguien le quita el permiso, **ese programa sigue pudiendo** usarlo hasta que lo cierre. Pero el **siguiente** que lo intente, no. (Este detalle explica por qué los *cambios de permisos* no "echan" a quien ya estaba dentro.)

2.6 · ¿Y el dueño? El dueño siempre puede `chmod`

Aunque un fichero tenga `---------` (`000`), **su dueño puede cambiarle los permisos** (y así recuperar el acceso). No depende de `rwx`: depende de **ser el dueño** (o `root`):

```
$ echo "contenido" > cerrado.txt
$ chmod 000 cerrado.txt
$ ls -l cerrado.txt
---------- 1 alumno alumno 10 Oct 16 17:08 cerrado.txt
$ cat cerrado.txt
cat: cerrado.txt: Permission denied
$ chmod u+r cerrado.txt
$ cat cerrado.txt
contenido
```

2.7 · Las operaciones habituales y el permiso que piden

Una chuleta de lo que cada comando necesita **sobre el fichero** (lo de la carpeta, en el capítulo 3):

Quiero…	Comando	Necesito en el fichero
Ver el contenido	`cat f`, `less f`, `head f`	`r`
Copiarlo	`cp f g`	`r` (y `w` en la carpeta destino)
Añadir al final	`echo x >> f`	`w`
Vaciarlo / sobrescribirlo	`echo x > f`, `truncate -s0 f`	`w`
Editarlo con `nano` / `vim`	`nano f`	`r` (para verlo) y `w` (para guardar)
Ejecutarlo (`./f`)	`./f`	`x` (y `r` si es script)
Cambiar su fecha	`touch f`	`w` **o** ser el dueño
Cambiarle los permisos	`chmod …`	**ser el dueño** (o root)
Cambiarle el nombre	`mv f g`	nada en el fichero (¡carpeta!)
Borrarlo	`rm f`	nada en el fichero (¡carpeta!)
Contarlo (`wc`), buscarlo (`grep`)	`wc -l f`	`r`

> ⚠️ **Los editores a veces "burlan" el `w` del fichero.** `vim` y otros no modifican el fichero: escriben uno nuevo y lo renombran sobre el viejo. Si el **fichero** es de solo lectura pero la **carpeta** es tuya, `vim` (con `:w!`) puede acabar guardándolo igualmente. De nuevo: manda la carpeta. (Capítulo 11.)

2.8 · `root` y los permisos de los ficheros

`root` se salta **la lectura y la escritura** de cualquier fichero. Pero con la **ejecución** es más puntilloso: para ejecutar un fichero **necesita que tenga `x` para alguien** (si no tiene ninguna `x`, ni `root` lo ejecuta):

```
$ printf '#!/bin/sh\necho "soy un script"\n' > prueba-root.sh
$ chmod 000 prueba-root.sh
$ sudo cat prueba-root.sh
#!/bin/sh
echo "soy un script"
$ sudo sh -c 'echo "echo root escribe donde nadie puede" >> prueba-root.sh'
$ sudo cat prueba-root.sh
#!/bin/sh
echo "soy un script"
echo root escribe donde nadie puede
$ sudo ./prueba-root.sh
sudo: ./prueba-root.sh: command not found
$ chmod 100 prueba-root.sh
$ sudo ./prueba-root.sh
soy un script
root escribe donde nadie puede
```

Con `000`, `root` **lee y escribe** sin problemas, pero **no puede ejecutarlo**. Cuando *alguien* tiene `x` (aquí `100`), `root` sí.

2.9 · 🏋️ Ejercicios

🟢 Ejercicio 2.1 · Leer o no leer

Crea `nota.txt` con el texto `hola` y quítale **solo** el permiso de lectura al dueño. Comprueba que `cat` falla. Devuélvelo.

> 💡 **Solución**

```
$ echo hola > nota.txt
$ chmod u-r nota.txt
$ ls -l nota.txt
--w-rw-r-- 1 alumno alumno 5 Oct 16 17:08 nota.txt
$ cat nota.txt
cat: nota.txt: Permission denied
$ chmod u+r nota.txt
$ cat nota.txt
hola
```

🟢 Ejercicio 2.2 · Solo lectura

Deja `nota.txt` en `r--r--r--` (`444`) y prueba a escribirle. ¿Qué dice el error?

> 💡 **Solución**

```
$ chmod 444 nota.txt
$ ls -l nota.txt
-r--r--r-- 1 alumno alumno 5 Oct 16 17:08 nota.txt
$ echo "más" >> nota.txt
bash: nota.txt: Permission denied
```

`Permission denied` (en español: `Permiso denegado`).

🟢 Ejercicio 2.3 · Tu primer script ejecutable

Crea `saluda.sh` que imprima `Hola` y consigue ejecutarlo con `./saluda.sh`.

> 💡 **Solución**

```
$ printf '#!/bin/bash\necho Hola\n' > saluda.sh
$ ./saluda.sh
bash: ./saluda.sh: Permission denied
$ chmod +x saluda.sh
$ ./saluda.sh
Hola
```

El primer intento falla por falta de `x`; con `chmod +x` funciona.

🟡 Ejercicio 2.4 · Ejecutar sin `x`

Quítale la `x` a `saluda.sh` y ejecútalo **sin** `./`, sin volver a ponérsela.

> 💡 **Solución**

```
$ chmod -x saluda.sh
$ bash saluda.sh
Hola
```

`bash` lo **lee** (basta `r`) como un texto cualquiera.

🟡 Ejercicio 2.5 · ¿Puede carla leerlo?

`datos/agenda.txt` es `rw-r--r--`. ¿Puede `carla` leerlo? ¿Y modificarlo?

💡 Solución

```
$ ls -l datos/agenda.txt
-rw-r--r-- 1 alumno alumno 56 Oct 12 09:30 datos/agenda.txt
$ sudo -u carla cat datos/agenda.txt
Reunión el lunes a las 10:00
Llevar el informe impreso
$ sudo -u carla sh -c 'echo "hola" >> datos/agenda.txt'
sh: 1: cannot create datos/agenda.txt: Permission denied
```

Leer sí (`r--` para otros), escribir no.

🟡 Ejercicio 2.6 · `w` no es borrar

Pon un fichero a `444` y bórralo **sin** forzarlo. Explica por qué se puede.

💡 Solución

```
$ touch inofensivo.txt
$ chmod 444 inofensivo.txt
$ rm -v inofensivo.txt < /dev/null
removed 'inofensivo.txt'
```

Se borra porque **tienes `w` en la carpeta** donde está. Los permisos del fichero no importan para borrar.

🟡 Ejercicio 2.7 · Un programa que se deja usar pero no ver

Copia `/usr/bin/whoami` como `mi-whoami` con permisos `--x--x--x` y comprueba que `carla` puede **ejecutarlo** pero **no leerlo**.

💡 Solución

```
$ cp /usr/bin/whoami mi-whoami
$ chmod 111 mi-whoami
$ ls -l mi-whoami
---x--x--x 1 alumno alumno 35336 Oct 16 17:08 mi-whoami
$ sudo -u carla ./mi-whoami
carla
$ sudo -u carla cat mi-whoami | head -c 4
cat: mi-whoami: Permission denied
```

Funciona porque es un **binario**. `cat` falla por la falta de `r`.

🔴 Ejercicio 2.8 · El script que no se deja ejecutar

`./mi.sh` con permisos `---x------` (`100`) da `Permission denied` aunque sea del dueño. ¿Por qué? Arréglalo cambiando **un solo permiso**.

💡 Solución

```
$ printf '#!/bin/bash\necho funciona\n' > mi.sh
$ chmod 100 mi.sh
$ ls -l mi.sh
---x------ 1 alumno alumno 26 Oct 16 17:08 mi.sh
$ ./mi.sh
/bin/bash: ./mi.sh: Permission denied
$ chmod u+r mi.sh
$ ./mi.sh
funciona
```

Porque un script necesita **leerse**: `r` + `x`.

🔴 Ejercicio 2.9 · Solo escritura

Crea un fichero `buzon.txt` donde tú **puedas añadir** líneas pero **no verlas** (`-w-------`, es decir, `200`). Añade una línea y comprueba que no la puedes leer.

💡 Solución

```
$ touch buzon.txt
$ chmod 200 buzon.txt
$ ls -l buzon.txt
--w------- 1 alumno alumno 0 Oct 16 17:08 buzon.txt
$ echo "mensaje secreto" >> buzon.txt
$ cat buzon.txt
cat: buzon.txt: Permission denied
```

🔴 Ejercicio 2.10 · Quitar todo a todos menos a ti

Haz que solo tú puedas leer y escribir `secreto2.txt`, y demuestra con `ana` que no puede ni leerlo.

💡 Solución

```
$ echo "mi secreto" > secreto2.txt
$ chmod 600 secreto2.txt
$ ls -l secreto2.txt
-rw------- 1 alumno alumno 11 Oct 16 17:08 secreto2.txt
$ sudo -u ana cat secreto2.txt
cat: secreto2.txt: Permission denied
```

`600` = `rw-------`: dueño lee y escribe; grupo y otros, nada.

• Ejercicio 2.11 · Root y la `x`

`root` puede leer y escribir cualquier fichero. ¿Puede ejecutar un script con permisos `000`?

💡 Solución

No. Para ejecutar un fichero se necesita **al menos una** `x` en sus permisos (de alguien). Lo vimos en la sección 2.8: `sudo ./prueba-root.sh` con `000` falla, y con `100` funciona.

• Ejercicio 2.12 · Pregunta de examen

Un usuario tiene un script con permisos `-r-xr-xr-x`*. ¿Qué puede hacer su dueño?* a) Solo ejecutarlo b) Leerlo y ejecutarlo, pero no modificarlo c) Modificarlo, leerlo y ejecutarlo d) Nada

💡 Solución

b). `r-x` para el dueño: lee y ejecuta; **no** tiene `w`, así que no puede **modificar el contenido** (aunque sí podría darse `w` con `chmod u+w`, porque es el dueño).

• Ejercicio 2.13 · ¿Cuál es la diferencia?

Explica, con un ejemplo, la diferencia entre `./f` y `bash f` en cuanto a permisos.

💡 Solución

- `./f` → el **sistema** debe poder **ejecutar** `f`: necesita `x` (y `r`, si es un script con *shebang*).
- `bash f` → es `bash` el que se ejecuta; `f` solo es un **texto de entrada** que tiene que poder **leer**: basta `r`.

Por eso un script **sin** `x` se puede lanzar con `bash script.sh`, pero **no** con `./script.sh`.

Capítulo 3 · `r`, `w` y `x` en los directorios: el capítulo que casi todos entienden mal

> 🎯 **Objetivo:** entender qué significan **listar**, **entrar** y **crear/borrar** en una carpeta, por qué **para borrar un fichero manda la carpeta (no el fichero)**, y por qué hace falta `x` en **todas** las carpetas del camino.
>
> 📘 **LPIC-1:** 104.5 (permisos de directorios) · 103.3.
>
> 🧪 **Antes de empezar:** `cd ~/lab-permisos`.

3.1 · Una carpeta es una lista de nombres

Primero, un cambio de mentalidad. Una carpeta **no "contiene" ficheros** como una caja contiene objetos. Una carpeta es, en realidad, **un fichero especial que contiene una lista**: *"el nombre* `agenda.txt` *corresponde al fichero número 598620; el nombre* `notas.txt`*, al 598622..."*. Los ficheros viven **en otro sitio**; la carpeta solo **apunta** a ellos.

Imagina el **portal de un edificio** con un listado de vecinos y su número de puerta:

```
        ┌──────────── PORTAL (la carpeta) ────────────┐
        │  Listado:   "López"   → puerta 3º A         │
        │             "Ruiz"    → puerta 1º B         │
        └─────────────────────────────────────────────┘
```

Con esa imagen, los tres permisos de una **carpeta** cobran sentido:

Letra	En una carpeta permite…	En el portal sería…
`r`	**leer la lista de nombres** (`ls`)	**leer el listado** de vecinos en la pared
`x`	**entrar / "pasar por"** la carpeta (`cd`) y **acceder a algo por su nombre**	**cruzar el portal** y llamar a una puerta concreta
`w`	**modificar la lista**: crear, borrar o renombrar entradas	**cambiar el listado**: añadir o quitar vecinos

Y dos reglas que lo cambian todo:

> 🔑 **Regla 1: `w` sin `x` no sirve de nada.** Para cambiar la lista tienes primero que poder *pasar* por la carpeta.
>
> 🔑 **Regla 2: borrar o renombrar un fichero es modificar la lista de la carpeta**, así que lo que importa es `w` + `x` **de la carpeta**, no los permisos del fichero.

3.2 · Las 8 combinaciones, medidas de verdad

Hay 8 combinaciones de `rwx` posibles (`---`, `--x`, `-w-`, `-wx`, `r--`, `r-x`, `rw-`, `rwx`). En lugar de contártelas, las **medimos**. Este pequeño programa crea una carpeta `d` con un fichero `f` dentro, le pone cada una de las 8 combinaciones **al dueño** (tú) y prueba 6 operaciones. Un `✔` es que funcionó; un `✘`, que dio `Permission denied`:

```
ok() { "$@" >/dev/null 2>&1 && printf '  ✔      ' || printf '  ✘      '; }
printf 'dueño   ls d    ls -l d  cat d/f  cd d    touch   rm d/f\n'
for m in 0 1 2 3 4 5 6 7; do
  chmod 700 d 2>/dev/null; rm -rf d; mkdir d; echo hola > d/f; chmod ${m}00 d
  printf ' %s ' "$(stat -c %A d | cut -c2-4)"
  ok ls d; ok ls -l d; ok cat d/f; ok sh -c 'cd d'; ok touch d/nuevo; ok rm d/f
  echo
done
chmod 700 d; rm -rf d
```

Resultado:

```
dueño   ls d    ls -l d  cat d/f  cd d    touch   rm d/f
 ---    ✘        ✘        ✘        ✘       ✘       ✘
 --x    ✘        ✘        ✔        ✔       ✘       ✘
 -w-    ✘        ✘        ✘        ✘       ✘       ✘
 -wx    ✘        ✘        ✔        ✔       ✔       ✔
 r--    ✔        ✘        ✘        ✘       ✘       ✘
 r-x    ✔        ✔        ✔        ✔       ✘       ✘
 rw-    ✔        ✘        ✘        ✘       ✘       ✘
 rwx    ✔        ✔        ✔        ✔       ✔       ✔
```

Tienes el mapa completo de las carpetas en una tabla. Léela **por filas**:

Permisos	Qué se puede hacer	Cómo se llama en la práctica
`---`	nada	carpeta **cerrada**
`--x`	**entrar y acceder a lo que ya sabes cómo se llama** (`cat d/f` , `cd d`), pero **no ver qué hay**	**pasillo a oscuras**
`-w-`	**nada** (la `w` sin `x` no funciona)	inútil
`-wx`	entrar, **crear y borrar**, pero no ver la lista	**buzón** (se deposita sin ver qué hay dentro)
`r--`	ver los **nombres** (`ls`), pero ni `ls -l` , ni `cat` , ni `cd`	**listado sin acceso**
`r-x`	ver, entrar, leer ficheros; **no crear ni borrar**	carpeta **de solo lectura** (la más común: `755`)
`rw-`	ver nombres... y nada más (la `w` sin `x` no funciona)	casi inútil
`rwx`	**todo**	carpeta **normal** del dueño (`755` , `700` ...)

> ⭐ **La combinación útil por excelencia:** `r-x` (leer, entrar). Y la de trabajo: `rwx` .

3.3 · Casos raros, uno a uno

🔍 `r` sin `x` : ves los nombres pero nada más

```
$ mkdir ra
$ touch ra/a ra/b
$ chmod 400 ra
$ ls ra
a  b
$ ls -l ra
ls: cannot access 'ra/b': Permission denied
ls: cannot access 'ra/a': Permission denied
total 0
-????????? ? ? ? ?            ? a
-????????? ? ? ? ?            ? b
$ cat ra/a
cat: ra/a: Permission denied
$ chmod 700 ra
```

Con `r--` , `ls` **te enseña los nombres** (ese es el listado) pero, para mostrar tamaño, dueño y fecha (`ls -l`), tendría que **consultar cada fichero**, y eso requiere pasar por la carpeta (`x`). Por eso sale `?????????` y errores.

🔦 `x` sin `r`: pasillo a oscuras

```
$ mkdir oscuro
$ echo "contenido secreto" > oscuro/secreto.txt
$ chmod 100 oscuro
$ ls oscuro
ls: cannot open directory 'oscuro': Permission denied
$ cat oscuro/secreto.txt
contenido secreto
$ cat oscuro/otro.txt
cat: oscuro/otro.txt: No such file or directory
$ chmod 700 oscuro
```

Con `--x`: **no puedes listar**, pero **si sabes el nombre exacto** (`secreto.txt`) puedes leerlo. Y fíjate en un detalle: si pruebas con un nombre que **no existe** (`otro.txt`), el error cambia a `No such file or directory`. O sea: **puedes comprobar si un nombre existe**. Esa es la razón de que "esconder" cosas con `--x` solo funcione si el nombre es difícil de adivinar.

📮 `-wx`: el buzón (¡tu captura!)

Esto es justo lo que tiene `/var/spool/cron/crontabs`: `drwx-wx--T root crontab`. El grupo `crontab` puede **entrar y escribir, pero no listar**. Lo simulamos con el grupo `equipo` (`ana` y `beto` son de `equipo`; `carla` no):

```
$ mkdir buzon
$ sudo chgrp equipo buzon
$ chmod 730 buzon
$ ls -ld buzon
drwx-wx--- 2 alumno equipo 4096 Oct 16 17:08 buzon
$ sudo -u ana touch buzon/carta-de-ana.txt
$ sudo -u beto touch buzon/carta-de-beto.txt
$ sudo -u ana ls buzon
ls: cannot open directory 'buzon': Permission denied
$ sudo -u carla touch buzon/intruso.txt
touch: cannot touch 'buzon/intruso.txt': Permission denied
$ ls buzon
carta-de-ana.txt  carta-de-beto.txt
```

- 👩 `ana` **puede depositar** un fichero (`-wx` del grupo), pero **no ver qué hay** (`ls` falla).
- 👩‍🦰 `carla` es "otros" (`---`): ni entra.
- 🧑‍🎓 Tú (dueño, `rwx`) **ves todo**.

Y ahora la pregunta que debería preocuparte: `beto` **tiene** `w` **en la carpeta, ¿puede borrar la carta de** `ana`**?** ¡Sí!, porque borrar manda la carpeta. (Usamos `rm -f` para que no pregunte "¿borrar fichero protegido contra escritura?"; sin `-f` te lo preguntaría, pero tras decir `y` lo borraría igual.)

```
$ sudo -u beto rm -f buzon/carta-de-ana.txt
$ ls buzon
carta-de-beto.txt
```

Por eso `crontabs` lleva **además** el bit **sticky** (`T` al final): *"solo el dueño de un fichero (o de la carpeta) puede borrarlo"*. Se estudia en el capítulo 10.

3.4 · 🔑 Regla 2 en acción: borrar manda la carpeta

Dos casos que confirman la regla:

① Fichero "blindado" (`000`), carpeta mía → lo puedo borrar:

```
$ mkdir mia
$ echo "blindado" > mia/f
$ chmod 000 mia/f
$ ls -l mia/f
---------- 1 alumno alumno 9 Oct 16 17:08 mia/f
$ rm -f mia/f
$ ls mia
```

② Fichero "abierto de par en par" (`777`), carpeta sin `w` → no lo puedo borrar ni renombrar ni crear nada:

```
$ mkdir ajena
$ echo "hola" > ajena/f
$ chmod 777 ajena/f
$ chmod 555 ajena
$ ls -l ajena/f
-rwxrwxrwx 1 alumno alumno 5 Oct 16 17:08 ajena/f
$ rm ajena/f
rm: cannot remove 'ajena/f': Permission denied
$ mv ajena/f ajena/g
mv: cannot move 'ajena/f' to 'ajena/g': Permission denied
$ touch ajena/nuevo
touch: cannot touch 'ajena/nuevo': Permission denied
$ echo "pero el CONTENIDO sí lo puedo cambiar" >> ajena/f
$ cat ajena/f
hola
pero el CONTENIDO sí lo puedo cambiar
$ chmod 755 ajena
```

El contenido de `f` **sí** se puede modificar (el fichero tiene `w`); pero **su nombre y su existencia** dependen de la carpeta (`r-x`, sin `w`).

Quiero…	Lo decide…
Cambiar el **contenido** de `f`	los permisos de `f` (`w`) + `x` en la carpeta
Crear un fichero nuevo	`w` + `x` de la **carpeta**
Borrar `f`	`w` + `x` de la **carpeta** (+ *sticky*, capítulo 10)
Renombrar `f`	`w` + `x` de la **carpeta**
Mover `f` a otra carpeta	`w` + `x` en las **dos** carpetas

3.5 · La ruta entera: `x` en todas las carpetas del camino

Para llegar a `/home/alumno/lab-permisos/datos/secreto.txt`, el sistema atraviesa `/`, `/home`, `/home/alumno`, `/home/alumno/lab-permisos`, `/home/alumno/lab-permisos/datos`. Necesitas `x` **en TODAS** esas carpetas, y después el permiso que corresponda en el fichero.

La herramienta para verlo es `namei -l`: recorre la ruta y te enseña los permisos de **cada tramo**:

```
$ namei -l /home/alumno/lab-permisos/datos/agenda.txt
f: /home/alumno/lab-permisos/datos/agenda.txt
drwxr-xr-x root   root   /
drwxr-xr-x root   root   home
drwxr-x--x alumno alumno alumno
drwxr-xr-x alumno alumno lab-permisos
drwxr-xr-x alumno alumno datos
-rw-r--r-- alumno alumno agenda.txt
```

Fíjate: tu carpeta personal `alumno` aparece como `drwxr-x--x`. Ese `x` final (para *otros*) es el que el script del laboratorio le añadió con `chmod o+x ~`. Sin él, `ana`, `beto` y `carla` **no podrían atravesar tu carpeta personal**, y todos los experimentos con ellos fallarían **por esa razón y no por la del fichero**. ¡Comprobémoslo!

```
$ sudo -u ana cat /home/alumno/lab-permisos/datos/agenda.txt
Reunión el lunes a las 10:00
Llevar el informe impreso
$ chmod o-x ~
$ ls -ld ~
drwxr-x--- 3 alumno alumno 4096 Oct 16 17:08 /home/alumno
$ sudo -u ana cat /home/alumno/lab-permisos/datos/agenda.txt
cat: /home/alumno/lab-permisos/datos/agenda.txt: Permission denied
$ chmod o+x ~
$ sudo -u ana cat /home/alumno/lab-permisos/datos/agenda.txt
Reunión el lunes a las 10:00
Llevar el informe impreso
```

El **mismo fichero** (legible para todos: `rw-r--r--`) pasa a ser **inaccesible** cuando se cierra una carpeta **por encima**. Así es como Ubuntu protege las carpetas personales: `drwxr-x---` (`750`): ni los demás **entran**, ni **ven** qué hay.

🧪 Otro ejemplo, con un tramo cerrado en medio

```
$ mkdir -p a/b/c
$ echo hola > a/b/c/f
$ chmod 000 a/b
$ cat a/b/c/f
cat: a/b/c/f: Permission denied
$ namei -l a/b/c/f
f: a/b/c/f
drwxrwxr-x alumno alumno a
d--------- alumno alumno b
                          c - Permission denied
$ chmod 755 a/b
$ cat a/b/c/f
hola
```

`namei` te señala el culpable: la carpeta `b` con `d---------`. (Y como el error siempre dice solo `Permission denied`, `namei -l` **es tu mejor amigo para diagnosticar**. Lo recomiendo hasta el cansancio en el capítulo 13.)

3.6 · Una trampa curiosa: dentro de la carpeta, lo relativo sigue funcionando

Cuando estás **dentro** de una carpeta (`cd a/b/c`), tu terminal ya "está dentro" y las rutas **relativas** no vuelven a comprobar las carpetas de arriba. Mira:

```
$ mkdir -p zz/yy
$ echo "estoy dentro" > zz/yy/f
$ cd zz/yy
$ chmod 000 ..
$ cat f
estoy dentro
$ ls ..
ls: cannot open directory '..': Permission denied
$ cd ..
bash: cd: ..: Permission denied
$ cd ~/lab-permisos
$ chmod 755 zz
```

Mientras sigues dentro de `yy`, `cat f` funciona (ruta relativa) aunque `zz` esté cerrada con `000`. Pero `ls ..` y `cd ..` fallan. Es una **rareza** del sistema, importante para entender por qué a veces un proceso "ya dentro" sigue funcionando después de cambiar permisos de carpetas superiores.

3.7 · Cuadro de operaciones y permisos de carpeta

Quiero…	Comando	Permisos que necesito en la carpeta
Ver los **nombres**	`ls d`	`r`
Ver detalles (tamaños, dueños…)	`ls -l d`	`r` **y** `x`
Entrar	`cd d`	`x`
Leer / ejecutar un fichero de dentro	`cat d/f`, `./d/f`	`x` (+ permiso del fichero)
Crear un fichero	`touch d/n`, `mkdir d/n`	`w` **y** `x`
Borrar un fichero	`rm d/f`	`w` **y** `x`
Renombrar dentro	`mv d/a d/b`	`w` **y** `x`
Copiar *a* la carpeta	`cp f d/`	`w` **y** `x`
Mover *de una a otra*	`mv d1/f d2/`	`w` + `x` en **las dos**
Borrar la carpeta (vacía)	`rmdir d`	`w` + `x` en su **carpeta padre**
Borrar con contenido	`rm -r d`	`rwx` en `d` y en cada subcarpeta, + `w` + `x` en el padre
Buscar dentro	`find d`	`r` y `x` en cada nivel
Autocompletar con `Tab`		`r` (para listar candidatos)

> 🧠 **Resumen en una línea:** `r` = *ver la lista*; `x` = *pasar y tocar por nombre*; `w` = *modificar la lista* (siempre con `x`).

3.8 · Permisos de carpeta típicos

Octal	Letras	Uso típico
`755`	`drwxr-xr-x`	carpeta **pública** (otros ven y entran, pero no tocan)
`700`	`drwx------`	carpeta **privada** (tu `~/.ssh`)
`750`	`drwxr-x---`	solo el dueño y su grupo (**HOME en Ubuntu 24.04**)
`775`	`drwxrwxr-x`	de trabajo en equipo (dueño y grupo escriben)
`711`	`drwx--x--x`	"pasillo": otros **atraviesan** sin ver (típico de `~` si compartes `public_html`)
`1777`	`drwxrwxrwt`	pública y compartida **con *sticky*** (`/tmp`)
`2775`	`drwxrwsr-x`	de equipo con **SGID** (los ficheros heredan el grupo)
`1730`	`drwx-wx--T`	**buzón** protegido (`crontabs`)

3.9 · 🏋 Ejercicios

🟢 Ejercicio 3.1 · Cerrar una carpeta a los demás

Crea `privada` y haz que **solo tú** puedas entrar y listar. Demuestra con `ana` que no puede ni listarla.

> 💡 **Solución**

```
$ mkdir privada
$ chmod 700 privada
$ ls -ld privada
drwx------ 2 alumno alumno 4096 Oct 16 17:08 privada
$ sudo -u ana ls privada
ls: cannot open directory 'privada': Permission denied
```

`700` = `rwx------`.

🟢 Ejercicio 3.2 · Carpeta pública de solo lectura

Crea `publica` con un fichero dentro. Que cualquiera pueda **ver y entrar** pero **no crear**. Pruébalo con `carla`.

💡 Solución

```
$ mkdir publica
$ echo "hola a todos" > publica/leeme.txt
$ chmod 755 publica
$ sudo -u carla ls publica
leeme.txt
$ sudo -u carla cat publica/leeme.txt
hola a todos
$ sudo -u carla touch publica/mio.txt
touch: cannot touch 'publica/mio.txt': Permission denied
```

`r-x` para otros: leer y entrar, **no** crear.

🟢 Ejercicio 3.3 · ¿`ls` o `cd`?

¿Con qué permiso **de carpeta** funciona `cd` pero no `ls`?

💡 Solución

Con `--x` (solo ejecución). Lo probamos arriba en la carpeta `oscuro`: `cd` y `cat` por nombre funcionan, `ls` no.

🟡 Ejercicio 3.4 · Crear sin poder borrar

Prepara una carpeta `entrada` donde **tú** puedas todo y **el grupo** `equipo` pueda **entrar y listar** pero **no crear**.

💡 Solución

```
$ mkdir entrada
$ sudo chgrp equipo entrada
$ chmod 750 entrada
$ ls -ld entrada
drwxr-x--- 2 alumno equipo 4096 Oct 16 17:08 entrada
$ sudo -u ana ls entrada
$ sudo -u ana touch entrada/x
touch: cannot touch 'entrada/x': Permission denied
$ sudo -u carla ls entrada
ls: cannot open directory 'entrada': Permission denied
```

`750` = `rwxr-x---`: dueño todo; grupo `r-x` (listar y entrar); otros nada.

🟡 Ejercicio 3.5 · El buzón compartido

Crea `buzon2` para que `ana` y `beto` (grupo `equipo`) puedan **dejar ficheros** pero no listar la carpeta, y `carla` no pueda ni entrar.

💡 Solución

```
$ mkdir buzon2
$ sudo chgrp equipo buzon2
$ chmod 730 buzon2
$ ls -ld buzon2
drwx-wx--- 2 alumno equipo 4096 Oct 16 17:08 buzon2
$ sudo -u ana sh -c 'echo "de ana" > buzon2/ana.txt'
$ sudo -u ana ls buzon2
ls: cannot open directory 'buzon2': Permission denied
$ sudo -u carla sh -c 'echo x > buzon2/carla.txt'
sh: 1: cannot create buzon2/carla.txt: Permission denied
```

`730` = `rwx-wx---`.

🟡 Ejercicio 3.6 · ¿Quién puede borrar?

En una carpeta `rwxrwxrwx` hay un fichero tuyo `rw-------`. ¿Puede `carla` borrarlo?

💡 Solución

Sí, porque borrar manda la carpeta (`w` para todos). Comprobémoslo:

```
$ mkdir abierta
$ chmod 777 abierta
$ echo mio > abierta/mio.txt
$ chmod 600 abierta/mio.txt
$ sudo -u carla rm -f abierta/mio.txt
$ ls abierta
```

Por eso existe el bit *sticky* (capítulo 10): en `/tmp` (`1777`) esto **no** pasa.

🔴 Ejercicio 3.7 · El tramo cerrado

Con la estructura `p/q/r/dato.txt`, quita **un solo permiso** para que `ana` no pueda leer `dato.txt` aunque el fichero sea legible para todos, y demuéstralo con `namei -l`.

💡 Solución

```
$ mkdir -p p/q/r
$ echo "dato" > p/q/r/dato.txt
$ chmod 644 p/q/r/dato.txt
$ sudo -u ana cat p/q/r/dato.txt
dato
$ chmod o-x p/q
$ sudo -u ana cat /home/alumno/lab-permisos/p/q/r/dato.txt
cat: /home/alumno/lab-permisos/p/q/r/dato.txt: Permission denied
$ namei -l /home/alumno/lab-permisos/p/q/r/dato.txt
f: /home/alumno/lab-permisos/p/q/r/dato.txt
drwxr-xr-x root   root   /
drwxr-xr-x root   root   home
drwxr-x--x alumno alumno alumno
drwxr-xr-x alumno alumno lab-permisos
drwxrwxr-x alumno alumno p
drwxrwxr-- alumno alumno q
drwxrwxr-x alumno alumno r
-rw-r--r-- alumno alumno dato.txt
```

(Usamos la ruta **absoluta** porque con una relativa se "salta" el control en algunos tramos, como vimos en 3.6.) `namei` muestra `drwxrwxr--` en `q`: a "otros" les falta la `x`.

🔴 Ejercicio 3.8 · Borrar con contenido

`rm -rf` de una carpeta que contiene una subcarpeta **sin** permisos. ¿Qué pasa y cómo se arregla?

💡 Solución

```
$ mkdir -p gran/pequena
$ touch gran/pequena/f
$ chmod 000 gran/pequena
$ rm -rf gran
rm: cannot remove 'gran/pequena': Permission denied
$ chmod 700 gran/pequena
$ rm -rf gran
$ ls
a    ajena  buzon2    datos    mia     p        publica  salario.sh  zz
abierta  buzon  cerve.sh  entrada  oscuro  privada  ra       saludos.sh
```

`rm -r` no puede **entrar** en `pequena` (falta `x`) ni **listarla** (falta `r`), así que no puede vaciarla. Hay que devolverle permisos primero (`chmod -R u+rwX gran` es la forma rápida).

🔴 Ejercicio 3.9 · Mover una carpeta exige `w` sobre ella

Intenta mover una **carpeta** `origen/sub` a `destino/`, con `sub` en `r-x`. Compara con mover un **fichero** con permiso `r--`.

> 💡 **Solución**

```
$ mkdir -p origen/sub destino
$ touch origen/fichero
$ chmod 444 origen/fichero
$ chmod 555 origen/sub
$ mv origen/fichero destino/
$ mv origen/sub destino/
mv: cannot move 'origen/sub' to 'destino/sub': Permission denied
$ chmod 755 origen/sub
$ mv origen/sub destino/
$ ls destino
fichero  sub
```

El **fichero** se mueve aunque sea `r--` (solo se toca la lista de las dos carpetas). Pero mover una **carpeta a otro padre** exige `w` **sobre ella misma**: hay que actualizar su entrada `..` (que apunta a su nuevo padre).

• Ejercicio 3.10 · Lo de dentro sigue funcionando

Explica por qué, estando dentro de `zz/yy`, `cat f` sigue funcionando después de `chmod 000 ..`.

> 💡 **Solución**

Porque una ruta **relativa** (`f`) parte de tu directorio actual, que ya está "abierto" por tu terminal. Linux **no vuelve a comprobar** `x` **en las carpetas superiores** para esa búsqueda: solo en `yy` (que sí tienes) y en el fichero. Para subir (`..`) sí tendría que atravesar `zz` otra vez, y ahí falla.

• Ejercicio 3.11 · Preguntas de examen

A. *¿Qué permisos mínimos hacen falta sobre un directorio para poder ejecutar* `ls -l` *dentro de él?* a) `r` b) `x` c) `r` y `x` d) `w` y `x`

B. *¿Qué permisos mínimos hacen falta sobre un directorio para poder* ***borrar un fichero*** *que contiene (suponiendo que el fichero tiene permisos* `000`*)?* a) `r` b) `w` y `x` del directorio c) `w` del fichero d) `rwx` del fichero

C. *Un usuario tiene* `r--` *sobre un directorio. ¿Qué puede hacer?* a) Entrar b) Ver los **nombres** de lo que contiene c) Crear ficheros d) Nada

> 💡 **Solución**

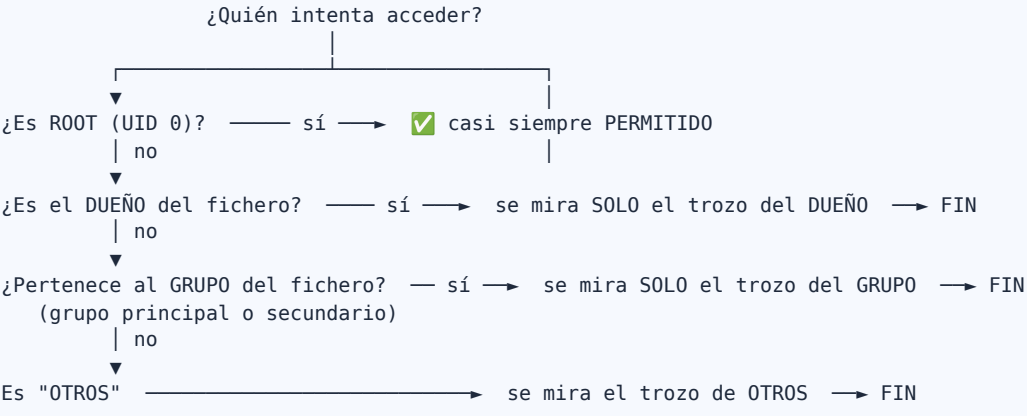
A → c). `r` para listar y `x` para consultar los metadatos de cada entrada. **B → b).** Borrar depende de `w` + `x` del **directorio**; los permisos del fichero no cuentan (salvo *sticky*). **C → b).** Con `r--` solo se ven los nombres (sin `x` no se puede ni `ls -l` ni acceder).

Capítulo 4 · ¿Quién decide qué? El algoritmo de los permisos

> 🎯 **Objetivo:** saber **qué trozo** de `rwxrwxrwx` se le aplica a cada persona, entender las **paradojas** (¡el dueño puede tener *menos* permisos que un extraño!) y manejar **grupos** (principal, secundarios, `newgrp`, `usermod -aG`).
>
> 📘 **LPIC-1:** 104.5 (usar el campo *group* para dar acceso a miembros) · 107.1 (grupos).
>
> 🧪 **Antes de empezar:** `cd ~/lab-permisos`.

4.1 · El algoritmo (cuando abres un fichero, Linux hace esto)

Cada vez que alguien intenta leer, escribir o ejecutar algo, el sistema se hace **tres preguntas, en este orden, y se queda con la primera que acierte**:



Tres consecuencias que **hay que grabarse a fuego**:

> 🔑 ① **Se mira UN solo trozo.** El que corresponda. Los otros **no se suman, no se heredan, no se consultan**.
>
> 🔑 ② **Gana el primero que acierta**, no el más permisivo. Si eres el **dueño**, se te aplica el trozo del dueño **aunque el del grupo o el de otros sea mejor**.
>
> 🔑 ③ `root` **va aparte**: no mira `rwx` (excepto la `x`, capítulo 2).

4.2 · Cuatro personas, un fichero: la matriz de acceso

Vamos a construir una herramienta muy útil: una función `quien` que, dado un fichero, te dice **qué pueden hacer** `alumno`, `ana`, `beto` y `carla` con él (leer ✔/✘, escribir, ejecutar). Es una "radiografía" de los permisos:

```
quien() {
  printf '%-8s leer  escribir  ejecutar\n' "$1"
  for u in alumno ana beto carla; do
    printf '%-8s' "$u"
    sudo -u $u test -r "$1" && printf '  ✔      ' || printf '  ✘      '
    sudo -u $u test -w "$1" && printf '  ✔       ' || printf '  ✘       '
    sudo -u $u test -x "$1" && printf '  ✔' || printf '  ✘'
    echo
  done
}
```

(`test -r f` pregunta "¿puede **quien lo ejecute** leer `f`?" y devuelve éxito o fallo.) Ahora creamos un fichero **nuestro** pero del **grupo** `equipo` con permisos `rw-r-----` (`640`):

```
$ cp saludos.sh f
$ sudo chgrp equipo f
$ chmod 640 f
$ ls -l f
-rw-r----- 1 alumno equipo 93 Oct 16 17:08 f
$ quien f
f        leer  escribir  ejecutar
alumno    ✔        ✔         ✘
ana       ✔        ✘         ✘
beto      ✔        ✘         ✘
carla     ✘        ✘         ✘
```

Léelo con la receta del capítulo 1 y el algoritmo del 4.1:

Persona	¿Dueño?	¿Del grupo `equipo`?	Trozo que se aplica	Resultado
🧑‍🎓 **alumno**	**sí**	—	dueño `rw-`	lee ✔ escribe ✔ ejecuta ✘
👩 **ana**	no	**sí**	grupo `r--`	lee ✔
👦 **beto**	no	**sí**	grupo `r--`	lee ✔
💁‍♀️ **carla**	no	no	otros `---`	nada

Para el mismo fichero, **cuatro personas distintas, cuatro resultados distintos**, y cada una usa un trozo diferente.

4.3 · 🤯 Las paradojas

Paradoja 1: el dueño con menos poder que un extraño

```
$ chmod 077 f
$ ls -l f
----rwxrwx 1 alumno equipo 93 Oct 16 17:08 f
$ quien f
f        leer  escribir  ejecutar
alumno    ✘        ✘         ✘
ana       ✔        ✔         ✔
beto      ✔        ✔         ✔
carla     ✔        ✔         ✔
```

`----rwxrwx`: el **dueño** (`alumno`) no tiene **nada**; el grupo y *otros* tienen **todo**. Como el dueño es el primero en comprobarse, `alumno` **se queda sin acceso** a su propio fichero (aunque, siendo dueño, puede rectificarlo con `chmod`). En cambio `carla` **(otros) puede leer, escribir y ejecutar**. ¡Es el mundo al revés!

> 😮 Es un error **de examen clásico**: *"el usuario es dueño y el grupo tiene más permisos, ¿qué se aplica?"* → **el del dueño**, aunque sea peor.

Paradoja 2: el grupo no ayuda al dueño

`ana` es **dueña** de un fichero del grupo `equipo` (donde ella está). El grupo tiene `rw-`, pero la dueña solo `r--`:

```
$ chmod 460 f
$ sudo chown ana:equipo f
$ ls -l f
-r--rw---- 1 ana equipo 93 Oct 16 17:08 f
$ quien f
f        leer  escribir  ejecutar
alumno    ✘        ✘         ✘
ana       ✔        ✘         ✘
beto      ✔        ✔         ✘
carla     ✘        ✘         ✘
```

> ⚠️ **Fíjate en el orden.** Primero `chmod` y **después** `chown`. En cuanto le regalas el fichero a `ana`, **dejas de ser el dueño** y ya no puedes cambiarle los permisos: `chmod: changing permissions of 'f': Operation not permitted`. Solo el **dueño** (o `root`) hace `chmod`.

- 👩 **ana**: dueña **y** del grupo, pero se aplica **el trozo del dueño** (`r--`): ¡no puede escribir!
- 👨 **beto**: del grupo → `rw-` → **sí** escribe.
- 🧑‍🎓 **alumno**: no es dueño ni del grupo → "otros" (`---`).

Es decir: **"ser del grupo" no suma al ser dueño**.

Paradoja 3: el grupo sin permiso, "otros" con permiso

```
$ sudo -u ana chmod 604 f
$ ls -l f
-rw----r-- 1 ana equipo 93 Oct 16 17:08 f
$ quien f
f        leer  escribir  ejecutar
alumno    ✔        ✘         ✘
ana       ✔        ✔         ✘
beto      ✘        ✘         ✘
carla     ✔        ✘         ✘
```

(El `chmod` lo hace `ana`, que ahora es la dueña.)

`rw----r--`: el **grupo** (`ana` es la dueña, pero `beto` es solo del grupo) tiene `---`, y *otros* tienen `r--`. `beto` **(grupo) no puede leer, pero** `carla` **(otros) sí.** Quien está en el grupo cae en el trozo del grupo (`---`) y **ya no mira "otros"**.

Permisos	Lo que ocurre
`---rwxrwx`	el dueño no puede nada; los demás, todo
`rw-rw-rw-`	todo el mundo lee y escribe (peligrosísimo)
`r--rw-rw-`	el dueño no puede escribir; su grupo y los otros sí
`rw----rw-`	el grupo no puede nada; los otros leen y escriben

4.4 · `root`: el que se salta las reglas

```
$ cp saludos.sh g
$ chmod 000 g
$ ls -l g
---------- 1 alumno alumno 93 Oct 16 17:08 g
$ cat g
cat: g: Permission denied
$ sudo cat g | head -n 1
#!/bin/bash
$ sudo sh -c 'echo "echo root escribe" >> g'
$ sudo tail -n 1 g
echo root escribe
```

`root` puede leer y escribir un fichero `000`. Por debajo, en Linux, esto es la **capacidad** `CAP_DAC_OVERRIDE` ("saltarse el control de acceso discrecional"). Y ojo, lo repasamos: para **ejecutar** hace falta al menos una `x` (capítulo 2).

> 🛑 **Moraleja de seguridad:** si un fichero necesita estar protegido **incluso de** `root`, los permisos no bastan (hay que usar atributos como `chattr +i`, capítulo 12).

4.5 · Grupos: el principal y los secundarios

Cada usuario tiene:

Qué	Para qué sirve	Se ve con
Grupo principal (uno)	es el grupo que se le pone a los ficheros **que crea**	`id -gn`
Grupos secundarios (0 o más)	dan **acceso** a lo que sea de esos grupos	`id -Gn`, `groups`

Fíjate en cómo decide el **grupo de un fichero nuevo**. Preparamos una carpeta en la que **todos** puedan crear (`777`):

```
$ mkdir comun
$ chmod 777 comun
$ sudo -u beto touch comun/de-beto-normal.txt
$ sudo -u beto sg equipo -c 'touch comun/de-beto-como-equipo.txt'
$ ls -l comun
total 0
-rw-r--r-- 1 beto equipo 0 Oct 16 17:08 de-beto-como-equipo.txt
-rw-r--r-- 1 beto beto   0 Oct 16 17:08 de-beto-normal.txt
```

`beto` tiene como principal `beto` y como secundarios `equipo` y `soporte`. Normalmente crea ficheros `beto:beto`. Con `sg equipo -c 'comando'` (o con `newgrp equipo`) cambia **temporalmente** su grupo principal a `equipo` y los ficheros nacen **del grupo** `equipo`.

Comando	Qué hace
`id` / `id -Gn`	ver tus grupos
`groups` / `groups ana`	listar los grupos (tuyos o de otro)
`newgrp equipo`	abre una **sub-shell** con `equipo` como grupo principal (`exit` para volver)
`sg equipo -c 'cmd'`	ejecuta **un comando** con `equipo` como grupo principal

4.6 · Añadir un usuario a un grupo: `usermod -aG` y el "cierra y abre sesión"

Un detalle que **cae en el examen** y descoloca a todos: cuando añades a alguien a un grupo, **no le afecta a lo que ya tiene abierto**. Los grupos de un proceso se fijan **al iniciar sesión**.

Lo probamos: `ana` crea un fichero del grupo `equipo` que solo el grupo puede leer. Tú (`alumno`) no eres de `equipo`:

```
$ sudo -u ana sh -c 'echo "informe del equipo" > comun/informe.txt'
$ sudo chgrp equipo comun/informe.txt
$ sudo chmod 640 comun/informe.txt
$ ls -l comun/informe.txt
-rw-r----- 1 ana equipo 19 Oct 16 17:08 comun/informe.txt
$ cat comun/informe.txt
cat: comun/informe.txt: Permission denied
```

Ahora **te añadimos** al grupo `equipo`:

```
$ sudo usermod -aG equipo alumno
$ id
uid=1001(alumno) gid=1001(alumno) groups=1001(alumno)
$ id alumno
uid=1001(alumno) gid=1001(alumno) groups=1001(alumno),1005(equipo)
$ cat comun/informe.txt
cat: comun/informe.txt: Permission denied
```

Fíjate en la trampa:

- `id alumno` (consulta la **base de datos**) → ya aparece `equipo`.
- `id` (a secas) → **todavía no**: tu terminal sigue con los grupos con los que arrancó.
- Y, por tanto, `cat` **sigue fallando** en esta terminal.

Cómo se ponen al día:

```
$ sudo -u alumno id
uid=1001(alumno) gid=1001(alumno) groups=1001(alumno),1005(equipo)
$ sg equipo -c 'cat comun/informe.txt'
informe del equipo
```

- `sudo -u alumno id` arranca un **proceso nuevo** y ya lleva el grupo.
- `sg equipo -c …` (o `newgrp equipo`) aplica el grupo en el momento.
- La solución "de verdad": **cerrar la sesión y volver a entrar** (o reiniciar la terminal/sesión gráfica).

Devolvemos el laboratorio a su estado:

```
$ sudo gpasswd -d alumno equipo
Removing user alumno from group equipo
$ id alumno
uid=1001(alumno) gid=1001(alumno) groups=1001(alumno)
```

⚠️ **¡Cuidado con `usermod -G` sin `-a`!** `usermod -G equipo alumno` **sustituye** *todos* los grupos secundarios por solo `equipo` (se pierden `sudo`, `adm`…). Hay que escribir `-aG` (*append* + *groups*) para **añadir**. Es un error que ha dejado a más de uno sin `sudo`.

4.7 · ¿Qué trozo me toca? La receta

Cuando mires un `ls -l` y te pregunten *"¿puede X hacer Y?"*:

1. **¿X es `root`?** → sí, salvo ejecutar sin ninguna `x`.
2. **¿X es el dueño** (3ª columna)? → mira **solo** el 1.er trozo.
3. Si no: **¿X está en el grupo** (4ª columna)? → mira **solo** el 2.º trozo. (Para saberlo: `id X`.)
4. Si no: → **3.er trozo** (otros).
5. ¿Y tiene X acceso a **todas las carpetas del camino** (`x`)? (capítulo 3)

```
-rw-r----- 1 alumno equipo ...  f
   |  |  └─ 3º trozo: OTROS ──── carla (ni dueño ni de equipo)
   |  └──── 2º trozo: GRUPO ──── ana y beto (de equipo)
   └─────── 1º trozo: DUEÑO ──── alumno
```

4.8 · 🏋️ Ejercicios

Para cada uno: **primero piensa quién puede qué**, luego comprueba con `quien`. (Si en tu terminal no tienes la función `quien`, vuelve a pegar su definición del apartado 4.2.)

🟢 Ejercicio 4.1 · Todo para todos

Fichero `abierto` con `rwxrwxrwx`. ¿Quién puede hacer qué?

💡 **Solución**

```
$ cp saludos.sh abierto
$ chmod 777 abierto
$ quien abierto
abierto  leer  escribir  ejecutar
alumno    ✔       ✔         ✔
ana       ✔       ✔         ✔
beto      ✔       ✔         ✔
carla     ✔       ✔         ✔
```

Todos pueden todo (777). (Casi nunca es buena idea.)

🟢 Ejercicio 4.2 · Solo el dueño

`secreto3` con `rw-------`.

💡 **Solución**

```
$ cp datos/notas.txt secreto3
$ chmod 600 secreto3
$ quien secreto3
secreto3 leer  escribir  ejecutar
alumno    ✔        ✔         ✘
ana       ✘        ✘         ✘
beto      ✘        ✘         ✘
carla     ✘        ✘         ✘
```

Solo `alumno` (dueño) lee y escribe. Nadie ejecuta (no hay `x`).

🟢 Ejercicio 4.3 · Lectura para el equipo

Haz que el grupo `equipo` **lea** `informe2` y solo tú lo **edites**. Los demás, nada.

💡 **Solución**

```
$ cp datos/notas.txt informe2
$ sudo chgrp equipo informe2
$ chmod 640 informe2
$ quien informe2
informe2 leer  escribir  ejecutar
alumno    ✔        ✔         ✘
ana       ✔        ✘         ✘
beto      ✔        ✘         ✘
carla     ✘        ✘         ✘
```

`640` = `rw-r-----`. `ana` y `beto` leen; `carla`, nada.

🟡 Ejercicio 4.4 · Edición para el equipo

Ahora que el grupo `equipo` también pueda **editar** `informe2`.

💡 **Solución**

```
$ chmod 660 informe2
$ ls -l informe2
-rw-rw---- 1 alumno equipo 39 Oct 16 17:08 informe2
$ quien informe2
informe2 leer  escribir  ejecutar
alumno    ✔        ✔         ✘
ana       ✔        ✔         ✘
beto      ✔        ✔         ✘
carla     ✘        ✘         ✘
```

`660` = `rw-rw----`. Ahora `ana` y `beto` escriben; `carla` sigue fuera.

🟡 Ejercicio 4.5 · El grupo "soporte"

Un fichero para el grupo `soporte`, legible por el grupo y por nadie más. ¿Quién puede leerlo?

💡 **Solución**

```
$ cp datos/notas.txt tickets.txt
$ sudo chgrp soporte tickets.txt
$ chmod 640 tickets.txt
$ quien tickets.txt
tickets.txt leer  escribir  ejecutar
alumno    ✔        ✔         ✘
ana       ✘        ✘         ✘
beto      ✔        ✘         ✘
carla     ✔        ✘         ✘
```

`soporte` = `beto` y `carla`. `ana` no. Fíjate en que `alumno` lo lee por ser **dueño**.

🟡 Ejercicio 4.6 · Dos grupos a la vez

`beto` pertenece a `equipo` y a `soporte`. Crea dos ficheros, uno de cada grupo, y demuestra que `beto` puede leer ambos pero `ana` solo el de `equipo`.

💡 Solución

```
$ echo "uno" > f-equipo; echo "dos" > f-soporte
$ sudo chgrp equipo f-equipo
$ sudo chgrp soporte f-soporte
$ chmod 640 f-equipo f-soporte
$ sudo -u beto cat f-equipo f-soporte
uno
dos
$ sudo -u ana cat f-equipo f-soporte
uno
cat: f-soporte: Permission denied
```

Un usuario puede tener **tantos grupos secundarios como haga falta**; cada fichero solo mira **su** grupo.

🔴 Ejercicio 4.7 · El error del dueño

Un compañero escribió `chmod 066 fichero` pensando que así lo "cerraba". ¿Qué ha logrado realmente?

💡 Solución

```
$ cp datos/notas.txt f066
$ chmod 066 f066
$ ls -l f066
----rw-rw- 1 alumno alumno 39 Oct 16 17:08 f066
$ quien f066
f066     leer  escribir  ejecutar
alumno    ✘        ✘         ✘
ana       ✔        ✔         ✘
beto      ✔        ✔         ✘
carla     ✔        ✔         ✘
```

`066` = `---rw-rw-`: **el dueño no puede nada** y **todos los demás leen y escriben**. Ha conseguido lo contrario de proteger.

🔴 Ejercicio 4.8 · Dueña sin permiso de escribir

`ana` es la dueña de `agenda-ana` (grupo `equipo`) con `r--rw----`. ¿Puede `ana` escribir? ¿Y `beto`? ¿Y cómo lo arreglaría ella?

💡 Solución

```
$ cp datos/agenda.txt agenda-ana
$ chmod 460 agenda-ana
$ sudo chown ana:equipo agenda-ana
$ quien agenda-ana
agenda-ana leer  escribir  ejecutar
alumno    ✘        ✘         ✘
ana       ✔        ✘         ✘
beto      ✔        ✔         ✘
carla     ✘        ✘         ✘
$ sudo -u ana chmod u+w agenda-ana
$ ls -l agenda-ana
-rw-rw---- 1 ana equipo 56 Oct 16 17:08 agenda-ana
```

`ana` (dueña) **no puede escribir** (se aplica `r--`), `beto` (grupo, `rw-`) sí. Pero como **dueña** puede cambiar los permisos: `chmod u+w` y listo.

🔴 Ejercicio 4.9 · El fichero invisible para su grupo

Haz un fichero del grupo `equipo` con `rw----r--`. ¿Puede `beto` (de `equipo`) leerlo? ¿Y `carla`?

💡 Solución

```
$ cp datos/notas.txt f604
$ sudo chgrp equipo f604
$ chmod 604 f604
$ quien f604
f604     leer  escribir  ejecutar
alumno    ✔        ✔         ✘
ana       ✘        ✘         ✘
beto      ✘        ✘         ✘
carla     ✔        ✘         ✘
```

`beto` y `ana` (de `equipo`) → trozo del grupo `---` → **no**. `carla` → otros `r--` → **sí**. "Ser del grupo" no te deja usar "otros".

🔴 Ejercicio 4.10 · Grupos nuevos y sesión vieja

Acabas de ejecutar `sudo usermod -aG docker pepe`, pero `pepe` (con la sesión abierta) sigue sin poder usar Docker. ¿Qué le dices?

💡 Solución

Los grupos se leen **al iniciar sesión**: debe **cerrar sesión y volver a entrar** (o usar `newgrp docker` para esa sub-shell). Se comprueba con `id` (no con `id pepe`, que lee la base de datos y ya lo mostraría).

• Ejercicio 4.11 · Quién puede qué (examen)

El fichero `informe.txt` *tiene* `-rw-r-----` *con propietario* `maria` *y grupo* `ventas`. `juan` *es miembro de* `ventas`, `luis` *no. ¿Cuál es correcta?* a) `juan` y `luis` pueden leerlo b) `juan` puede leerlo, `luis` no c) `luis` puede leerlo, `juan` no d) `juan` puede escribirlo

💡 Solución

b). `juan` → grupo `r--` → lee, no escribe. `luis` → otros `---` → nada.

• Ejercicio 4.12 · Quién puede qué (examen 2)

`-r--rw-rw-` *con dueño* `maria`. *¿Puede* `maria` *escribir?* a) Sí, como dueña b) Sí, porque tiene `rw-` el grupo y es del grupo c) **No** d) Solo si usa `sudo`

💡 Solución

c). Como dueña se le aplica **solo** el trozo del dueño (`r--`). Aunque esté en el grupo, **no se suma**. (Ella podría cambiarse los permisos con `chmod u+w`.) La d) es cierta técnicamente (root se salta los permisos), pero la respuesta que busca el examen es la c).

• Ejercicio 4.13 · Verdadero o falso

"Si un usuario es propietario de un fichero y también pertenece a su grupo, se aplican los permisos más amplios de ambos."

💡 Solución

Falso. Se aplican **solo** los del propietario (el primero que coincide). Lo hemos comprobado con `ana` en la paradoja 2.

Capítulo 5 · Octal, binario, decimal y letras: entender 755, 644 o 4755 de un vistazo

> 🎯 **Objetivo:** que `755`, `644`, `4755` y `rwxr-xr-x` dejen de ser códigos y pasen a ser la **misma cosa escrita de cuatro maneras**. Convertir entre octal, letras, binario, decimal y hexadecimal **de cabeza y rápido**.
>
> 📘 **LPIC-1:** 104.5 (`chmod` en modo octal) · 103.1.
>
> 🧪 **Antes de empezar:** `cd ~/lab-permisos`. Además vamos a usar la herramienta `permisos` de este repositorio (se instala en el apartado 5.9).

5.1 · La idea en una frase

> **Cada trozo `rwx` son 3 interruptores (encendido/apagado). Tres interruptores = un número del 0 al 7. Tres trozos = tres cifras.**

Y eso es *todo*. `755` no es un número mágico: son **tres cifras** (dueño, grupo, otros) y cada cifra **resume** un trozo `rwx`.

```
        dueño   grupo   otros
         rwx     r-x     r-x
          7       5       5       →   755
```

¿Por qué se usa el sistema **octal** (base 8, cifras 0-7)? Porque con **3 interruptores** hay exactamente **$2^3 = 8$** combinaciones posibles: del 0 al 7. **Una cifra octal = un trozo `rwx`.** No es casualidad: se eligió justo por eso.

5.2 · El valor de cada letra: 4, 2, 1

A cada interruptor se le asigna un **peso**:

```
            r     w     x
peso:       4     2     1
binario:  100   010   001
```

Para sacar la cifra de un trozo, **suma los pesos de las letras que están encendidas** (las que no son raya):

```
r - x   →   4 + 0 + 1 = 5
r w -   →   4 + 2 + 0 = 6
r w x   →   4 + 2 + 1 = 7
- - -   →   0 + 0 + 0 = 0
```

Los 8 casos posibles, completos:

Cifra	Binario (`r w x`)	Letras	Se suma	Significa
0	`000`	`---`	0	nada
1	`001`	`--x`	1	ejecutar
2	`010`	`-w-`	2	escribir
3	`011`	`-wx`	2 + 1	escribir + ejecutar
4	`100`	`r--`	4	leer
5	`101`	`r-x`	4 + 1	leer + ejecutar
6	`110`	`rw-`	4 + 2	leer + escribir
7	`111`	`rwx`	4 + 2 + 1	todo (leer + escribir + ejecutar)

🧠 **Memoriza solo estos cinco** y habrás aprendido el 95 %:

7	6	5	4	0
`rwx` todo	`rw-` leer y escribir	`r-x` leer y ejecutar	`r--` solo leer	`---` nada

(Los otros tres — `3`, `2`, `1` — son raros: escribir sin leer, o ejecutar sin leer.)

5.3 · De letras a número (receta de 3 pasos)

Tomamos `-rwxr-xr-x` (el típico de un programa):

1. **Quita el primer carácter** (el tipo): `rwxr-xr-x`.
2. **Corta de 3 en 3**: `rwx` · `r-x` · `r-x`.
3. **Suma los pesos de cada trozo** (4 la `r`, 2 la `w`, 1 la `x`): `4+2+1` · `4+0+1` · `4+0+1` = **7 · 5 · 5**.

→ `755`.

Ahora tú, con los de tu captura (los tienes resueltos, tápalos y comprueba):

`ls -l`	Trozos	Cuentas	Octal
`-rwxr-x--x` (`saludos.sh`)	`rwx` `r-x` `--x`	7 · 5 · 1	**751**
`-rwxrwxr-x` (`salario.sh`, `cerve.sh`)	`rwx` `rwx` `r-x`	7 · 7 · 5	**775**
`-rw-r-----` (`/etc/shadow`)	`rw-` `r--` `---`	6 · 4 · 0	**640**
`-rw-r--r--` (`/etc/passwd`)	`rw-` `r--` `r--`	6 · 4 · 4	**644**
`drwxrwxrwt` (`/tmp`)	`rwx` `rwx` `rwt`	**1** · 7 · 7 · 7	**1777**
`drwx-wx--T` (`crontabs`)	`rwx` `-wx` `--T`	**1** · 7 · 3 · 0	**1730**

(Las dos últimas llevan **cuatro cifras** por el bit *sticky*: se explica en el apartado 5.11 y en el capítulo 10.)

Compruébalo **con el sistema** (no te fíes de mí): `stat -c '%a'` imprime el octal.

```
$ stat -c '%a  %A  %n' saludos.sh salario.sh cerve.sh /etc/shadow /etc/passwd /tmp
751  -rwxr-x--x  saludos.sh
775  -rwxrwxr-x  salario.sh
775  -rwxrwxr-x  cerve.sh
640  -rw-r-----  /etc/shadow
644  -rw-r--r--  /etc/passwd
1777  drwxrwxrwt  /tmp
```

5.4 · De número a letras (receta de 3 pasos)

Tomamos `640`:

1. **Separa las cifras**: `6` `4` `0`.
2. **Cada cifra → su trozo** (con la tabla de arriba, o restando: `6 = 4+2 = rw-`): `rw-` `r--` `---`.
3. **Júntalos** y pon delante el tipo (`-` si es fichero): `-rw-r-----`.

Para **adivinar el trozo de una cifra** sin tabla, pregúntate de arriba abajo:

```
¿la cifra es ≥ 4?  → sí: pon "r" y réstale 4 · no: pon "-"
¿lo que queda es ≥ 2?  → sí: pon "w" y réstale 2 · no: pon "-"
¿lo que queda es ≥ 1?  → sí: pon "x" · no: pon "-"
```

Ejemplo con el **5**: `5 ≥ 4` → `r` (queda 1) · `1 < 2` → `-` · `1 ≥ 1` → `x` → `r-x`. ✔

5.5 · Atajos mentales para ir rápido

Atajo	Ejemplo
Los tres dígitos son siempre `dueño-grupo-otros`	`750` → dueño todo, grupo lee y entra, otros nada
`7` **= todo; resta lo que falte**	`7 − 2 = 5` → falta la `w` → `r-x`
Pares y trozos "pegados": `6` = `r` + `w`; `5` = `r` + `x`; `3` = `w` + `x`	
El `4` **solo lee; el** `2` **solo escribe; el** `1` **solo ejecuta**	`644` = dueño lee y escribe (6), los demás **solo leen** (4)
Los números "bonitos" que debes reconocer sin pensar	`644` `755` `600` `700` `640` `750` `664` `775` `777`
Un `x` **sin** `r` **es raro:** si ves `1`, `3` o `2` en un script, sospecha	`--x` en un script no sirve (capítulo 2)
Más cifra = más abierto	`600` < `640` < `644` < `664` < `666`

> 🏃 **Entrenamiento:** coge cualquier fichero de tu laboratorio, adivina su `chmod` en octal y comprueba con `stat -c '%a %n'`. En una semana lo harás sin pensar.

5.6 · El binario: cada letra es un interruptor

Un permiso son **9 interruptores** (3 trozos × 3). Cada letra presente es un `1` y cada raya es un `0`. Mira `rwxr-xr-x`:

```
letras:    r w x   r - x   r - x
binario:   1 1 1   1 0 1   1 0 1      =  111 101 101
octal:      7       5       5         =  755
```

Conversión directa, sin sumar: letra = `1`, raya = `0`, y cada grupo de 3 bits **es** una cifra octal (porque 3 bits = 0…7).

Letras	Binario	Octal
`rwx`	`111`	7
`r-x`	`101`	5
`r--`	`100`	4
`-w-`	`010`	2
`---`	`000`	0

Convertir binario ↔ octal ↔ letras

Si te dan `110100100`:

```
110 100 100      ← agrupa de 3 en 3, empezando por la DERECHA
 6   4   4       ← cada grupo a decimal (4+2, 4, 4)     →  644
rw- r-- r--      ← o directamente: 1=letra, 0=raya
```

5.7 · El decimal: por qué *no* es "755" (y cuándo lo vas a ver)

Aquí viene la trampa. Si el número `755` lo lees **en base 10** (como siempre), vale *setecientos cincuenta y cinco*. Pero **en octal** (base 8) `755` es:

```
7×8² + 5×8¹ + 5×8⁰  =  7×64 + 5×8 + 5  =  448 + 40 + 5  =  493 (en decimal)
```

O sea: `755` **(octal) =** `493` **(decimal)** y `chmod` **siempre** espera **octal**.

Lo comprobamos con la terminal (tres formas de convertir):

```
$ echo $(( 8#755 ))
493
$ printf '%d\n' 0755
493
$ printf '%o\n' 493
755
$ echo "obase=8; 493" | bc
755
$ echo "obase=2; 493" | bc
111101101
$ echo $(( 2#111101101 ))
493
$ printf '%o\n' $(( 2#111101101 ))
755
```

Quiero…	Truco
octal → decimal	`echo $(( 8#755 ))` o `printf '%d\n' 0755`
decimal → octal	`printf '%o\n' 493`
binario → octal	`printf '%o\n' $(( 2#111101101 ))`
decimal → binario	`echo "obase=2; 493" | bc`
octal → binario	`echo "obase=2; $(( 8#755 ))" | bc`
hexadecimal → octal	`printf '%o\n' 0x1ed`

> ⚠️ En `bc`, **pon** `obase` **antes que** `ibase` si cambias los dos (`obase=8; ibase=2; 111101101`), porque una vez cambiada la base de entrada, el `8` ya no se lee como *ocho*.

❌ `chmod 493` no existe

```
$ touch decimal.txt
$ chmod 493 decimal.txt
chmod: invalid mode: ‘493’
Try 'chmod --help' for more information.
$ chmod 755 decimal.txt
$ ls -l decimal.txt
-rwxr-xr-x 1 alumno alumno 0 Oct 16 17:09 decimal.txt
```

`493` no es un octal válido (¡tiene un 9!), por eso `chmod` protesta. `chmod` **solo entiende octal (o letras)**: si escribes `644`, lo lee como octal aunque tú pensaras en decimal.

💣 La trampa de los lenguajes de programación

En muchos lenguajes, una función como `os.chmod(fichero, modo)` espera un **número**. Si escribes `755` a secas, el lenguaje lo lee como **decimal** (755 = `1363` en octal), ¡y el fichero queda con unos permisos absurdos!

```
$ touch trampa.txt
$ python3 -c "import os; os.chmod('trampa.txt', 755)"
$ ls -l trampa.txt
--wxrw--wt 1 alumno alumno 0 Oct 16 17:09 trampa.txt
$ stat -c '%a' trampa.txt
1363
$ python3 -c "import os; os.chmod('trampa.txt', 0o755)"
$ ls -l trampa.txt
-rwxr-xr-x 1 alumno alumno 0 Oct 16 17:09 trampa.txt
```

La primera vez quedó `--wxrw--wt` (¡ni siquiera puedes leerlo!). La forma correcta en Python es `0o755` (el prefijo `0o` dice "esto es octal"). En Bash y C sirve con un **0 delante** (`0755`). Es un fallo clásico: "`chmod(path, 755)` *me ha dejado el fichero roto*".

Prefijo	Base	Ejemplo	Vale
(nada)	decimal	`755`	setecientos cincuenta y cinco

Prefijo	Base	Ejemplo	Vale
`0` / `0o`	**octal**	`0755` / `0o755`	493
`0b`	binario	`0b111101101`	493
`0x`	hexadecimal	`0x1ed`	493

5.8 · Una vuelta de tuerca: el modo "entero" y los tipos de fichero

Cuando un programa pide el modo **completo** de un fichero (como hace `stat`), el número incluye también **qué clase de cosa es** el fichero, en las cifras de más a la izquierda. Se ve con `%f`, que imprime el modo en **hexadecimal**:

```
$ ln -s datos/notas.txt enlace
$ stat -c '%f  %n' cerve.sh datos /dev/null enlace
81fd  cerve.sh
41ed  datos
21b6  /dev/null
a1ff  enlace
$ printf '%o\n' 0x81fd
100775
$ printf '%o\n' 0x41ed
40755
```

`0x81fd` es `100775` en octal: las **tres cifras de la derecha son los permisos** (`775`) y el `100` de delante es el tipo (**fichero normal**). Del mismo modo, `0x41ed` es `40755`: `040` = **directorio** y `755` = los permisos de `datos`. Esta es la tabla de tipos:

Octal (parte izquierda)	Tipo	`ls -l`
`100…`	fichero normal	`-`
`040…`	directorio	`d`
`120…`	enlace simbólico	`l`
`020…`	dispositivo de caracteres	`c`
`060…`	dispositivo de bloques	`b`
`010…`	tubería con nombre	`p`
`140…`	socket	`s`

(Con `chmod` **nunca** tocas esa parte: es el *tipo*, fijo desde que se crea.)

5.9 · 🛠️ La herramienta `permisos`: tu calculadora de bolsillo

En este repositorio hay un pequeño programa (un script de Bash, sin dependencias) que **traduce** permisos entre todas las representaciones. Instálalo así:

```
# Opción A: para todos los usuarios
sudo install -m 755 ~/Guias-y-Herramientas/guias/permisos-linux-lpic1/herramientas/permisos
/usr/local/bin/permisos

# Opción B: solo para ti (sin sudo)
mkdir -p ~/bin
install -m 755 ~/Guias-y-Herramientas/guias/permisos-linux-lpic1/herramientas/permisos ~/bin/permisos
# (en Ubuntu, ~/bin entra en el PATH al volver a abrir la terminal)
```

Se le puede dar **casi cualquier cosa**:

```
$ permisos 755
755

  Octal     755
  Letras    rwxr-xr-x      (en ls -l: -rwxr-xr-x)
  Binario   111 101 101
  Decimal   493
  Hex       0x1ed

  dueño   rwx  (7)  leer, escribir, ejecutar
  grupo   r-x  (5)  leer, ejecutar
  otros   r-x  (5)  leer, ejecutar

  chmod 755 FICHERO
  chmod u=rwx,g=rx,o=rx FICHERO
```

```
for x in rwxr-xr-x -rwxr-xr-x 0b111101101 d:493 0x1ed; do
  printf '%-14s → ' "$x"; permisos "$x" | grep Octal
done
```

Resultado:

```
rwxr-xr-x      →   Octal     755
-rwxr-xr-x     →   Octal     755
0b111101101    →   Octal     755
d:493          →   Octal     755
0x1ed          →   Octal     755
```

(Las cinco entradas dan el mismo `755`: son **cinco formas de escribir lo mismo**: letras, línea de `ls -l`, binario, decimal y hexadecimal.) También lee los **ficheros reales** y entiende las expresiones de `chmod`:

```
$ permisos saludos.sh
regular file: saludos.sh   (dueño alumno, grupo alumno)

  Octal     751
  Letras    rwxr-x--x      (en ls -l: -rwxr-x--x)
  Binario   111 101 001
  Decimal   489
  Hex       0x1e9

  dueño   rwx  (7)  leer, escribir, ejecutar
  grupo   r-x  (5)  leer, ejecutar
  otros   --x  (1)  ejecutar

  chmod 751 FICHERO
  chmod u=rwx,g=rx,o=x FICHERO
$ permisos 644 g+w,o-r
Base 644  +  chmod g+w,o-r

Resultado:

  Octal     660
  Letras    rw-rw----      (en ls -l: -rw-rw----)
  Binario   110 110 000
  Decimal   432
  Hex       0x1b0

  dueño   rw-  (6)  leer, escribir
  grupo   rw-  (6)  leer, escribir
  otros   ---  (0)  nada

  chmod 660 FICHERO
  chmod u=rw,g=rw,o= FICHERO
$ permisos u=rwx,go=rx
chmod u=rwx,go=rx   aplicado a 000

Resultado:

  Octal     755
  Letras    rwxr-xr-x      (en ls -l: -rwxr-xr-x)
  Binario   111 101 101
  Decimal   493
  Hex       0x1ed

  dueño   rwx  (7)  leer, escribir, ejecutar
  grupo   r-x  (5)  leer, ejecutar
  otros   r-x  (5)  leer, ejecutar

  chmod 755 FICHERO
  chmod u=rwx,g=rx,o=rx FICHERO
```

Y los modos de 4 cifras (los verás a fondo en el capítulo 10):

```
$ permisos 4755
4755

  Octal     4755
  Letras    rwsr-xr-x      (en ls -l: -rwsr-xr-x)
  Binario   100 111 101 101      (3 bits especiales + 3 × 3)
  Decimal   2541
  Hex       0x9ed

  dueño   rws  (7)  leer, escribir, ejecutar
  grupo   r-x  (5)  leer, ejecutar
  otros   r-x  (5)  leer, ejecutar

  Bits especiales:
    · SUID (4)    se ejecuta con los privilegios del DUEÑO

  chmod 4755 FICHERO
  chmod u=rwx,g=rx,o=rx,u+s FICHERO
```

> 🧪 Úsala para **comprobar tus propias conversiones**, no para sustituirlas: el examen LPIC-1 es sin ordenador.
>
> 🌐 También hay una **calculadora visual** en `herramientas/calculadora-chmod.html` (ábrela en el navegador: casillas para marcar `r` / `w` / `x` y ver el octal al instante).

5.10 · 📋 Los modos que debes reconocer al instante

Octal	Letras	Para qué se usa
`000`	`---------`	nada para nadie (carpeta/fichero "cerrado")
`400`	`r--------`	solo lectura, solo el dueño (claves privadas muy protegidas)
`440`	`r--r-----`	solo lectura para dueño y grupo (`/etc/sudoers` : `-r--r-----`)
`444`	`r--r--r--`	solo lectura para todos
`600`	`rw-------`	**privado**: lee y escribe solo el dueño (clave SSH privada, `secreto.txt`)
`640`	`rw-r-----`	el dueño edita, el grupo lee (`/etc/shadow` , logs)
`644`	`rw-r--r--`	**el clásico de los ficheros**: todos leen, solo el dueño escribe (`/etc/passwd`)
`660`	`rw-rw----`	dueño y grupo editan (trabajo en equipo)
`664`	`rw-rw-r--`	como 660 pero todos leen (el que dan `umask 002` y Ubuntu)
`666`	`rw-rw-rw-`	todos leen y escriben (¡evítalo!)
`700`	`rwx------`	**privado**: script o carpeta solo del dueño (`~/.ssh`)
`711`	`rwx--x--x`	el dueño todo; los demás solo "atraviesan" (`--x`)
`750`	`rwxr-x---`	dueño todo; grupo lee y entra (**HOME en Ubuntu 24.04**)
`751`	`rwxr-x--x`	como 750 pero otros pueden ejecutar (`saludos.sh` de tu captura)
`755`	`rwxr-xr-x`	**el clásico de programas y carpetas**: todos leen/entran, solo el dueño escribe
`770`	`rwxrwx---`	dueño y grupo todo; otros nada
`775`	`rwxrwxr-x`	dueño y grupo todo; otros leen/ejecutan (`cerve.sh` y `salario.sh` de tu captura)
`777`	`rwxrwxrwx`	todo para todos (¡casi siempre un error!)
`1777`	`rwxrwxrwt`	**sticky**: carpeta pública donde cada uno borra lo suyo (`/tmp`)
`1730`	`rwx-wx--T`	buzón: el grupo deposita, nadie lista (`/var/spool/cron/crontabs`)
`2755`	`rwxr-sr-x`	**SGID** en programa: corre con el grupo del dueño (`/usr/bin/crontab`)
`2775`	`rwxrwsr-x`	**SGID** en carpeta de equipo: lo nuevo hereda el grupo
`4755`	`rwsr-xr-x`	**SUID**: corre con los privilegios del dueño (`/usr/bin/passwd`)

(Las letras de esta tabla las genera un programa a partir del número, así que no hay erratas.)

5.11 · El cuarto dígito, de momento solo una pista

Hasta aquí, **tres cifras** (dueño, grupo, otros). A veces ves **cuatro**: `4755` , `2775` , `1777` . La cifra **de delante** son los **bits especiales** y vale igual de fácil:

```
  4        7  5  5
  │        └──┴──┴── dueño · grupo · otros   (como siempre)
  └─ especiales:  4 = SUID   2 = SGID   1 = sticky     (se suman igual que r w x)
```

Por ejemplo, `1777` = `1` (sticky) + `777` . En letras, esos bits **sustituyen a la** `x` : `rwxrwxrwt` . El capítulo 10 los explica uno a uno.

5.12 · Tabla de conversión completa (la de cabecera)

Octal	Letras	Binario	Decimal	Hex
`644`	`rw-r--r--`	`110 100 100`	420	`0x1a4`
`755`	`rwxr-xr-x`	`111 101 101`	493	`0x1ed`
`700`	`rwx------`	`111 000 000`	448	`0x1c0`
`777`	`rwxrwxrwx`	`111 111 111`	511	`0x1ff`
`640`	`rw-r-----`	`110 100 000`	416	`0x1a0`
`600`	`rw-------`	`110 000 000`	384	`0x180`
`4755`	`rwsr-xr-x`	`100 111 101 101`	2541	`0x9ed`
`1777`	`rwxrwxrwt`	`001 111 111 111`	1023	`0x3ff`
`2775`	`rwxrwsr-x`	`010 111 111 101`	1533	`0x5fd`

Cómo leer la columna *Decimal*: es el **valor numérico** del modo completo (lo que devuelven `os.stat()` o `$((8#…))`). **No** se usa para `chmod`, solo para entender qué guarda el sistema por dentro.

5.13 · 🏋️ Entrenamiento de velocidad

Haz cada serie **cronometrándote**. Objetivo: **menos de 3 segundos por pregunta**. Tapa las respuestas hasta terminar. (🌐 Para entrenar **sin límite**, con preguntas aleatorias y pasos de cálculo, abre `herramientas/entrenador-conversion.html` en el navegador.)

🟢 Serie 5.1 · Letras → octal

Letras → octal

1. `rwxr-xr-x`
2. `rw-r--r--`
3. `rwx------`
4. `r--r--r--`
5. `rw-rw----`
6. `rwxrwxrwx`
7. `rwxr-x---`
8. `rw-------`
9. `r-xr-xr-x`
10. `-wx-wx-wx`
11. `rwx--x--x`
12. `r--------`
13. `rw-rw-r--`
14. `---rwxrwx`
15. `r-x------`

> 💡 **Respuestas**
>
> 1. `rwxr-xr-x` → **755**
> 2. `rw-r--r--` → **644**
> 3. `rwx------` → **700**
> 4. `r--r--r--` → **444**
> 5. `rw-rw----` → **660**
> 6. `rwxrwxrwx` → **777**
> 7. `rwxr-x---` → **750**
> 8. `rw-------` → **600**

9. `r-xr-xr-x` → **555**
10. `-wx-wx-wx` → **333**
11. `rwx--x--x` → **711**
12. `r--------` → **400**
13. `rw-rw-r--` → **664**
14. `---rwxrwx` → **077**
15. `r-x------` → **500**

🟢 Serie 5.2 · Octal → letras

Octal → letras

1. `640`
2. `755`
3. `700`
4. `444`
5. `664`
6. `750`
7. `711`
8. `600`
9. `777`
10. `005`
11. `111`
12. `520`
13. `372`
14. `000`
15. `421`

> 💡 **Respuestas**
>
> 1. `640` → `rw-r-----`
> 2. `755` → `rwxr-xr-x`
> 3. `700` → `rwx------`
> 4. `444` → `r--r--r--`
> 5. `664` → `rw-rw-r--`
> 6. `750` → `rwxr-x---`
> 7. `711` → `rwx--x--x`
> 8. `600` → `rw-------`
> 9. `777` → `rwxrwxrwx`
> 10. `005` → `------r-x`
> 11. `111` → `--x--x--x`
> 12. `520` → `r-x-w----`
> 13. `372` → `-wxrwx-w-`
> 14. `000` → `---------`
> 15. `421` → `r---w---x`

🟡 Serie 5.3 · Binario → octal y letras

Binario → octal y letras

1. `111101101`
2. `110100100`

3. `111000000`
4. `100100100`
5. `110110000`
6. `111111111`
7. `101101101`
8. `011010001`

> 💡 **Respuestas**
>
> 1. `111101101` → octal **755** → `rwxr-xr-x`
> 2. `110100100` → octal **644** → `rw-r--r--`
> 3. `111000000` → octal **700** → `rwx------`
> 4. `100100100` → octal **444** → `r--r--r--`
> 5. `110110000` → octal **660** → `rw-rw----`
> 6. `111111111` → octal **777** → `rwxrwxrwx`
> 7. `101101101` → octal **555** → `r-xr-xr-x`
> 8. `011010001` → octal **321** → `-wx-w---x`

🟡 Serie 5.4 · Decimal → octal y letras

(Pista: para pasar un decimal a octal, **divide entre 8** y apunta los restos de abajo arriba: `493 ÷ 8 = 61 resto 5`, `61 ÷ 8 = 7 resto 5`, `7` → **755**.)

Decimal → octal y letras

1. `493`
2. `420`
3. `448`
4. `511`
5. `384`
6. `292`
7. `488`
8. `438`

> 💡 **Respuestas**
>
> 1. `493` → octal **755** → `rwxr-xr-x`
> 2. `420` → octal **644** → `rw-r--r--`
> 3. `448` → octal **700** → `rwx------`
> 4. `511` → octal **777** → `rwxrwxrwx`
> 5. `384` → octal **600** → `rw-------`
> 6. `292` → octal **444** → `r--r--r--`
> 7. `488` → octal **750** → `rwxr-x---`
> 8. `438` → octal **666** → `rw-rw-rw-`

🔴 Serie 5.5 · Cuatro cifras → letras

Cuatro cifras → letras

1. `4755`
2. `2755`
3. `1777`
4. `6755`
5. `1730`
6. `2770`

7. `4711`
8. `7777`
9. `2750`
10. `4000`

> 💡 **Respuestas**
>
> 1. `4755` → `rwsr-xr-x`
> 2. `2755` → `rwxr-sr-x`
> 3. `1777` → `rwxrwxrwt`
> 4. `6755` → `rwsr-sr-x`
> 5. `1730` → `rwx-wx--T`
> 6. `2770` → `rwxrws---`
> 7. `4711` → `rws--x--x`
> 8. `7777` → `rwsrwsrwt`
> 9. `2750` → `rwxr-s---`
> 10. `4000` → `--S------`

🔴 Serie 5.6 · Letras con `s`/`t` → octal de 4 cifras

Letras con `s`, `S`, `t`, `T` → octal de 4 cifras

1. `rwsr-xr-x`
2. `rwxr-sr-x`
3. `rwxrwxrwt`
4. `rwx-wx--T`
5. `rwxrws---`
6. `rwSr--r--`
7. `rwxrwsr-x`
8. `rwx--S---`

> 💡 **Respuestas**
>
> 1. `rwsr-xr-x` → **4755**
> 2. `rwxr-sr-x` → **2755**
> 3. `rwxrwxrwt` → **1777**
> 4. `rwx-wx--T` → **1730**
> 5. `rwxrws---` → **2770**
> 6. `rwSr--r--` → **4644**
> 7. `rwxrwsr-x` → **2775**
> 8. `rwx--S---` → **2700**

5.14 · 🏋 Ejercicios de pensar

🟢 Ejercicio 5.7 · El número justo

¿Qué octal da: **dueño** lee y escribe, **grupo** solo lee, **otros** nada?

> 💡 **Solución**
>
> `rw-` `r--` `---` → 6 · 4 · 0 → **640**. Compruébalo:

```
$ permisos u=rw,g=r,o=
chmod u=rw,g=r,o=   aplicado a 000

Resultado:

  Octal     640
  Letras    rw-r-----      (en ls -l: -rw-r-----)
  Binario   110 100 000
  Decimal   416
  Hex       0x1a0

  dueño   rw-  (6)  leer, escribir
  grupo   r--  (4)  leer
  otros   ---  (0)  nada

  chmod 640 FICHERO
  chmod u=rw,g=r,o= FICHERO
```

🟢 Ejercicio 5.8 · ¿Cuántos modos hay?

¿Cuántas combinaciones **distintas** de `rwxrwxrwx` existen? ¿Y contando los 3 bits especiales?

💡 Solución

9 interruptores → 2^9 = **512** (de `000` a `777`). Con los 3 especiales: 12 interruptores → 2^{12} = **4096** (de `0000` a `7777`).

🟡 Ejercicio 5.9 · 755 frente a 775

¿En qué se diferencian exactamente `755` y `775`?

💡 Solución

El **grupo**: `r-x` (5) frente a `rwx` (7). Con `775` los miembros del grupo pueden además **crear y borrar** (en carpetas) o **modificar** (en ficheros). Es la diferencia entre "trabajo en equipo" y "solo el dueño edita".

```
$ permisos 755 | grep -E 'dueño|grupo|otros'
  dueño   rwx  (7)  leer, escribir, ejecutar
  grupo   r-x  (5)  leer, ejecutar
  otros   r-x  (5)  leer, ejecutar
$ permisos 775 | grep -E 'dueño|grupo|otros'
  dueño   rwx  (7)  leer, escribir, ejecutar
  grupo   rwx  (7)  leer, escribir, ejecutar
  otros   r-x  (5)  leer, ejecutar
```

🟡 Ejercicio 5.10 · ¿Cuál no es válido?

¿Cuál de estos **no** es un modo octal válido? `0644` · `1777` · `888` · `4755`

💡 Solución

`888`: el 8 no existe en octal (las cifras son 0-7). `0644` es válido (el `0` de delante son "sin bits especiales").

```
$ touch valido.txt
$ chmod 888 valido.txt
chmod: invalid mode: ‘888’
Try 'chmod --help' for more information.
$ chmod 0644 valido.txt
$ ls -l valido.txt
-rw-r--r-- 1 alumno alumno 0 Oct 16 17:09 valido.txt
```

🟡 Ejercicio 5.11 · Sin recalcular

Tienes un fichero `644` y quieres darle escritura al grupo. ¿Qué nuevo octal sale? ¿Y qué `chmod` cómodo lo hace sin pensar en números?

💡 Solución

El grupo pasa de `r--` (4) a `rw-` (6) → **664**. Con letras no hace falta recalcular nada: `chmod g+w fichero`.

```
$ permisos 644 g+w
Base 644  +  chmod g+w

Resultado:

  Octal     664
  Letras    rw-rw-r--      (en ls -l: -rw-rw-r--)
  Binario   110 110 100
  Decimal   436
  Hex       0x1b4

  dueño   rw-  (6)  leer, escribir
  grupo   rw-  (6)  leer, escribir
  otros   r--  (4)  leer

  chmod 664 FICHERO
  chmod u=rw,g=rw,o=r FICHERO
```

🔴 Ejercicio 5.12 · Dos números que se parecen mucho

`chmod 077` y `chmod 770` suenan parecido. ¿Qué hace cada uno?

💡 Solución

```
$ permisos 077 | grep -E 'dueño|grupo|otros'
  dueño   ---  (0)  nada
  grupo   rwx  (7)  leer, escribir, ejecutar
  otros   rwx  (7)  leer, escribir, ejecutar
$ permisos 770 | grep -E 'dueño|grupo|otros'
  dueño   rwx  (7)  leer, escribir, ejecutar
  grupo   rwx  (7)  leer, escribir, ejecutar
  otros   ---  (0)  nada
```

`077` = dueño nada, **todos los demás todo** (¡absurdo y peligroso!). `770` = dueño y grupo todo, otros nada (muy normal en carpetas de equipo). Por eso la **posición** de cada cifra es sagrada.

🔴 Ejercicio 5.13 · La trampa del decimal

Un compañero ejecuta `os.chmod("f", 644)` en Python y se queja de que "los permisos salen raros". ¿Qué modo ha puesto realmente? ¿Cómo se corrige?

💡 Solución

644 **decimal** = `1204` octal → `--w----r-T`. Se corrige con `0o644`.

```
$ printf '%o\n' 644
1204
$ permisos 1204 | head -n 6
1204

  Octal     1204
  Letras    -w----r-T      (en ls -l: --w----r-T)
  Binario   001 010 000 100      (3 bits especiales + 3 × 3)
  Decimal   644
/var/tmp/permgen/bin/permisos: line 139: printf: write error: Broken pipe
/var/tmp/permgen/bin/permisos: line 140: printf: write error: Broken pipe
/var/tmp/permgen/bin/permisos: line 141: printf: write error: Broken pipe
/var/tmp/permgen/bin/permisos: line 144: printf: write error: Broken pipe
/var/tmp/permgen/bin/permisos: line 155: printf: write error: Broken pipe
/var/tmp/permgen/bin/permisos: line 159: printf: write error: Broken pipe
/var/tmp/permgen/bin/permisos: line 160: printf: write error: Broken pipe
```

• Ejercicio 5.14 · Examen: leer un número

¿Qué significa `chmod 640 informe.txt` *?* a) Dueño: leer/escribir; grupo: leer; otros: nada b) Dueño: todo; grupo: leer; otros: nada c) Dueño: leer; grupo: leer/escribir; otros: nada d) Todos: leer

💡 Solución

a). 6 = `rw-`, 4 = `r--`, 0 = `---`.

• Ejercicio 5.15 · Examen: elegir el número

Quieres que el dueño pueda leer/escribir/ejecutar un script, el grupo solo leer y ejecutar, y los demás nada. ¿Qué `chmod` *?* a) 755 b) 750 c) 705 d) 570

💡 Solución

b) 750. `rwx` (7) · `r-x` (5) · `---` (0).

a) daría a "otros" lectura y ejecución; c) daría al grupo nada y a otros `r-x`; d) invierte dueño y grupo.

• Ejercicio 5.16 · Examen: dos formas de lo mismo

¿Cuál de estas órdenes es equivalente a `chmod 644 f` *?* a) `chmod u=rw,go=r f` b) `chmod u=rwx,go=r f` c) `chmod a=r f` d) `chmod +rw f`

💡 Solución

a). `u=rw` (6), `go=r` (4 y 4) → 644. Compruébalo:

```
$ permisos u=rw,go=r | head -n 4
chmod u=rw,go=r   aplicado a 000

Resultado:

/var/tmp/permgen/bin/permisos: line 134: printf: write error: Broken pipe
/var/tmp/permgen/bin/permisos: line 136: printf: write error: Broken pipe
/var/tmp/permgen/bin/permisos: line 137: printf: write error: Broken pipe
/var/tmp/permgen/bin/permisos: line 138: printf: write error: Broken pipe
/var/tmp/permgen/bin/permisos: line 139: printf: write error: Broken pipe
/var/tmp/permgen/bin/permisos: line 140: printf: write error: Broken pipe
/var/tmp/permgen/bin/permisos: line 141: printf: write error: Broken pipe
/var/tmp/permgen/bin/permisos: line 159: printf: write error: Broken pipe
/var/tmp/permgen/bin/permisos: line 160: printf: write error: Broken pipe
```

Capítulo 6 · `chmod` en modo simbólico: `u+x`, `go-w`, `a=r`…

> 🎯 **Objetivo:** cambiar permisos **diciendo lo que quieres con letras**: a **quién** (`u g o a`), **qué haces** (`+ - =`) y **con qué permisos** (`r w x X s t`). Dominar `-R`, `X`, copiar permisos y las trampas de `chmod +x` / `chmod -w`.
>
> 📘 **LPIC-1:** 104.5 (`chmod` simbólico, recursivo) · 103.3.
>
> 🧪 **Antes de empezar:** `cd ~/lab-permisos`. Para ver el resultado cómodamente usaremos `stat -c '%a %A %n'` (octal, letras y nombre).

6.1 · La frase: *"a este quién, hazle esto con estos permisos"*

```
chmod   g  +  w   informe.txt
        │  │  └─ QUÉ permisos:  r  w  x  X  s  t
        │  └──── QUÉ hago:      +  añadir   -  quitar   =  fijar exactamente
        └─────── A QUIÉN:       u dueño · g grupo · o otros · a todos
```

Se lee en voz alta: **"al grupo, añádele escritura"**. Es tan natural que se aprende en minutos.

A quién	Letra
el **dueño** (*user*)	`u`
el **grupo** (*group*)	`g`
los **otros** (*others*)	`o`
todos (*all* = u+g+o)	`a`

Operación	Símbolo	Significa
añadir	`+`	**enciende** esos permisos y **no toca los demás**
quitar	`-`	**apaga** esos permisos y **no toca los demás**
fijar	`=`	pone **exactamente** esos permisos y **apaga el resto** de esa categoría

Y los permisos: `r` leer · `w` escribir · `x` ejecutar · `X` (ejecutar "inteligente", 6.6) · `s` SUID/SGID (cap. 10) · `t` sticky (cap. 10).

6.2 · Primeros pasos: `+`, `-` y `=` con un fichero

Creamos un fichero y lo dejamos en `644` (`rw-r--r--`). Para ver el efecto de cada cambio usamos una función de ayuda que **ejecuta el `chmod` y muestra el resultado**:

```
cp datos/notas.txt prueba.txt
chmod 644 prueba.txt
v() { chmod "$@"; stat -c '%a %A' prueba.txt; }
```

Ahora experimenta (cada línea parte del resultado de la anterior):

```
$ stat -c '%a %A' prueba.txt
644 -rw-r--r--
$ v g+w prueba.txt
664 -rw-rw-r--
$ v o-r prueba.txt
660 -rw-rw----
$ v u+x prueba.txt
760 -rwxrw----
$ v a+x prueba.txt
771 -rwxrwx--x
$ v go-x prueba.txt
760 -rwxrw----
```

Línea a línea:

- `g+w` → al **grupo** le **añado** `w`: `rw-` `rw-` `r--`.
- `o-r` → a **otros** les **quito** `r`: `rw-rw----`.
- `u+x` → al **dueño** le **añado** `x`: `rwxrw----`.
- `a+x` → **a todos** les añado `x`: `rwxrwx--x`.
- `go-x` → a **grupo y otros** (juntos) les quito `x`: `rwxrw----`.

El `=` es el "borra y pon":

```
$ v u=rw prueba.txt
660 -rw-rw----
$ v u=rwx,g=rx,o= prueba.txt
750 -rwxr-x---
$ v g=r prueba.txt
740 -rwxr-----
$ v o=r prueba.txt
744 -rwxr--r--
$ v a= prueba.txt
0 ----------
$ v a=rw prueba.txt
666 -rw-rw-rw-
```

`=` **sustituye** lo que hubiera: `g=r` deja al grupo con `r--` aunque antes tuviera `rwx`. `o=` (sin nada tras el `=`) **lo deja sin permisos**. `a=` los deja a todos sin nada: `----------`.

Quiero…	Con `+` / `-`	Con `=`
Darle `w` al grupo **sin tocar lo demás**	`g+w` ✅	—
Que el grupo tenga **exactamente** `r-x`	—	`g=rx` ✅
Quitar **todo** a los demás	`o-rwx`	`o=` ✅ (más corto)
Poner **exactamente** `rw-r--r--`	—	`u=rw,go=r` ✅

6.3 · Varias órdenes a la vez: la coma, y varios "quién" juntos

Se pueden poner **varias órdenes separadas por comas (sin espacios)**, y **varias letras de "quién" juntas**:

```
$ v u=rwx,g=rx,o=rx prueba.txt
755 -rwxr-xr-x
$ v ug+w,o-rwx prueba.txt
770 -rwxrwx---
$ v a=r,u+w prueba.txt
644 -rw-r--r--
```

- `ug+w,o-rwx` → (dueño y grupo) añaden `w` **y** (otros) pierden todo.
- `a=r,u+w` → todos `r--` y luego el dueño añade `w`: `rw-r--r--`. (Se aplican **de izquierda a derecha**.)

> ⚠️ **Sin espacios.** `chmod u+x, g+w f` (con espacio tras la coma) falla: `chmod` cree que `g+w` es un fichero.

6.4 · Copiar los permisos de otro: `g=u`, `o=g`

A la derecha del operador, en lugar de `r`, `w`, `x`, puedes poner `u`, `g` **u** `o`: *"copia lo que tiene el dueño/grupo/otros"*.

```
$ v u=rwx,g=,o= prueba.txt
700 -rwx------
$ v g=u prueba.txt
770 -rwxrwx---
$ v o=g prueba.txt
777 -rwxrwxrwx
```

`g=u` → "que el **grupo** tenga **lo mismo que el dueño**"; `o=g` → "que **otros** tengan lo mismo que el grupo". Muy útil para **igualar** sin pensar en números: `chmod g=u f` es "dale al grupo lo que ya tengo yo".

6.5 · ⚠️ La trampa del "sin quién": `chmod +x` no es lo mismo que `chmod a+x`

Si **no pones quién** (`chmod +x f`, `chmod -w f`, `chmod =rw f`), `chmod` actúa como `a` **pero respetando tu** `umask`: *los bits que la umask bloquea, no se tocan*. Con una `umask 022` típica casi no se nota, pero con otra...

```
probar() {
  ( umask "$1"
    touch -- t; chmod 666 t; chmod +x t;   a=$(stat -c %a t)
    chmod 777 t; chmod -w t 2>/dev/null;   b=$(stat -c %a t)
    chmod 000 t; chmod =rw t;              c=$(stat -c %a t)
    chmod 777 t; chmod a-w t;              d=$(stat -c %a t)
    printf 'umask %s | 666 +x → %s | 777 -w → %s | 000 =rw → %s | 777 a-w → %s\n' "$1" "$a" "$b" "$c" "$d"
    rm -f t )
}
probar 022
probar 002
probar 077
probar 027
```

Resultado:

```
umask 022 | 666 +x → 777 | 777 -w → 577 | 000 =rw → 644 | 777 a-w → 555
umask 002 | 666 +x → 777 | 777 -w → 557 | 000 =rw → 664 | 777 a-w → 555
umask 077 | 666 +x → 766 | 777 -w → 577 | 000 =rw → 600 | 777 a-w → 555
umask 027 | 666 +x → 776 | 777 -w → 577 | 000 =rw → 640 | 777 a-w → 555
```

Lee la **segunda columna** (`777 -w`): `chmod -w f` sobre un `777`:

- con `umask 022` → `577` (¡solo le quitó la `w` al dueño!)
- con `umask 077` → `577` también
- pero `a-w` (con la `a` explícita) → `555` siempre (última columna).

La razón: la `umask 022` "protege" (no toca) los bits de escritura de grupo y otros, y `chmod` obedece. Y `chmod` hasta **avisa**:

```
$ touch aviso.txt
$ chmod 777 aviso.txt
$ chmod -w aviso.txt
chmod: aviso.txt: new permissions are r-xr-xrwx, not r-xr-xr-x
$ stat -c '%a %A' aviso.txt
557 -r-xr-xrwx
```

> 🎓 **Regla práctica:** cuando **quites** permisos, escribe siempre el *quién* (`a-w`, `go-w`, `u-x`...). Con `+x` sin quién no hay problema en la práctica (la umask casi nunca bloquea la `x`), pero `chmod -w` a secas **te engaña**. (Y `chmod =rw f` también: sobre un `000` con `umask 022` da `644`, **no** `666`.)

6.6 · `X` (mayúscula): "ejecuta, pero solo si tiene sentido"

La `X` mayúscula pone `x` **solo si** (a) es un **directorio**, o (b) el fichero **ya tenía** `x` para alguien. Es la solución a un problema muy típico: *dar acceso a todo un árbol sin convertir los ficheros de texto en ejecutables*.

```
$ mkdir -p arbol/sub
$ echo "texto" > arbol/doc.txt
$ printf '#!/bin/bash\necho hola\n' > arbol/script.sh
$ chmod 744 arbol/script.sh
$ echo "otro" > arbol/sub/dato.txt
$ chmod -R go-rwx arbol
$ find arbol -exec stat -c '%a %A %n' {} +
700 drwx------ arbol
700 -rwx------ arbol/script.sh
700 drwx------ arbol/sub
600 -rw------- arbol/sub/dato.txt
600 -rw------- arbol/doc.txt
$ chmod -R go=rX arbol
$ find arbol -exec stat -c '%a %A %n' {} +
755 drwxr-xr-x arbol
755 -rwxr-xr-x arbol/script.sh
755 drwxr-xr-x arbol/sub
644 -rw-r--r-- arbol/sub/dato.txt
644 -rw-r--r-- arbol/doc.txt
```

Resultado de `go=rX`:

- **Directorios** (`arbol`, `arbol/sub`) → `r-x` para grupo y otros (entrar y listar).
- `script.sh` (que ya tenía `x` para el dueño) → `r-x`.
- **Ficheros de texto** (`doc.txt`, `dato.txt`) → solo `r--`: **¡sin `x`!** Eso no lo habríamos conseguido con `-R go=rx`.

> ⭐ **La receta más usada para "publicar" una carpeta:** `chmod -R u=rwX,go=rX carpeta` (dueño todo, los demás leen y entran, sin ejecutables por accidente). La estudiamos en el capítulo 7.

6.7 · Las opciones útiles de `chmod`

Opción	Hace
`-R`	**recursivo**: entra en las subcarpetas
`-v`	**v**erbose: cuenta cada fichero procesado
`-c`	como `-v`, pero **solo los que cambian**
`-f`	**f**orce/silencio: calla los errores
`--reference=OTRO`	copia los permisos de **otro fichero**
`--preserve-root`	se niega a actuar sobre `/` con `-R`

```
$ touch a b c
$ chmod 600 a b
$ chmod 644 c
$ chmod -v 600 a b c
mode of 'a' retained as 0600 (rw-------)
mode of 'b' retained as 0600 (rw-------)
mode of 'c' changed from 0644 (rw-r--r--) to 0600 (rw-------)
$ chmod -c 640 a b
mode of 'a' changed from 0600 (rw-------) to 0640 (rw-r-----)
mode of 'b' changed from 0600 (rw-------) to 0640 (rw-r-----)
$ chmod -c 640 a b c
mode of 'c' changed from 0600 (rw-------) to 0640 (rw-r-----)
```

`-v` te dice **todo** (aunque no cambie nada: *"retained as"*); `-c` solo lo que **cambió** (*"changed from"*). Fíjate en la última orden: `a` y `b` **ya** estaban en `640`, así que solo informa de `c`.

```
$ touch modelo.txt
$ chmod 604 modelo.txt
$ chmod --reference=modelo.txt a b
$ stat -c '%a %n' modelo.txt a b
604 modelo.txt
604 a
604 b
```

`--reference=modelo.txt` significa "ponles los permisos que tiene `modelo.txt`" (y **no** se escribe modo alguno). Es una forma elegante de **clonar** permisos sin saber el número.

> 🛑 `chmod -R` **es potente y peligroso.** Un `chmod -R 777 /` (o `-R 000 /`) destroza un sistema. Por eso `chmod` moderno trae `--preserve-root`. **Pregúntate siempre en qué carpeta estás antes de un** `-R` (`pwd`).

6.8 · Quién puede hacer `chmod`

Solo el dueño del fichero (o `root`). Aunque tengas `rwx` completo sobre un fichero ajeno, **no** puedes cambiarle los permisos:

```
$ touch comun.txt
$ chmod 666 comun.txt
$ sudo -u carla sh -c 'echo "puedo escribir" >> comun.txt'
$ sudo -u carla chmod 600 comun.txt
chmod: changing permissions of 'comun.txt': Operation not permitted
$ cat comun.txt
puedo escribir
```

`carla` puede **modificar el contenido** (porque `666`), pero **no** `chmod`: `Operation not permitted`. Cambiar permisos es un derecho del **dueño**, no una consecuencia de `rwx`. (Fíjate en el **mensaje**: `Operation not permitted` ≠ `Permission denied`. El primero dice *"esta acción está reservada"*, el segundo *"te falta un permiso"*.)

6.9 · `chmod` y los enlaces simbólicos

`chmod` sobre un enlace simbólico cambia **el fichero al que apunta** (el enlace siempre es `lrwxrwxrwx`):

```
$ touch real.txt
$ chmod 644 real.txt
$ ln -s real.txt acceso
$ chmod 600 acceso
$ stat -c '%a %A %n' real.txt acceso
600 -rw------- real.txt
777 lrwxrwxrwx acceso
$ mkdir -p arb2
$ ln -s ../real.txt arb2/otro-acceso
$ chmod 644 real.txt
$ chmod -R 700 arb2
$ stat -c '%a %A %n' real.txt arb2/otro-acceso
644 -rw-r--r-- real.txt
777 lrwxrwxrwx arb2/otro-acceso
```

Dos cosas:

- `chmod 600 acceso` **cambió** `real.txt` (a `600`), y el enlace sigue `lrwxrwxrwx`.
- Con `-R`, `chmod` **NO sigue** los enlaces de dentro (`real.txt` se quedó en `644`): evita que un enlace malintencionado te haga cambiar permisos fuera de la carpeta.

6.10 · Los permisos especiales con letras (adelanto)

Con letras los bits especiales se escriben fácil:

Quiero…	Comando
poner **SUID**	`chmod u+s programa`
poner **SGID**	`chmod g+s carpeta`
poner **sticky**	`chmod +t carpeta` (o `o+t`)
quitarlos	`chmod u-s`, `g-s`, `-t`

(El capítulo 10 les dedica su espacio.) Y una **diferencia importante** con el modo octal (capítulo 7): `g-s` quita el SGID de una carpeta; `chmod 755` **no** se lo quita.

6.11 · Recetas del día a día

Quiero…	Receta
Hacer ejecutable un script	`chmod u+x script.sh` (o `+x`)
Quitar la escritura a todos	`chmod a-w f`
Hacer algo privado	`chmod go= f` (o `chmod 600 f`)
Que el grupo pueda editar	`chmod g+w f`
Dejar un fichero de solo lectura	`chmod a=r f` (o `chmod 444 f`)
Publicar una carpeta (no ejecutables por accidente)	`chmod -R u=rwX,go=rX carpeta`
Copiar permisos de otro	`chmod --reference=modelo f`
Igualar al grupo con el dueño	`chmod g=u f`
Quitar `x` solo a ficheros (no carpetas)	`find . -type f -exec chmod a-x {} +`

6.12 · 🏋️ Ejercicios

Parte siempre de un fichero nuevo para que sea reproducible. `stat -c '%a %A %n'` es tu amigo.

🟢 Ejercicio 6.1 · Hacer ejecutable

Crea `hola2.sh`, comprueba que no se puede ejecutar y hazlo ejecutable **solo para ti**.

💡 Solución

```
$ printf '#!/bin/bash\necho "hola desde hola2"\n' > hola2.sh
$ ./hola2.sh
bash: ./hola2.sh: Permission denied
$ chmod u+x hola2.sh
$ ./hola2.sh
hola desde hola2
$ stat -c '%a %A' hola2.sh
764 -rwxrw-r--
```

🟢 Ejercicio 6.2 · Quitar escritura a grupo y otros

Un fichero `rw-rw-rw-` debe quedar `rw-r--r--` **sin usar números**.

💡 Solución

```
$ touch ej.txt
$ chmod 666 ej.txt
$ chmod go-w ej.txt
$ stat -c '%a %A' ej.txt
644 -rw-r--r--
```

🟢 Ejercicio 6.3 · Privado

Deja `ej.txt` accesible **solo para ti**, sin números.

> 💡 **Solución**

```
$ chmod go= ej.txt
$ stat -c '%a %A' ej.txt
600 -rw-------
```

(Otra forma: `chmod go-rwx ej.txt`, o `chmod u=rw,go= ej.txt`.)

🟡 Ejercicio 6.4 · Fijar exactamente

Deja `ej.txt` en **exactamente** `rwxr-x---` usando `=`.

> 💡 **Solución**

```
$ chmod u=rwx,g=rx,o= ej.txt
$ stat -c '%a %A' ej.txt
750 -rwxr-x---
```

Ese es el `750`.

🟡 Ejercicio 6.5 · Igualar con `g=u`

El dueño tiene `rwx`. Haz que el grupo tenga **lo mismo**, y que otros tengan **lo mismo que el grupo**, de un tirón.

> 💡 **Solución**

```
$ chmod u=rwx,g=,o= ej.txt
$ chmod g=u,o=g ej.txt
$ stat -c '%a %A' ej.txt
777 -rwxrwxrwx
```

Resultado `777`. En la práctica: `chmod a=u` hace lo mismo.

🟡 Ejercicio 6.6 · Un árbol sin ejecutables accidentales

Con `arbol/` (carpetas, textos y un script), deja: dueño `rwx` en todo lo que ya fuera ejecutable o carpeta, y el resto de gente solo `r-x` en eso, `r--` en los textos.

> 💡 **Solución**

```
$ chmod -R u=rwX,go=rX arbol
$ find arbol -exec stat -c '%a %A %n' {} +
755 drwxr-xr-x arbol
755 -rwxr-xr-x arbol/script.sh
755 drwxr-xr-x arbol/sub
644 -rw-r--r-- arbol/sub/dato.txt
644 -rw-r--r-- arbol/doc.txt
```

La `X` ya sabe distinguir carpetas, ejecutables y textos.

🟡 Ejercicio 6.7 · Ver lo que cambia

Pon `a`, `b` y `c` en `600`; luego usa `chmod -c` para que **solo te diga lo que cambió** al aplicar `644` a los tres, y repítelo.

> 💡 **Solución**

```
$ chmod 600 a b c
$ chmod -c 644 a b c
mode of 'a' changed from 0600 (rw-------) to 0644 (rw-r--r--)
mode of 'b' changed from 0600 (rw-------) to 0644 (rw-r--r--)
mode of 'c' changed from 0600 (rw-------) to 0644 (rw-r--r--)
$ chmod -c 644 a b c
```

La primera vez dice los tres cambios; la segunda, **nada** (ya estaban en `644`). Con `-v` hubiera listado los tres las dos veces.

🔴 Ejercicio 6.8 · El `chmod -w` que quitó de menos

Con `umask 022`, ¿qué obtienes de `chmod -w` sobre un `666`? ¿Y de `chmod a-w`? Explícalo.

💡 Solución

```
$ ( umask 022; touch dos.txt; chmod 666 dos.txt; chmod -w dos.txt; stat -c '%a %A' dos.txt )
chmod: dos.txt: new permissions are r--rw-rw-, not r--r--r--
466 -r--rw-rw-
$ ( umask 022; chmod 666 dos.txt; chmod a-w dos.txt; stat -c '%a %A' dos.txt )
444 -r--r--r--
```

`chmod -w` (sin quién) respeta la `umask`: como la `022` "protege" la escritura de grupo y otros, **solo** se la quita al dueño (`466`). Con `a-w` (quién explícito) se la quita a **todos** (`444`).

🔴 Ejercicio 6.9 · Dos órdenes equivalentes

¿`chmod u=rw,go=r f` y `chmod 644 f` hacen lo mismo? ¿Y qué da `chmod =rw f` sobre un fichero en `000` (con tu `umask`)?

💡 Solución

```
$ touch eq.txt
$ chmod u=rw,go=r eq.txt; stat -c '%a' eq.txt
644
$ chmod 644 eq.txt;       stat -c '%a' eq.txt
644
$ chmod 000 eq.txt
$ umask
0002
$ chmod =rw eq.txt;       stat -c '%a' eq.txt
664
```

Las dos primeras: **sí**, ambas dan `644`. La tercera **depende de la umask** (no lleva "quién"): con la tuya (`0002`) obtienes `664`, **no** `666`, porque la `002` protege la `w` de *otros* y `chmod` la respeta.

🔴 Ejercicio 6.10 · Solo el dueño hace `chmod`

`carla` tiene `rw-` sobre `compartido.txt`. ¿Puede hacerle `chmod 600`? ¿Y `sudo chmod`?

💡 Solución

```
$ touch compartido.txt
$ chmod 666 compartido.txt
$ sudo -u carla chmod 600 compartido.txt
chmod: changing permissions of 'compartido.txt': Operation not permitted
$ sudo chmod 600 compartido.txt
$ stat -c '%a %A %U' compartido.txt
600 -rw------- alumno
```

`carla` no (no es dueña → `Operation not permitted`). `root` sí.

🔴 Ejercicio 6.11 · ¿Qué mod queda?

Parte de `rw-r--r--` (644). Aplica **en este orden**: `chmod u+x,g-r,o+w f`. ¿Qué queda?

💡 Solución

```
$ permisos 644 u+x,g-r,o+w | grep -E 'Octal|Letras'
  Octal     706
  Letras    rwx---rw-      (en ls -l: -rwx---rw-)
```

`u+x` → `rwx`; `g-r` → `---`; `o+w` → `rw-`. Queda `rwx---rw-` = **706**.

🔴 Ejercicio 6.12 · Un solo comando para "publicar"

Deja `arbol/` (con carpetas y ficheros) listo para **servir por web**: carpetas `755`, ficheros `644`, **sin** perder la `x` de los scripts. Hazlo con **un solo** `chmod -R`.

💡 Solución

```
$ chmod -R u=rwX,go=rX arbol
$ find arbol -exec stat -c '%a %n' {} +
755 arbol
755 arbol/script.sh
755 arbol/sub
644 arbol/sub/dato.txt
644 arbol/doc.txt
```

Carpetas → `755`; ficheros → `644`; el script que ya era ejecutable → `755`. (Si quisieras forzar `644` aun en los ejecutables, tendrías que usar `find`: capítulo 7.)

• Ejercicio 6.13 · Examen: `+x` frente a `a+x`

¿En qué se diferencia `chmod +x f` *de* `chmod a+x f` *?* a) En nada b) `+x` respeta la `umask`; `a+x` no c) `a+x` solo afecta al dueño d) `+x` quita la `x`

💡 Solución

b). Sin "quién", `chmod` actúa como `a` pero **no toca los bits bloqueados por la umask**. Con `a+x` (quién explícito) la `umask` no interviene.

• Ejercicio 6.14 · Examen: ¿qué queda?

Un fichero tiene `-rw-r--r--`*. Tras* `chmod g=u,o= f`*, ¿qué permisos tiene?* a) `-rw-rw----` b) `-rw-r-----` c) `-rw-rw-r--` d) `-rwxrwx---`

💡 Solución

a). `g=u` copia el `rw-` del dueño al grupo; `o=` borra los de otros.

```
$ permisos 644 g=u,o= | grep -E 'Octal|Letras'
  Octal     660
  Letras    rw-rw----      (en ls -l: -rw-rw----)
```

• Ejercicio 6.15 · Examen: ¿cuál NO es válida?

¿Cuál de estas órdenes `chmod` *es sintácticamente incorrecta?* a) `chmod u+x,g-w f` b) `chmod ug=rw f` c) `chmod u + x f` d) `chmod a=u f`

💡 Solución

c): los **espacios** separan argumentos (`u`, `+`, `x` serían tres argumentos distintos). Las demás son válidas.

```
$ touch sint.txt
$ chmod u + x sint.txt
chmod: invalid mode: ‘u’
Try 'chmod --help' for more information.
```

Capítulo 7 · `chmod` en modo octal: `755`, `644`, `4755`… y cómo aplicarlos a un árbol

> 🎯 **Objetivo:** usar `chmod` con **números** con total soltura, saber **cuándo conviene** el modo octal y cuándo el simbólico, aplicar permisos a **árboles enteros sin estropear nada** (`find`, `X`) y entender las sutilezas de los **bits especiales** con `chmod` numérico.
>
> 📘 **LPIC-1:** 104.5 (`chmod` octal, `find` para aplicar permisos).
>
> 🧪 **Antes de empezar:** `cd ~/lab-permisos`. Si no tienes la herramienta `permisos` instalada, repasa el capítulo 5.

7.1 · Lo básico: `chmod NNN fichero`

En modo octal el modo se escribe como **un número de 3 o 4 cifras** y `chmod` **fija exactamente** esos permisos (no suma ni resta: **sustituye**):

```
chmod  7 5 5  script.sh
       | | └─ OTROS  (5 = r-x)
       | └─── GRUPO  (5 = r-x)
       └───── DUEÑO  (7 = rwx)
```

```
$ touch f1
$ chmod 755 f1
$ stat -c '%a %A' f1
755 -rwxr-xr-x
$ chmod 640 f1
$ stat -c '%a %A' f1
640 -rw-r-----
$ chmod 000 f1
$ stat -c '%a %A' f1
0 ----------
$ chmod 0644 f1
$ stat -c '%a %A' f1
644 -rw-r--r--
$ chmod 5 f1
$ stat -c '%a %A' f1
5 -------r-x
$ chmod 75 f1
$ stat -c '%a %A' f1
75 ----rwxr-x
```

Cosas que enseña este ejemplo:

- `chmod 755 f` y `chmod 0755 f` **son lo mismo**: el `0` de delante significa "sin bits especiales".
- Si das **menos de 3 cifras**, se rellenan **por la izquierda con ceros**: `chmod 5 f` = `005` (solo "otros", `r-x`… ¡y el dueño y el grupo sin nada!) y `chmod 75 f` = `075`. Es una forma típica de **cometer errores**: no abrevies nunca.
- `chmod` **no pregunta, no suma**: lo que escribes es **el estado final**.
- `stat -c '%a'` **no escribe los ceros de la izquierda**: donde ves `0`, `5` o `75` hay que leer `000`, `005` y `075`.

7.2 · Octal ⇄ simbólico: la tabla de equivalencias

Casi todo lo que se hace con números se puede decir con letras (y viceversa). Los más habituales:

Octal	Simbólico equivalente (desde cero)	Se lee…
`755`	`u=rwx,go=rx`	el dueño todo; los demás leen y ejecutan
`644`	`u=rw,go=r`	el dueño lee/escribe; los demás leen

Octal	Simbólico equivalente (desde cero)	Se lee…
`700`	`u=rwx,go=`	solo el dueño, todo
`600`	`u=rw,go=`	solo el dueño, leer/escribir
`750`	`u=rwx,g=rx,o=`	el grupo lee y ejecuta; otros nada
`640`	`u=rw,g=r,o=`	el grupo lee; otros nada
`664`	`ug=rw,o=r`	dueño y grupo escriben; otros leen
`775`	`ug=rwx,o=rx`	dueño y grupo todo; otros leen/ejecutan
`444`	`a=r`	todos solo lectura
`777`	`a=rwx`	**todo para todos**
`000`	`a=`	nada para nadie

Los comprobamos con la herramienta (la columna izquierda y la derecha deben dar **lo mismo**):

```
$ permisos 755 | grep -E 'Octal'
  Octal     755
$ permisos 'u=rwx,go=rx' | grep -E 'Octal'
  Octal     755
$ permisos 664 | grep -E 'Octal'
  Octal     664
$ permisos 'ug=rw,o=r' | grep -E 'Octal'
  Octal     664
```

7.3 · ¿Octal o simbólico? Cómo elegir

Situación	Mejor	Por qué
Quiero un **estado final concreto** (`rw-r-----`)	**octal**	corto, exacto, no depende de cómo estuviera antes
Quiero **ajustar una cosa** y no tocar el resto (`+x`)	**simbólico**	no tienes que saber el valor actual
Aplicar permisos **a un árbol** con ficheros y carpetas mezclados	**simbólico con** `X` o `find`	el octal trata igual carpetas y ficheros (¡problema!)
Escribir en **un script / documentación**	**octal**	no hay ambigüedad
"Quitar" un permiso	**simbólico**	`a-x` es obvio; en octal hay que recalcular
Probar a mano	el que te sea más cómodo	¡los dos llevan al mismo sitio!

> 🎓 En el examen LPIC-1 aparecen **los dos**: tienes que saber leer `chmod 750` y `chmod u=rwx,g=rx,o=`.

7.4 · Recalcular el octal tras un cambio (y entender el *porqué*)

Para sumar o restar un permiso a un modo octal **sin pasar por letras**, mira esas tres cifras como **bits**:

Quiero…	Qué hago con los bits	Octal	Ejemplo
añadir `w` al grupo	**O** (`|`) con `020`	`| 020`	`644 → 664`
quitar `x` a todos	**Y NO** (`& ~`) con `111`	`& ~111`	`755 → 644`
quitar todo a otros	`& ~007`	`& ~007`	`755 → 750`
añadir `x` al dueño	`| 100`	`| 100`	`644 → 744`

Lo comprobamos con la aritmética de Bash (`8#` = "en octal"):

```
$ printf '%o\n' $(( 8#644 | 8#020 ))
664
$ printf '%o\n' $(( 8#755 & ~8#111 ))
644
$ printf '%o\n' $(( 8#755 & ~8#007 ))
750
$ printf '%o\n' $(( 8#644 | 8#100 ))
744
```

- `+` en simbólico = **O** (`|`): enciende bits.
- `-` en simbólico = **Y NO** (`& ~`): apaga bits.
- `=` en simbólico = **sustituir** esa categoría.

🧠 **¿Por qué no se "suma" a pelo (`644 + 020`)?** Porque si el permiso **ya estaba**, sumarías dos veces y te saldría otra cosa (`664 + 020 = 704`, ¡no `664`!). Con **O** (`|`) añadir algo que ya existe **no cambia nada**. Esta idea de "bits que se encienden o se apagan" es **justo** lo que hace la `umask` (capítulo 8).

7.5 · El gran problema: `chmod -R` con números en un árbol

Cuando aplicas `chmod -R 755 carpeta`, **todo** recibe `755`: carpetas (*bien*) **y ficheros de texto** (*mal*: no deberían ser ejecutables). Preparamos un "sitio web" de prueba con una carpeta pública mal configurada:

```
$ mkdir -p sitio/css sitio/img
$ echo "<html></html>" > sitio/index.html
$ echo "body{}" > sitio/css/estilo.css
$ echo "PNG" > sitio/img/logo.png
$ printf '#!/bin/bash\necho desplegando\n' > sitio/despliegue.sh
$ chmod 700 sitio/despliegue.sh
$ chmod -R go-rwx sitio
$ find sitio | sort | xargs stat -c '%a %n'
700 sitio
700 sitio/css
600 sitio/css/estilo.css
700 sitio/despliegue.sh
700 sitio/img
600 sitio/img/logo.png
600 sitio/index.html
```

❌ Intento 1: `chmod -R 755`

```
$ chmod -R 755 sitio
$ find sitio -type f | sort | xargs stat -c '%a %n'
755 sitio/css/estilo.css
755 sitio/despliegue.sh
755 sitio/img/logo.png
755 sitio/index.html
```

¡Los HTML, CSS y PNG quedaron **ejecutables** (`755`)! Es inofensivo aquí, pero (1) es **desordenado**, (2) en un servidor, un fichero subido que sea ejecutable puede ser peligroso, y (3) ensucia `ls --color` (todo en verde).

✅ Solución A: `find` separando carpetas y ficheros

```
$ chmod -R go-rwx sitio
$ find sitio -type d -exec chmod 755 {} +
$ find sitio -type f -exec chmod 644 {} +
$ find sitio | sort | xargs stat -c '%a %n'
755 sitio
755 sitio/css
644 sitio/css/estilo.css
644 sitio/despliegue.sh
755 sitio/img
644 sitio/img/logo.png
644 sitio/index.html
```

- `find … -type d` → solo **directorios** → `755`.
- `find … -type f` → solo **ficheros** → `644`.
- `-exec chmod 755 {} +` = "ejecuta `chmod 755` sobre **lo que encuentres** (agrupando muchos por llamada)".

Pero `despliegue.sh` **perdió su** `x` (ahora es `644`). Para el script ejecutable hay que repetir `chmod 755 sitio/despliegue.sh`, o usar una solución mejor:

✅ Solución B: `X` mayúscula (conserva la `x` que ya existía)

```
$ find sitio -type f -exec chmod 600 {} +
$ find sitio -type d -exec chmod 700 {} +
$ chmod 700 sitio/despliegue.sh
$ find sitio | sort | xargs stat -c '%a %n'
700 sitio
700 sitio/css
600 sitio/css/estilo.css
700 sitio/despliegue.sh
700 sitio/img
600 sitio/img/logo.png
600 sitio/index.html
$ chmod -R u=rwX,go=rX sitio
$ find sitio | sort | xargs stat -c '%a %n'
755 sitio
755 sitio/css
644 sitio/css/estilo.css
755 sitio/despliegue.sh
755 sitio/img
644 sitio/img/logo.png
644 sitio/index.html
```

(Las cuatro primeras órdenes solo **restauran el estado inicial**: ficheros `600`, carpetas `700` y el script `700`. Después viene la orden que importa.)

`u=rwX,go=rX`: el dueño `rw` + `x` si procede; grupo/otros `r` + `x` si procede. **Carpetas →** `755`**; ficheros normales →** `644`**; el script, que ya tenía** `x`**, →** `755`. Es la receta más usada por los administradores.

✅ Solución C: dos modos distintos con `find` (cuando quieres forzar)

```
$ find sitio -type f -name '*.sh' -exec chmod 755 {} +
$ find sitio -type f ! -name '*.sh' -exec chmod 644 {} +
$ find sitio -type d -exec chmod 755 {} +
$ find sitio | sort | xargs stat -c '%a %n'
755 sitio
755 sitio/css
644 sitio/css/estilo.css
755 sitio/despliegue.sh
755 sitio/img
644 sitio/img/logo.png
644 sitio/index.html
```

Aquí **tú decides** qué es ejecutable (los `*.sh`) y qué no.

Método	Resultado para ficheros normales	Para scripts ya ejecutables	Para carpetas
`chmod -R 755`	**755** 😬	755	755
`find -type f` / `-type d`	644	**644** (pierde `x`)	755
`chmod -R u=rwX,go=rX`	644	**755** (conserva)	755

> 🛑 **Nunca** hagas `chmod -R 777` para "arreglar" un problema de permisos. Convierte todo en escribible por cualquiera. Si algo "no funciona", averigua **quién** debe poder hacer **qué** (capítulo 13) en lugar de abrirlo todo.

7.6 · Las cuatro cifras: los bits especiales también se ponen con números

El cuarto dígito, el de la izquierda, se suma igual que `r w x`: **4 = SUID**, **2 = SGID**, **1 = sticky**. (Los explica el capítulo 10; aquí solo vemos cómo se escriben.)

```
$ touch programa programa-sgid
$ chmod 4755 programa
$ chmod 2755 programa-sgid
$ stat -c '%a %A %n' programa programa-sgid
4755 -rwsr-xr-x programa
2755 -rwxr-sr-x programa-sgid
$ mkdir publica
$ chmod 1777 publica
$ stat -c '%a %A %n' publica
1777 drwxrwxrwt publica
```

- `4755` → `rwsr-xr-x`: la `s` del dueño es el SUID.
- `2755` → `rwxr-sr-x`: la `s` del grupo es el SGID.
- `1777` → `rwxrwxrwt`: la `t` final es el sticky.

7.7 · Una sutileza muy de examen: ¿`chmod 755` quita los bits especiales?

Depende de **qué** sea el objeto. Lo verificamos:

```
$ touch fichero-suid
$ chmod 4755 fichero-suid
$ stat -c 'fichero:  %a %A' fichero-suid
fichero:  4755 -rwsr-xr-x
$ chmod 755 fichero-suid
$ stat -c 'tras 755: %a %A' fichero-suid
tras 755: 755 -rwxr-xr-x
$ echo ---
---
$ mkdir carpeta-sgid
$ chmod 2775 carpeta-sgid
$ stat -c 'carpeta:  %a %A' carpeta-sgid
carpeta:  2775 drwxrwsr-x
$ chmod 755 carpeta-sgid
$ stat -c 'tras 755: %a %A' carpeta-sgid
tras 755: 2755 drwxr-sr-x
$ chmod 0755 carpeta-sgid
$ stat -c 'tras 0755: %a %A' carpeta-sgid
tras 0755: 2755 drwxr-sr-x
$ chmod 00755 carpeta-sgid
$ stat -c 'tras 00755: %a %A' carpeta-sgid
tras 00755: 755 drwxr-xr-x
```

La regla (la de **GNU** `chmod`):

Objeto	`chmod 755` **(3 cifras)**	`chmod 0755` **(4 cifras)**	`chmod 00755` **(5 cifras)**	`chmod g-s`
Fichero con SUID/SGID	**los quita**	los quita	los quita	los quita
Carpeta con SUID/SGID	**los conserva** 😮	**los conserva**	**los quita**	los quita

Es decir: **en una carpeta,** `chmod 755` **NO le quita el SGID** (ni el SUID). Para quitarlo hay que usar `chmod g-s` o escribir el modo con **cinco** cifras (`00755`). Es una protección para que un `chmod` casual no rompa una carpeta de equipo.

7.8 · 📋 Recetas con números: ¿qué `chmod` poner?

Qué es	Modo	Por qué
Fichero de texto / documento / configuración normal	**644**	todos leen, solo tú escribes
Script o programa que otros pueden ejecutar	**755**	todos leen y ejecutan
Carpeta normal	**755**	todos entran y listan
Fichero **privado** (notas, contraseñas, claves SSH privadas)	**600**	solo tú
Carpeta **privada** (`~/.ssh` , `~/privado`)	**700**	solo tú
Fichero de **configuración con secretos**	**640** o **600**	el grupo o nadie más
Documento **compartido en equipo**	**664**	dueño y grupo escriben
Carpeta **compartida en equipo**	**2775** (con SGID)	lo nuevo hereda el grupo (cap. 10)
Carpeta **pública con sticky** (`/tmp`)	**1777**	todos crean, cada uno borra lo suyo
Clave pública SSH (`~/.ssh/id_rsa.pub`)	**644**	se puede compartir
`~/.ssh/authorized_keys`	**600**	solo tú

⚠️ **SSH es muy estricto:** si tu clave privada (`~/.ssh/id_rsa`) tiene permisos demasiado abiertos (digamos `644`), `ssh` **se niega a usarla** con *"WARNING: UNPROTECTED PRIVATE KEY FILE!"* y *"Permissions 0644 for 'id_rsa' are too open"*. La solución: `chmod 600 ~/.ssh/id_rsa` y `chmod 700 ~/.ssh` .

7.9 · 🏋️ Ejercicios

🟢 Ejercicio 7.1 · Tres números

Deja `n1` en `644` , `n2` en `755` y `n3` en `600` . Comprueba con `stat` .

💡 **Solución**

```
$ touch n1 n2 n3
$ chmod 644 n1
$ chmod 755 n2
$ chmod 600 n3
$ stat -c '%a %A %n' n1 n2 n3
644 -rw-r--r-- n1
755 -rwxr-xr-x n2
600 -rw------- n3
```

🟢 Ejercicio 7.2 · Un script ejecutable

Crea `s.sh` (un script) y déjalo **ejecutable por todos pero editable solo por ti**, con un solo número.

💡 **Solución**

```
$ printf '#!/bin/bash\necho hola\n' > s.sh
$ chmod 755 s.sh
$ ./s.sh
hola
$ stat -c '%a %A' s.sh
755 -rwxr-xr-x
```

🟢 Ejercicio 7.3 · Carpeta privada

Crea `mis-cosas` accesible **solo para ti** (con un número) y prueba con `ana` .

💡 **Solución**

```
$ mkdir mis-cosas
$ chmod 700 mis-cosas
$ sudo -u ana ls mis-cosas
ls: cannot open directory 'mis-cosas': Permission denied
```

🟡 Ejercicio 7.4 · Del octal a las letras

Sin ejecutar: ¿qué permisos en letras tiene un fichero con `chmod 640`? ¿Y con `chmod 751`? Después compruébalo.

💡 Solución

```
$ touch q1 q2
$ chmod 640 q1; chmod 751 q2
$ stat -c '%a %A %n' q1 q2
640 -rw-r----- q1
751 -rwxr-x--x q2
```

`640` = `rw-r-----`; `751` = `rwxr-x--x`.

🟡 Ejercicio 7.5 · El número que falta

Un fichero está en `644` y quieres **darle escritura al grupo** y **quitar toda lectura a otros**. ¿Qué número y qué `chmod` simbólico?

💡 Solución

`rw-` `rw-` `---` → **660**. En símbolos: `chmod g+w,o-r f`. Compruébalo:

```
$ permisos 644 g+w,o-r | grep Octal
  Octal     660
```

🟡 Ejercicio 7.6 · Calcular con bits

Sin usar `chmod`, calcula con aritmética qué sale al **quitar la** `x` **a todos** de un `755`, y al **añadir** `w` **al grupo** de un `644`.

💡 Solución

```
$ printf '%o\n' $(( 8#755 & ~8#111 ))
644
$ printf '%o\n' $(( 8#644 | 8#020 ))
664
```

`755 → 644` y `644 → 664`.

🔴 Ejercicio 7.7 · Publicar una web sin ejecutables accidentales

Con el árbol `web/` (carpetas, textos y un script `.sh`), deja: **carpetas** `755`, **textos** `644`, **scripts** `.sh` `755`, usando `find`.

💡 Solución

```
$ mkdir -p web/js web/img
$ echo "<html>" > web/index.html
$ echo "x=1" > web/js/app.js
$ echo "PNG" > web/img/logo.png
$ printf '#!/bin/bash\necho ok\n' > web/publicar.sh
$ chmod -R 700 web
$ find web -type d -exec chmod 755 {} +
$ find web -type f -name '*.sh' -exec chmod 755 {} +
$ find web -type f ! -name '*.sh' -exec chmod 644 {} +
$ find web | sort | xargs stat -c '%a %n'
755 web
755 web/img
644 web/img/logo.png
644 web/index.html
755 web/js
644 web/js/app.js
755 web/publicar.sh
```

🔴 Ejercicio 7.8 · Lo mismo con un solo `chmod`

Consigue el mismo resultado que el ejercicio anterior (suponiendo que el script ya tuviera `x`) con **un único** `chmod -R`.

💡 **Solución**

```
$ find web -type f -exec chmod 600 {} +
$ find web -type d -exec chmod 700 {} +
$ chmod 700 web/publicar.sh
$ chmod -R u=rwX,go=rX web
$ find web | sort | xargs stat -c '%a %n'
755 web
755 web/img
644 web/img/logo.png
644 web/index.html
755 web/js
644 web/js/app.js
755 web/publicar.sh
```

`X` pone `x` a las carpetas y al script (que ya la tenía), pero no a los textos.

🔴 Ejercicio 7.9 · Detectar ejecutables que sobran

Tras un `chmod -R 755` descuidado, lista **solo los ficheros normales** que tengan `x` para el dueño y **no** terminen en `.sh`.

💡 **Solución**

```
$ chmod -R 755 web
$ find web -type f -perm -u+x ! -name '*.sh'
web/img/logo.png
web/index.html
web/js/app.js
```

`-perm -u+x` = "con `x` para el dueño"; `!` niega la condición siguiente. (El apartado 10.7 explica `-perm` con detalle.)

🔴 Ejercicio 7.10 · `chmod 755` sobre una carpeta con SGID

Tienes `equipo-dir` con `2775`. Haces `chmod 755 equipo-dir`. ¿Pierde el SGID? Si quieres quitárselo a propósito, ¿cómo?

💡 **Solución**

```
$ mkdir equipo-dir
$ chmod 2775 equipo-dir
$ chmod 755 equipo-dir
$ stat -c '%a %A' equipo-dir
2755 drwxr-sr-x
$ chmod g-s equipo-dir
$ stat -c '%a %A' equipo-dir
755 drwxr-xr-x
```

No lo pierde (sigue `2755`). Para quitarlo: `chmod g-s` (o `chmod 00755`, con cinco cifras).

🔴 Ejercicio 7.11 · El `chmod` con menos cifras de la cuenta

Un compañero escribe `chmod 75 informe.txt`. ¿Qué permisos deja? ¿Puede leerlo él?

💡 **Solución**

```
$ touch informe.txt
$ chmod 75 informe.txt
$ stat -c '%a %A' informe.txt
75 ----rwxr-x
$ cat informe.txt
cat: informe.txt: Permission denied
```

`75` se lee como `075`: dueño `---`, grupo `rwx`, otros `r-x`. ¡El propio dueño no puede ni leerlo! Lección: **escribe siempre tres cifras**.

• Ejercicio 7.12 · Examen: leer el número

¿Qué hace `chmod 4750 programa`*?* a) SUID, dueño todo, grupo leer/ejecutar, otros nada b) SUID, dueño todo, grupo todo, otros nada c) SGID, dueño todo, grupo leer/ejecutar d) Sticky

💡 **Solución**

a). `4` = SUID; `7` = `rwx` dueño; `5` = `r-x` grupo; `0` = otros nada.

```
$ permisos 4750 | grep -E 'Letras|SUID'
  Letras    rwsr-x---      (en ls -l: -rwsr-x---)
    · SUID (4)    se ejecuta con los privilegios del DUEÑO
```

• Ejercicio 7.13 · Examen: ¿cuál es válido?

Elige el `chmod` ***válido*** *para dejar un fichero* `rw-r-----`*:* a) `chmod 650 f` b) `chmod 640 f` c) `chmod 460 f` d) `chmod 064 f`

💡 **Solución**

b). `rw-` = 6, `r--` = 4, `---` = 0 → **640**.

• Ejercicio 7.14 · Examen: el `chmod -R`

Quieres que todos los ***directorios*** *de un árbol sean* `755` *y todos los* ***ficheros*** `644`*. ¿Cuál es la mejor solución?* a) `chmod -R 755 .` b) `chmod -R 644 .` c) `find . -type d -exec chmod 755 {} +` y `find . -type f -exec chmod 644 {} +` d) `chmod -R 777 .`

💡 **Solución**

c). `a)` hace ejecutables los ficheros; `b)` quita la `x` a las **carpetas** y ya no se puede entrar en ellas (¡pruébalo y luego arréglalo!); `d)` es un desastre de seguridad.

Capítulo 8 · `umask`: por qué tus ficheros nuevos nacen con esos permisos

> 🎯 **Objetivo:** entender **de dónde salen** los permisos de un fichero recién creado (`-rw-rw-r--` , `-rw-r--r--` ...), saber **calcular** una `umask` en un segundo, evitar la **trampa de la resta**, cambiarla de forma temporal y permanente y saber **qué comandos la obedecen y cuáles no**.
>
> 📘 **LPIC-1:** 104.5 (`umask` , permisos por defecto de ficheros nuevos).
>
> 🧪 **Antes de empezar:** `cd ~/lab-permisos` .

8.1 · La pregunta del millón

¿Por qué cuando haces `touch nuevo.txt` sale `-rw-rw-r--` , y cuando haces `mkdir nueva` sale `drwxrwxr-x` ? Nadie te preguntó qué permisos querías. **Los decide la `umask` .**

```
$ cd ~/lab-permisos
$ touch nuevo.txt
$ mkdir nueva
$ ls -ld nuevo.txt nueva
drwxrwxr-x 2 alumno alumno 4096 Oct 16 17:09 nueva
-rw-rw-r-- 1 alumno alumno    0 Oct 16 17:09 nuevo.txt
$ umask
0002
```

La idea, en una imagen:

> 🧱 **Los programas piden los permisos "máximos"** (`666` para un fichero, `777` para una carpeta) y **la `umask` es una plantilla de lo que NO se les concede**: ella **apaga** bits.

```
                  FICHERO                    CARPETA
pide el programa:  666  rw-rw-rw-             777  rwxrwxrwx
umask (0002):      002  -------w-             002  -------w-      ← "estos NO"
                   ─────────────             ─────────────
resultado:         664  rw-rw-r--             775  rwxrwxr-x
```

- **¿Por qué 666 y no 777 para los ficheros?** Por **seguridad**: un fichero **nuevo nunca nace ejecutable**. Si quieres que lo sea, tienes que pedirlo tú con `chmod +x` . Así evitas que un fichero descargado o creado por error se pueda lanzar sin querer. (Por eso `hola.sh` no funcionaba en el capítulo 2.)
- **¿Y las carpetas?** Necesitan la `x` para poder entrar, así que parten de `777` .

8.2 · Ver la `umask`

```
$ umask
0002
$ umask -S
u=rwx,g=rwx,o=rx
```

Comando	Qué muestra
`umask`	la máscara en **octal** (4 cifras): `0002` = lo que **se quita**
`umask -S`	lo contrario, en **letras**: `u=rwx,g=rwx,o=rx` = lo que **se concede**

Cuidado con la **inversión**: `umask` te enseña lo que **se apaga**; `umask -S` lo que **queda permitido** (para carpetas). Son *complementarios*. Con `0002` : se apaga la `w` de otros → `umask -S` dice que **otros** pueden `r-x` .

> 💡 La primera cifra de `0002` (el cero) corresponde a los bits especiales y casi siempre es `0`. Las tres siguientes son **dueño, grupo, otros**, como en `chmod`.

8.3 · El cálculo: `permisos = base & ~umask`

La fórmula exacta (y la única que siempre funciona) es:

```
permisos nuevos  =  permisos pedidos  Y  (NO umask)
                 =  666 & ~umask     (ficheros)
                 =  777 & ~umask     (carpetas)
```

Es el **Y NO** (`& ~`) que vimos en el capítulo 7: "enciende solo los bits que el programa pide **y** que la umask no bloquea". Verlo con números:

```
umask 027  →  bits que se APAGAN:   ---  -w-  rwx
carpeta  777 =  rwx  rwx  rwx
                 |    |    |
                 ▼    ▼    ▼
             quita ---  -w-  rwx     →   rwx  r-x  ---   =  750
fichero  666 =  rw-  rw-  rw-
             quita ---  -w-  rwx     →   rw-  r--  ---   =  640
```

La tabla de las `umask` que debes conocer (calculada por un programa, así que sin erratas):

umask	Ficheros nuevos (666 & ~umask)	Carpetas nuevas (777 & ~umask)	Uso típico
`0000`	`666` `rw-rw-rw-`	`777` `rwxrwxrwx`	nada se quita (¡peligroso!)
`0002`	`664` `rw-rw-r--`	`775` `rwxrwxr-x`	**Ubuntu** para usuarios normales (grupo escribe)
`0007`	`660` `rw-rw----`	`770` `rwxrwx---`	equipo cerrado: otros no ven nada, el grupo trabaja
`0022`	`644` `rw-r--r--`	`755` `rwxr-xr-x`	**clásica**: `root`, Debian, casi todo (grupo y otros solo leen)
`0027`	`640` `rw-r-----`	`750` `rwxr-x---`	servidores: grupo lee, otros nada
`0037`	`640` `rw-r-----`	`740` `rwxr-----`	grupo solo lee (sin `x`), otros nada
`0077`	`600` `rw-------`	`700` `rwx------`	**privado**: solo el dueño
`0222`	`444` `r--r--r--`	`555` `r-xr-xr-x`	ficheros y carpetas "de solo lectura" al nacer

Y la comprobamos **con el sistema** (en subshells `( … )`, para no cambiar tu `umask` real):

```
$ oct() { printf '%03o' "0$(stat -c %a "$1")"; }
$ for u in 000 002 022 027 077 007; do
  ( umask $u; rm -rf pf pd; touch pf; mkdir pd
    echo "umask $u → fichero $(oct pf)  carpeta $(oct pd)" )
done
umask 000 → fichero 666  carpeta 777
umask 002 → fichero 664  carpeta 775
umask 022 → fichero 644  carpeta 755
umask 027 → fichero 640  carpeta 750
umask 077 → fichero 600  carpeta 700
umask 007 → fichero 660  carpeta 770
$ rm -rf pf pd
```

8.4 · ⚠️ La trampa: la umask no es una resta

Casi todo el mundo aprende *"restas la umask de 666 (o de 777)"*. Y casi siempre funciona (con `022`, `002`, `027`, `077`), **pero no siempre**. Mira:

umask	Resta "de cabeza" (666 − umask)	Real (666 & ~umask)	¿Coincide?
`022`	`644`	`644`	✅
`002`	`664`	`664`	✅
`027`	`640`	`640`	✅
`077`	`600`	`600`	✅
`011`	`655`	`666`	❌ **¡no!**
`033`	`633`	`644`	❌ **¡no!**
`013`	`653`	`664`	❌ **¡no!**
`035`	`631`	`642`	❌ **¡no!**
`055`	`611`	`622`	❌ **¡no!**
`101`	`565`	`666`	❌ **¡no!**

¿Por qué falla? Porque los ficheros parten de `666`, que **no tiene** `x` (el bit `1`). Si la `umask` apaga un bit que **ya estaba apagado**, no pasa nada: no se puede quitar lo que no existe. Pero una **resta** sí "quita" un 1, y falsea el resultado. Prueba: `umask 011` apaga las `x`; los ficheros ya no tenían `x`, así que quedan `666`, no `655`.

> 🎓 **Regla segura para ficheros, sin fórmulas: calcula como si fuera una carpeta** (`7 − cifra`) **y después quítale la** `x` a todo lo que haya salido (7 → 6, 5 → 4, 3 → 2, 1 → 0).
>
> ```
> umask 033  →  carpeta: 7-0, 7-3, 7-3 = 744  →  ficheros: 644 ✔
> umask 027  →  carpeta: 750                  →  ficheros: 640 ✔
> umask 011  →  carpeta: 766                  →  ficheros: 666 ✔
> ```

Y con **la herramienta** lo ves y se avisa de la trampa (la calculadora visual `herramientas/calculadora-chmod.html` también tiene una sección de `umask`, con el mismo aviso):

```
$ permisos -u 033
umask 0033

  Ficheros nuevos    666 & ~033 = 644   rw-r--r--
  Carpetas nuevas    777 & ~033 = 744   rwxr--r--

  ⚠  Restar 666 − 033 cifra a cifra daría 633, pero el resultado real es 644.
     La umask es una MÁSCARA (se apagan bits), no una resta.
$ permisos -u 022
umask 0022

  Ficheros nuevos    666 & ~022 = 644   rw-r--r--
  Carpetas nuevas    777 & ~022 = 755   rwxr-xr-x
```

8.5 · Cambiar la `umask`

🔹 Solo para esta terminal

```
$ umask
0002
$ umask 077
$ umask
0077
$ touch secreto-nuevo.txt
$ mkdir carpeta-nueva
$ stat -c '%a %A %n' secreto-nuevo.txt carpeta-nueva
600 -rw------- secreto-nuevo.txt
700 drwx------ carpeta-nueva
$ umask 002
```

Con `umask 077` los ficheros nacen `600` y las carpetas `700`. El cambio afecta **a esa shell y a los procesos que lance** (los hijos la **heredan**), y se pierde al cerrar la terminal. (Aquí hemos vuelto a `002` al final.)

🔹 Solo para un comando o un grupo de comandos: subshell

```
$ umask
0002
$ ( umask 077; touch dentro.txt; stat -c '%a %n' dentro.txt )
600 dentro.txt
$ umask
0002
$ touch fuera.txt
$ stat -c '%a %n' fuera.txt
664 fuera.txt
```

Dentro de `( … )` la `umask` es `077`; fuera sigue como estaba. **No hay forma de que un hijo cambie la `umask` del padre** (igual que no puedes cambiar el `cd` de otro con un script).

🔹 Para siempre: ¿dónde se configura?

Dónde	Efecto
`~/.bashrc` o `~/.profile` (`umask 027`)	solo **tú**, en tus terminales
`/etc/profile`, `/etc/bash.bashrc`	**todos** los usuarios que usen bash
`/etc/login.defs` (`UMASK 022`)	valor por defecto del sistema (lo usa `pam_umask`)
`/etc/pam.d/common-session` (`session optional pam_umask.so`)	aplica `login.defs` al iniciar sesión
systemd (`UMask=0027` en el `.service`)	para un **servicio** concreto

Echemos un vistazo a **tu Ubuntu**:

```
$ grep -n -E '^(UMASK|USERGROUPS_ENAB)' /etc/login.defs
151:UMASK       022
238:USERGROUPS_ENAB yes
$ grep -n pam_umask /etc/pam.d/common-session
22:# The pam_umask module will set the umask according to the system default in
25:# See "man pam_umask".
26:session optional         pam_umask.so
$ grep -n umask ~/.profile
7:# the default umask is set in /etc/profile; for setting the umask
8:# for ssh logins, install and configure the libpam-umask package.
9:#umask 022
```

🐧 El misterio de Ubuntu: ¿por qué `0002` si `login.defs` dice `022`?

Porque `login.defs` también dice `USERGROUPS_ENAB yes`. Con esa opción, `pam_umask` hace un truco: **si tu grupo privado se llama igual que tú** (y el UID es igual que el GID, como `alumno:alumno`), copia los bits del **dueño** al **grupo**: `022` → `002`. Se hace porque, al tener **cada usuario su grupo propio** (UPG), dar escritura al grupo no arriesga nada (¡el grupo solo eres tú!).

Usuario	Grupo principal	`umask` resultante
`alumno` (grupo privado `alumno`)	`alumno`	`0002` → ficheros `664`
`root`	`root`	`0022` → ficheros `644`
usuario de un grupo compartido (`ana` con grupo `users`)	`users`	`0022`

Esa es la razón de que tu `cerve.sh` tenga `-rwxrwxr-x` (creado `664`, luego `chmod +x`), y los demás ficheros del sistema, `-rw-r--r--`.

🔹 ¿Y `sudo`? Siempre al menos `022`

```
$ umask
0002
$ sudo sh -c umask
0022
$ sudo -u ana sh -c umask
0022
```

`sudo` ejecuta con `umask 0022` (el valor más restrictivo entre la tuya y la de `sudoers`), así que **los ficheros que crees con `sudo` serán `644`/`755`**, no `664`/`775`.

8.6 · ¿Qué comandos obedecen la `umask`? (y cuáles no)

Comando	¿La obedece?	Detalle
`touch f`, `> f`, `>> f`	✅	ficheros nuevos: `666 & ~umask`
`mkdir d`	✅	`777 & ~umask`
`cp origen destino`	✅	al crear el destino: `modo-del-origen & ~umask`
`cp -p`	❌	**conserva** el modo del origen
`mv`	❌	no crea nada: **mantiene** el modo
`mkdir -m 755 d`	❌	el modo explícito **manda** (no usa la umask)
`install -m 640 a b`	❌	el modo explícito manda
`chmod`	❌ (salvo "sin quién", cap. 6)	el modo que pides es el que queda
`tar x`	✅ / ❌	aplica la umask salvo con `-p` (root lo hace por defecto)
editores (`nano f`)	✅	un fichero nuevo, sí; uno existente, mantiene su modo

Lo comprobamos:

```
$ ( umask 077
  cp saludos.sh copia-umask.sh
  cp -p saludos.sh copia-p.sh
  mkdir -m 755 d-explicita
  mkdir d-normal
  install -m 640 saludos.sh instalado.sh
  stat -c '%a %n' saludos.sh copia-umask.sh copia-p.sh d-explicita d-normal instalado.sh )
751 saludos.sh
700 copia-umask.sh
751 copia-p.sh
755 d-explicita
700 d-normal
640 instalado.sh
$ mv copia-p.sh movido.sh
$ stat -c '%a %n' movido.sh
751 movido.sh
```

- `saludos.sh` es `751`. Con `umask 077`: `cp` lo deja en `700` (751 sin lo de grupo/otros); `cp -p` **conserva** `751`; `mv` no toca nada.
- `mkdir -m 755` **ignora** la `umask` (sale `755`); `mkdir` a secas sale `700`.
- `install -m 640` pone exactamente `640`.

8.7 · `umask` y seguridad

Para trabajar con cosas sensibles, pon `umask 077` **antes** de crear los ficheros. Es mejor que crearlos y luego hacer `chmod 600`, porque entre una orden y otra **hay un instante en que el fichero es legible** (una *condición de carrera*). Con la `umask` bien puesta, **nace ya protegido**:

```
$ cat > crea-secreto.sh <<'EOF'
#!/bin/bash
umask 077                       # a partir de aquí, todo lo que cree nace privado
echo "clave=supersecreta" > mis-claves.txt
mkdir copias-privadas
EOF
$ chmod +x crea-secreto.sh
$ ./crea-secreto.sh
$ stat -c '%a %A %n' mis-claves.txt copias-privadas
600 -rw------- mis-claves.txt
700 drwx------ copias-privadas
```

(El `umask` dentro del script **solo afecta a ese script**.)

> ⚠️ **La `umask` no protege ficheros ya existentes** y tampoco los copiados con `cp -p` ni los extraídos de un `tar` con `-p`: ahí mandan los permisos del origen.

8.8 · 🏋️ Ejercicios

🟢 Ejercicio 8.1 · Tu `umask`

¿Qué `umask` tienes? ¿Qué permisos tienen tus ficheros nuevos y tus carpetas nuevas?

💡 Solución

```
$ umask
0002
$ touch ej-um.txt
$ mkdir ej-um-dir
$ stat -c '%a %A %n' ej-um.txt ej-um-dir
664 -rw-rw-r-- ej-um.txt
775 drwxrwxr-x ej-um-dir
```

Con `0002`: ficheros `664`, carpetas `775`.

🟢 Ejercicio 8.2 · Privado por defecto

Haz que **todo lo que crees a partir de ahora, en un subshell**, sea privado (solo tú).

💡 Solución

```
$ ( umask 077; touch priv.txt; mkdir priv-dir; stat -c '%a %n' priv.txt priv-dir )
600 priv.txt
700 priv-dir
```

🟢 Ejercicio 8.3 · Calcular con la fórmula

Con `umask 027`, ¿qué permisos tendrán un fichero y una carpeta nuevos?

💡 Solución

Carpeta: `7−0, 7−2, 7−7` → **750**. Fichero (quitando la `x`): **640**.

```
$ permisos -u 027 | grep -E 'Ficheros|Carpetas'
  Ficheros nuevos    666 & ~027 = 640   rw-r-----
  Carpetas nuevas    777 & ~027 = 750   rwxr-x---
```

🟡 Ejercicio 8.4 · Hacia atrás

Quieres que los **ficheros nuevos** nazcan `640` y las **carpetas** `750`. ¿Qué `umask`?

> 💡 **Solución**
>
> La umask es **lo que se apaga**: para las carpetas, `777 − 750 = 027`. `umask 027`. Comprobación:
>
> ```
> $ ( umask 027; touch u1; mkdir u2; stat -c '%a %n' u1 u2 )
> 640 u1
> 750 u2
> ```

🟡 Ejercicio 8.5 · Equipo

Para un equipo que **comparte** una carpeta, ¿qué `umask` conviene: `022` o `002`? ¿Por qué?

> 💡 **Solución**
>
> `002`. Con ella, lo que cada uno crea sale `664` / `775`: el **grupo** puede escribir. Con `022`, los compañeros solo podrían leer y nadie más podría modificar tus ficheros. (Y se combina con la carpeta **SGID**, capítulo 10, para que además hereden el grupo.)

🟡 Ejercicio 8.6 · `cp` frente a `cp -p`

Con `umask 077`, copia `salario.sh` (`775`) con `cp` y con `cp -p`. ¿Qué modos salen?

> 💡 **Solución**
>
> ```
> $ ( umask 077
>   cp salario.sh s1.sh
>   cp -p salario.sh s2.sh
>   stat -c '%a %n' salario.sh s1.sh s2.sh )
> 775 salario.sh
> 700 s1.sh
> 775 s2.sh
> ```
>
> `cp` aplica la umask al modo del origen: `775 & ~077 = 700`. `cp -p` **conserva** `775`.

🟡 Ejercicio 8.7 · Los ficheros nunca nacen ejecutables

Con `umask 000` (¡la más permisiva!), ¿un fichero nuevo sale `777`?

> 💡 **Solución**
>
> ```
> $ ( umask 000; touch ej0; mkdir ej0d; stat -c '%a %A %n' ej0 ej0d )
> 666 -rw-rw-rw- ej0
> 777 drwxrwxrwx ej0d
> ```
>
> **No**: sale `666`. Los programas piden `666` para ficheros; **la `x` no se concede nunca por defecto**, ni siquiera sin `umask`. La carpeta sí sale `777` (se pide `777`).

🔴 Ejercicio 8.8 · La trampa de la resta

Con `umask 033`, ¿qué permisos tendrá un fichero nuevo? ¿Y por qué **no** es `633`?

> 💡 **Solución**
>
> ```
> $ ( umask 033; touch tr1; stat -c '%a' tr1 )
> 644
> ```
>
> **644**. La resta daría `633`, pero `666 & ~033`: en las cifras de grupo y otros, `6 = rw-` y la umask `3 = -wx` apaga **la** `w` (y la `x`, que ya estaba apagada): quedan `r--` → `4`. Restar `6–3` da `3 = -wx`, que ¡ni existe en el fichero!

🔴 Ejercicio 8.9 · ¿Qué `umask` da carpetas `700` y ficheros `600`?

> 💡 **Solución**
>
> `077`. Carpetas: `777 − 077 = 700`; ficheros: `600`. (La más usada para trabajar en privado.)

🔴 Ejercicio 8.10 · Un script seguro

Escribe un script que cree un fichero `datos-sensibles.csv` **que nunca sea legible por otros**, ni siquiera un instante.

> 💡 **Solución**
>
> ```
> $ cat > seguro.sh <<'EOF'
> #!/bin/bash
> umask 077
> echo "id;valor" > datos-sensibles.csv
> EOF
> $ chmod +x seguro.sh
> $ ./seguro.sh
> $ stat -c '%a %A %n' datos-sensibles.csv
> 600 -rw------- datos-sensibles.csv
> ```
>
> El `umask 077` va **antes** de crear el fichero, así que nace `600` (no hay "ventana" entre `touch` y `chmod`).

🔴 Ejercicio 8.11 · El `sudo` y los 644

Si ejecutas `sudo touch f`, ¿qué permisos tiene `f`? ¿Por qué no `664`?

> 💡 **Solución**
>
> ```
> $ sudo touch por-sudo.txt
> $ stat -c '%a %A %U:%G %n' por-sudo.txt
> 644 -rw-r--r-- root:root por-sudo.txt
> ```
>
> `644`, y es de `root`: `sudo` pone su `umask 0022`, que es **más restrictiva** que tu `0002` y prevalece.

• Ejercicio 8.12 · ¿Quién obedece?

De `touch`, `cp`, `cp -p`, `mv`, `mkdir -m 700`, `install -m 600`: ¿cuáles **respetan** la `umask` y cuáles la **ignoran**?

> 💡 **Solución**
>
> - **La respetan:** `touch`, `cp` (a secas), `mkdir` (a secas).
> - **La ignoran:** `cp -p` (conserva el modo original), `mv` (no crea nada), `mkdir -m …` y `install -m …` (modo explícito).

• Ejercicio 8.13 · Entrenamiento de velocidad

Umask → permisos de los ficheros y de las carpetas nuevas

1. `umask 0022`
2. `umask 0002`
3. `umask 0077`
4. `umask 0027`
5. `umask 0007`
6. `umask 0037`
7. `umask 0033`
8. `umask 0011`
9. `umask 0026`
10. `umask 0222`

> 💡 **Respuestas**
>
> 1. `0022` → ficheros **644** (`rw-r--r--`) · carpetas **755** (`rwxr-xr-x`)
> 2. `0002` → ficheros **664** (`rw-rw-r--`) · carpetas **775** (`rwxrwxr-x`)
> 3. `0077` → ficheros **600** (`rw-------`) · carpetas **700** (`rwx------`)
> 4. `0027` → ficheros **640** (`rw-r-----`) · carpetas **750** (`rwxr-x---`)
> 5. `0007` → ficheros **660** (`rw-rw----`) · carpetas **770** (`rwxrwx---`)
> 6. `0037` → ficheros **640** (`rw-r-----`) · carpetas **740** (`rwxr-----`)
> 7. `0033` → ficheros **644** (`rw-r--r--`) · carpetas **744** (`rwxr--r--`)
> 8. `0011` → ficheros **666** (`rw-rw-rw-`) · carpetas **766** (`rwxrw-rw-`)
> 9. `0026` → ficheros **640** (`rw-r-----`) · carpetas **751** (`rwxr-x--x`)
> 10. `0222` → ficheros **444** (`r--r--r--`) · carpetas **555** (`r-xr-xr-x`)

• Ejercicio 8.14 · Entrenamiento: hacia atrás

Quiero estos permisos → ¿qué `umask`?

1. ficheros **640** y carpetas **750**
2. ficheros **600** y carpetas **700**
3. ficheros **664** y carpetas **775**
4. ficheros **660** y carpetas **770**
5. ficheros **644** y carpetas **755**
6. ficheros **666** y carpetas **777**
7. ficheros **444** y carpetas **555**
8. ficheros **640** y carpetas **751**

> 💡 **Respuestas**
>
> 1. ficheros 640 / carpetas 750 → `umask 027`
> 2. ficheros 600 / carpetas 700 → `umask 077`
> 3. ficheros 664 / carpetas 775 → `umask 002`
> 4. ficheros 660 / carpetas 770 → `umask 007`
> 5. ficheros 644 / carpetas 755 → `umask 022`
> 6. ficheros 666 / carpetas 777 → `umask 000`
> 7. ficheros 444 / carpetas 555 → `umask 222`
> 8. ficheros 640 / carpetas 751 → `umask 026`

(Hay **varias** `umask` válidas para un mismo resultado en ficheros: lo que cuenta es la que sirva también para las **carpetas**.)

• Ejercicio 8.15 · Examen: la `umask` y el nuevo fichero

Con `umask 0027`, ¿qué permisos tendrá un fichero nuevo creado con `touch`? a) `rw-r-----` b) `rwxr-x---` c) `rw-r--r--` d) `rwx------`

> 💡 **Solución**
>
> **a) `rw-r-----` (640).** `666 & ~027`. (b) sería una carpeta; los ficheros nuevos nunca llevan `x`.

• Ejercicio 8.16 · Examen: la `umask` de un usuario normal

Un usuario ejecuta `umask 077` y después `mkdir secreto`. ¿Qué permisos tiene `secreto`? a) `drwx------` b) `drwxr-xr-x` c) `drwxrwxrwx` d) `d---------`

> 💡 **Solución**

a) `drwx------` **(700).**

Capítulo 9 · `chown` y `chgrp`: cambiar el dueño y el grupo

> 🎯 **Objetivo:** cambiar **quién es el dueño** y **qué grupo** tiene un fichero, saber **quién tiene permiso para hacerlo** (¡no es quien crees!), dominar todas las sintaxis (`usuario:grupo`, `:grupo`, `usuario:`), el modo recursivo y los casos de la vida real (ficheros que se quedan de `root` por usar `sudo`).
>
> 📘 **LPIC-1:** 104.5 (`chown`, `chgrp`).
>
> 🧪 **Antes de empezar:** `cd ~/lab-permisos`.

9.1 · Qué cambia cada comando

Comando	Cambia…	Ejemplo
`chown` (*change owner*)	el **dueño** (y opcionalmente el grupo)	`chown ana f`
`chgrp` (*change group*)	solo el **grupo**	`chgrp equipo f`
`chmod`	los **permisos** (capítulos 6-7)	`chmod 640 f`

Los tres son independientes: cambiar el dueño **no** cambia los permisos, y al revés.

9.2 · ¿Quién puede cambiar qué? La regla que sorprende

Acción	¿Quién puede?
Cambiar el **dueño** (`chown ana f`)	**solo** `root` (¡ni el propio dueño!)
Cambiar el **grupo** (`chgrp g f`)	`root`, **o el dueño** si **él mismo pertenece al grupo** `g`
Cambiar los **permisos** (`chmod`)	el **dueño** o `root`

¿Por qué **el dueño no puede regalar** su fichero? Para que nadie pueda **endosar** ficheros a otros (cuotas de disco, ficheros "trampa" que hagan parecer culpable a otro usuario…). Lo comprobamos:

```
$ echo "contenido" > regalo.txt
$ chown ana regalo.txt
chown: changing ownership of 'regalo.txt': Operation not permitted
$ chgrp equipo regalo.txt
chgrp: changing group of 'regalo.txt': Operation not permitted
$ sudo chown ana regalo.txt
$ ls -l regalo.txt
-rw-rw-r-- 1 ana alumno 10 Oct 16 17:08 regalo.txt
```

- `chown ana …` → `Operation not permitted`: no eres `root`.
- `chgrp equipo …` → también falla: **tú no eres miembro de** `equipo` (el dueño solo puede asignar grupos **a los que pertenece**).
- Con `sudo` sí funciona.

Y ahora, con una dueña que **sí** es de `equipo` (`ana`) y de ningún `soporte`:

```
$ sudo -u ana chgrp equipo regalo.txt
$ ls -l regalo.txt
-rw-rw-r-- 1 ana equipo 10 Oct 16 17:08 regalo.txt
$ sudo -u ana chgrp soporte regalo.txt
chgrp: changing group of 'regalo.txt': Operation not permitted
```

`ana` (dueña) puede ponerle `equipo` (es miembro) pero **no** `soporte` (no lo es).

> 🧠 **Resumen para el examen:** *solo* `root` *puede cambiar el dueño*; el dueño solo puede cambiar el grupo **a uno de sus propios grupos**.

9.3 · Todas las sintaxis

```
chown  [usuario] [ : [grupo] ]  fichero…
         │            │   │
         │            │   └─ el grupo (opcional)
         │            └──── los dos puntos separan usuario y grupo
         └─────────────────── el usuario (opcional)
```

Forma	Efecto	Ejemplo
`chown ana f`	solo el **dueño**	dueño `ana`, el grupo no cambia
`chown ana:equipo f`	**dueño y grupo**	`ana` y `equipo`
`chown :equipo f`	solo el **grupo** (igual que `chgrp equipo f`)	
`chown ana: f`	dueño `ana` y grupo **= su grupo de inicio de sesión**	`ana:ana`
`chown ana.equipo f`	igual que `ana:equipo` (**sintaxis antigua**, evítala)	los usuarios con punto en el nombre la rompen
`chown 1002:1005 f`	con **números** (UID y GID)	
`chown --reference=otro f`	copia dueño y grupo de `otro`	

```
$ touch c1 c2 c3 c4 c5
$ sudo chown ana c1
$ sudo chown ana:equipo c2
$ sudo chown :soporte c3
$ sudo chown beto: c4
$ sudo chown 1004:1006 c5
$ ls -l c1 c2 c3 c4 c5
-rw-rw-r-- 1 ana    alumno  0 Oct 16 17:08 c1
-rw-rw-r-- 1 ana    equipo  0 Oct 16 17:08 c2
-rw-rw-r-- 1 alumno soporte 0 Oct 16 17:08 c3
-rw-rw-r-- 1 beto   beto    0 Oct 16 17:08 c4
-rw-rw-r-- 1 carla  soporte 0 Oct 16 17:08 c5
```

Lee la salida:

- `c1` → dueño `ana`; el grupo se quedó `alumno`.
- `c2` → `ana:equipo`.
- `c3` → solo cambió el **grupo** (`soporte`).
- `c4` → `beto:` = `beto` **y su grupo principal** (`beto`).
- `c5` → con **números** (UID y GID). En este laboratorio 1004 = `carla` y 1006 = `soporte`; en tu máquina pueden ser otros (míralos con `id carla`). ¡`ls` calcula los nombres al listar!

> ⚠️ Con `chown ana: f` los **dos puntos sin grupo detrás** significan "su grupo principal". Con `chown ana f` (sin dos puntos) **no se toca** el grupo.

Para ver los números reales

```
$ ls -ln c5
-rw-rw-r-- 1 1004 1006 0 Oct 16 17:08 c5
$ sudo chown 4242:4242 c5
$ ls -l c5
-rw-rw-r-- 1 4242 4242 0 Oct 16 17:08 c5
```

Si pones un número que **no corresponde a nadie**, `ls -l` enseña el **número** (el fichero queda **huérfano**). Los ficheros sin dueño válido se buscan con `find / -nouser -o -nogroup`.

9.4 · Las opciones útiles

Opción	Hace
`-R`	**recursivo**: carpeta y todo lo que contiene
`-v`	cuenta cada fichero procesado
`-c`	informa **solo de los que cambian**
`--from=ANTIGUO`	cambia **solo si** ahora el dueño (o `:grupo`) es `ANTIGUO`
`--reference=MODELO`	copia dueño y grupo de `MODELO`
`-h`	si es un **enlace simbólico**, cambia **el enlace** (no el destino)
`--preserve-root`	no actúa sobre `/` con `-R`

```
$ touch o1 o2 o3
$ sudo chown -v ana:equipo o1 o2 o3
changed ownership of 'o1' from alumno:alumno to ana:equipo
changed ownership of 'o2' from alumno:alumno to ana:equipo
changed ownership of 'o3' from alumno:alumno to ana:equipo
$ sudo chown -c ana o1 o2 o3
$ sudo chown -c beto o1
changed ownership of 'o1' from ana to beto
```

`-v` lo dice todo; `-c` solo lo que **cambia** (la segunda orden no cambia nada: ya eran `ana`).

`--from`: cambiar solo "si procede"

```
$ sudo chown --from=beto carla o1 o2
$ ls -l o1 o2
-rw-rw-r-- 1 carla equipo 0 Oct 16 17:08 o1
-rw-rw-r-- 1 ana   equipo 0 Oct 16 17:08 o2
```

`o1` era de `beto` → pasó a `carla`; `o2` era de `ana` → **no se tocó**. Útil para "reasignar lo que era de un compañero que se fue".

`--reference`

```
$ touch igual.txt
$ sudo chown --reference=o2 igual.txt
$ ls -l o2 igual.txt
-rw-rw-r-- 1 ana equipo 0 Oct 16 17:08 igual.txt
-rw-rw-r-- 1 ana equipo 0 Oct 16 17:08 o2
```

9.5 · `chown -R`: la bomba de relojería

Con `-R` cambias **toda la carpeta**: potente, pero **cuidado**:

```
$ mkdir -p proyecto/src proyecto/docs
$ touch proyecto/src/a.c proyecto/docs/leeme.txt
$ sudo chown -R ana:equipo proyecto
$ find proyecto -exec ls -ld {} +
drwxrwxr-x 4 ana equipo 4096 Oct 16 17:08 proyecto
drwxrwxr-x 2 ana equipo 4096 Oct 16 17:08 proyecto/docs
-rw-rw-r-- 1 ana equipo    0 Oct 16 17:08 proyecto/docs/leeme.txt
drwxrwxr-x 2 ana equipo 4096 Oct 16 17:08 proyecto/src
-rw-rw-r-- 1 ana equipo    0 Oct 16 17:08 proyecto/src/a.c
```

Reglas de oro:

1. **Mira dónde estás** (`pwd`) y **a qué apuntas** antes de pulsar Intro con `-R`.
2. **Nunca** `chown -R usuario /` ni `chown -R usuario /etc` (rompe el sistema; `--preserve-root` evita lo de `/`).
3. **Cuidado con** `chown -R usuario .*`: en shells antiguas `.*` incluye `..` ¡y cambias la carpeta **padre** entera! (En la Bash de Ubuntu 24.04 ya no sucede; por si acaso, **no lo escribas**.)

4. Con enlaces simbólicos: por defecto `-R` **no sigue** los enlaces (opción `-P`). Con `-L` los seguiría: **no lo uses** salvo que sepas muy bien qué haces.

9.6 · `chown` y los bits especiales: se borran

Cambiar el dueño o el grupo de un fichero **quita los bits SUID/SGID** de un programa. Es una medida de seguridad: nadie debe heredar un "superprograma" sin darse cuenta.

```
$ cp /usr/bin/id mi-id
$ chmod 4755 mi-id
$ ls -l mi-id
-rwsr-xr-x 1 alumno alumno 39432 Oct 16 17:08 mi-id
$ sudo chown ana mi-id
$ ls -l mi-id
-rwxr-xr-x 1 ana alumno 39432 Oct 16 17:08 mi-id
```

El `s` desapareció (`-rwsr-xr-x` → `-rwxr-xr-x`). Si lo quieres, hay que **volver a ponerlo después** con `chmod u+s` (ver capítulo 10): siempre **primero** `chown`**, luego** `chmod`.

9.7 · Casos reales (los que de verdad vas a vivir)

🔥 ① El fichero que se quedó de `root` por usar `sudo`

Cuando haces `sudo` + un comando que **crea** ficheros, los crea `root`. Y luego tú no puedes modificarlos:

```
$ sudo cp datos/agenda.txt copia-root.txt
$ ls -l copia-root.txt
-rw-r--r-- 1 root root 56 Oct 16 17:08 copia-root.txt
$ echo "añado una línea" >> copia-root.txt
bash: copia-root.txt: Permission denied
$ sudo chown alumno: copia-root.txt
$ echo "ahora sí" >> copia-root.txt
$ ls -l copia-root.txt
-rw-r--r-- 1 alumno alumno 66 Oct 16 17:08 copia-root.txt
```

La solución: `sudo chown tu-usuario: fichero` (los dos puntos finales ponen también tu grupo). Para buscar todo lo que `root` dejó en tu carpeta personal: `find ~ -user root`.

🌐 ② Una carpeta web

El servidor web (Apache/Nginx) corre como el usuario `www-data`. Si sube ficheros, estos deben ser suyos:

```
sudo chown -R www-data:www-data /var/www/html
sudo find /var/www/html -type d -exec chmod 755 {} +
sudo find /var/www/html -type f -exec chmod 644 {} +
```

👥 ③ Una carpeta de equipo

```
$ mkdir carpeta-equipo
$ sudo chown :equipo carpeta-equipo
$ chmod 770 carpeta-equipo
$ ls -ld carpeta-equipo
drwxrwx--- 2 alumno equipo 4096 Oct 16 17:08 carpeta-equipo
$ sudo -u ana touch carpeta-equipo/de-ana.txt
$ sudo -u beto touch carpeta-equipo/de-beto.txt
$ sudo -u carla touch carpeta-equipo/de-carla.txt
touch: cannot touch 'carpeta-equipo/de-carla.txt': Permission denied
$ ls -l carpeta-equipo
total 0
-rw-r--r-- 1 ana  ana  0 Oct 16 17:08 de-ana.txt
-rw-r--r-- 1 beto beto 0 Oct 16 17:08 de-beto.txt
```

Solo `ana` y `beto` (de `equipo`) pueden crear dentro (y tú, `alumno`, **eres el dueño**: entras por la parte del dueño). Fíjate en el **grupo de los ficheros nuevos**: `ana:ana` y `beto:beto`, **no** `equipo`. Esto es un problema: el resto del equipo no podrá editar lo que otro crea. Lo arregla el bit **SGID** del capítulo 10.

🔁 ④ Dejar un fichero a nombre de otro usuario

```
$ echo "entregable" > para-ana.txt
$ chmod 600 para-ana.txt
$ sudo chown ana:ana para-ana.txt
$ ls -l para-ana.txt
-rw------- 1 ana ana 11 Oct 16 17:08 para-ana.txt
$ cat para-ana.txt
cat: para-ana.txt: Permission denied
$ sudo -u ana cat para-ana.txt
entregable
```

Con `600` y dueña `ana`, **solo ana** (y root) lo lee; **tú, que lo creaste, ya no**. Y recuerda (capítulo 4): ya **no puedes hacerle** `chmod` (no eres dueño).

9.8 · Resumen de comandos

Quiero…	Comando
Ver dueño y grupo	`ls -l`, `stat -c '%U:%G %n' f`
Ver los **números**	`ls -ln`, `stat -c '%u:%g %n' f`
Cambiar el dueño	`sudo chown ana f`
Cambiar dueño y grupo	`sudo chown ana:equipo f`
Cambiar solo el grupo	`chgrp equipo f` (si eres de `equipo`) o `sudo chgrp …`
Recursivo	`sudo chown -R ana:equipo carpeta`
Cambiar solo lo que era de X	`sudo chown --from=beto carla *`
Que sea mío otra vez	`sudo chown "$USER": f`
Buscar lo que dejó `root` en mi carpeta	`find ~ -user root`
Buscar ficheros huérfanos	`sudo find / -nouser -o -nogroup 2>/dev/null`

9.9 · 🏋️ Ejercicios

🟢 Ejercicio 9.1 · Ver dueño y grupo

Muestra dueño y grupo de `datos/agenda.txt` con **dos** comandos distintos (uno con letras y otro con números).

💡 Solución

```
$ ls -l datos/agenda.txt
-rw-r--r-- 1 alumno alumno 56 Oct 12 09:30 datos/agenda.txt
$ ls -ln datos/agenda.txt
-rw-r--r-- 1 1001 1001 56 Oct 12 09:30 datos/agenda.txt
$ stat -c '%U:%G  (%u:%g)' datos/agenda.txt
alumno:alumno  (1001:1001)
```

🟢 Ejercicio 9.2 · Cambiar el dueño

Haz que `ana` sea la dueña de `e1.txt`. ¿Qué necesitas?

💡 Solución

```
$ touch e1.txt
$ sudo chown ana e1.txt
$ ls -l e1.txt
-rw-rw-r-- 1 ana alumno 0 Oct 16 17:08 e1.txt
```

`sudo`: solo `root` puede cambiar el dueño.

🟢 Ejercicio 9.3 · Dueño y grupo de golpe

Deja `e2.txt` como `beto:soporte` con una sola orden.

💡 Solución

```
$ touch e2.txt
$ sudo chown beto:soporte e2.txt
$ ls -l e2.txt
-rw-rw-r-- 1 beto soporte 0 Oct 16 17:08 e2.txt
```

🟡 Ejercicio 9.4 · Solo el grupo

Cambia únicamente el grupo de `e1.txt` a `equipo` de **dos formas** distintas.

💡 Solución

```
$ sudo chgrp equipo e1.txt
$ ls -l e1.txt
-rw-rw-r-- 1 ana equipo 0 Oct 16 17:08 e1.txt
$ sudo chown :soporte e1.txt
$ ls -l e1.txt
-rw-rw-r-- 1 ana soporte 0 Oct 16 17:08 e1.txt
```

`chgrp g f` ≡ `chown :g f`.

🟡 Ejercicio 9.5 · ¿Puede el dueño regalar?

Tú eres el dueño de `e3.txt`. Intenta ponerle como dueña a `ana` **sin** `sudo`. ¿Qué pasa y por qué?

💡 Solución

```
$ touch e3.txt
$ chown ana e3.txt
chown: changing ownership of 'e3.txt': Operation not permitted
```

`Operation not permitted`: **solo** `root` **puede cambiar el dueño**. Es una regla del sistema, no un fallo de permisos del fichero.

🟡 Ejercicio 9.6 · Un grupo que no es tuyo

`ana` es dueña de `e4.txt`. ¿Puede cambiarle el grupo a `equipo`? ¿Y a `soporte`?

💡 Solución

```
$ touch e4.txt
$ sudo chown ana e4.txt
$ sudo -u ana chgrp equipo e4.txt
$ ls -l e4.txt
-rw-rw-r-- 1 ana equipo 0 Oct 16 17:08 e4.txt
$ sudo -u ana chgrp soporte e4.txt
chgrp: changing group of 'e4.txt': Operation not permitted
```

A `equipo` sí (es miembro); a `soporte` no.

🟡 Ejercicio 9.7 · Recuperar lo que es tuyo

Tienes `copia2.txt` creado con `sudo` (dueño `root`). Hazlo tuyo otra vez.

💡 Solución

```
$ sudo touch copia2.txt
$ ls -l copia2.txt
-rw-r--r-- 1 root root 0 Oct 16 17:08 copia2.txt
$ sudo chown "$USER": copia2.txt
$ ls -l copia2.txt
-rw-r--r-- 1 alumno alumno 0 Oct 16 17:08 copia2.txt
```

`$USER` vale `alumno`; los dos puntos finales ponen también tu grupo.

🔴 Ejercicio 9.8 · Reasignar con `--from`

Tienes `r1`, `r2`, `r3`: `r1` y `r3` de `beto` y `r2` de `ana`. Pasa **solo los de** `beto` a `carla`.

💡 Solución

```
$ touch r1 r2 r3
$ sudo chown beto r1 r3
$ sudo chown ana r2
$ sudo chown --from=beto carla r1 r2 r3
$ ls -l r1 r2 r3
-rw-rw-r-- 1 carla alumno 0 Oct 16 17:08 r1
-rw-rw-r-- 1 ana   alumno 0 Oct 16 17:08 r2
-rw-rw-r-- 1 carla alumno 0 Oct 16 17:08 r3
```

🔴 Ejercicio 9.9 · Primero `chown`, luego `chmod`

Quieres un programa `miprog` **SUID** de `root`. ¿Por qué debes hacer `chown` **antes** que `chmod u+s`?

💡 Solución

```
$ cp /usr/bin/id miprog-mal
$ chmod u+s miprog-mal
$ sudo chown root miprog-mal
$ ls -l miprog-mal
-rwxr-xr-x 1 root alumno 39432 Oct 16 17:08 miprog-mal
$ cp /usr/bin/id miprog-bien
$ sudo chown root miprog-bien
$ sudo chmod u+s miprog-bien
$ ls -l miprog-bien
-rwsr-xr-x 1 root alumno 39432 Oct 16 17:08 miprog-bien
```

En `miprog-mal` se puso el SUID **y luego** se hizo el `chown`: el `chown` **borró el SUID** (`-rwxr-xr-x`). En `miprog-bien` se hizo primero el `chown` y después el `chmod`: el `s` se queda.

🔴 Ejercicio 9.10 · Reparar una carpeta

Tienes `reparar/` con ficheros, todo de `root` por culpa de un `sudo`. Devuélvelo **a ti** de un solo comando.

💡 Solución

```
$ mkdir -p reparar/sub
$ sudo touch reparar/a reparar/sub/b
$ find reparar -exec ls -ld {} +
drwxrwxr-x 3 alumno alumno 4096 Oct 16 17:08 reparar
-rw-r--r-- 1 root   root      0 Oct 16 17:08 reparar/a
drwxrwxr-x 2 alumno alumno 4096 Oct 16 17:08 reparar/sub
-rw-r--r-- 1 root   root      0 Oct 16 17:08 reparar/sub/b
$ sudo chown -R "$USER": reparar
$ find reparar -exec ls -ld {} +
drwxrwxr-x 3 alumno alumno 4096 Oct 16 17:08 reparar
-rw-r--r-- 1 alumno alumno    0 Oct 16 17:08 reparar/a
drwxrwxr-x 2 alumno alumno 4096 Oct 16 17:08 reparar/sub
-rw-r--r-- 1 alumno alumno    0 Oct 16 17:08 reparar/sub/b
```

• Ejercicio 9.11 · Huérfanos

Crea un fichero con dueño `4242` (un UID que no existe). ¿Cómo lo ves en `ls -l`? ¿Cómo lo **encuentras** en un árbol?

> 💡 **Solución**
>
> ```
> $ touch huerfano.txt
> $ sudo chown 4242:4242 huerfano.txt
> $ ls -l huerfano.txt
> -rw-rw-r-- 1 4242 4242 0 Oct 16 17:08 huerfano.txt
> $ find . -nouser -o -nogroup
> ./huerfano.txt
> ./c5
> ```
>
> `ls -l` enseña el número; `find -nouser -o -nogroup` lo encuentra. Es una **comprobación de auditoría** típica.

• Ejercicio 9.12 · Examen: el dueño y `chown`

¿Quién puede cambiar el propietario de un fichero con `chown`? a) Cualquier usuario con permiso de escritura b) Solo el propietario actual c) Solo `root` d) Cualquier miembro del grupo propietario

> 💡 **Solución**
>
> **c) Solo `root`.**

• Ejercicio 9.13 · Examen: ¿qué hace `chown :web f`?

a) Cambia el dueño a `web` b) Cambia **solo el grupo** a `web` c) Es un error de sintaxis d) Cambia dueño y grupo a `web`

> 💡 **Solución**
>
> **b)** Con `:grupo` sin usuario delante solo cambia el grupo (equivale a `chgrp web f`).

• Ejercicio 9.14 · Examen: `chown ana: f`

¿Qué grupo queda en `f` tras `chown ana: f`? a) El que tuviera b) `root` c) El grupo **principal de** `ana` d) Ninguno

> 💡 **Solución**
>
> **c)**. Los dos puntos **sin grupo detrás** ponen el grupo de inicio de sesión del usuario (`ana:ana`).

Capítulo 10 · Permisos especiales: SUID, SGID y *sticky* (`s`, `S`, `t`, `T`, `4755`, `2775`, `1777`)

> 🎯 **Objetivo:** entender los **tres bits especiales**, **verlos** en un `ls -l`, **ponerlos y quitarlos**, saber **por qué existen** (`passwd`, carpetas de equipo, `/tmp`) y **auditar** un sistema buscándolos con `find`.
>
> 📘 **LPIC-1:** 104.5 (SUID, SGID, sticky) · **110.1** (auditar ficheros con SUID/SGID).
>
> 🧪 **Antes de empezar:** `cd ~/lab-permisos`.

10.1 · Los tres bits especiales de un vistazo

Además de `rwx`, cada fichero guarda **tres interruptores más**. Se escriben como una **cuarta cifra** en octal y **se dibujan sobre la** `x` en `ls -l`:

Bit	Valor	Dónde se dibuja	Letra	En un fichero ejecutable	En una carpeta
SUID (*Set User ID*)	**4**	en la `x` del **dueño**	`s`	se ejecuta **con los privilegios del DUEÑO** del fichero	*(sin efecto en Linux)*
SGID (*Set Group ID*)	**2**	en la `x` del **grupo**	`s`	se ejecuta **con los privilegios del GRUPO** del fichero	lo que se **cree dentro hereda el grupo** de la carpeta
Sticky (*bit pegajoso*)	**1**	en la `x` de **otros**	`t`	*(sin efecto en Linux)*	**solo el dueño** de cada fichero (o de la carpeta, o `root`) puede **borrarlo o renombrarlo**

> 🧠 **Truco de memoria:** el orden **4-2-1** coincide con el orden **dueño-grupo-otros**: el **4** (SUID) va con el **u**suario, el **2** (SGID) con el **g**rupo y el **1** (sticky) con **o**tros. Y están en **la misma columna** que la `x` de cada uno.

```
      -  r w s   r - s   r - t        ← los 3 bits especiales, dibujados sobre la x
         | | |   | | |   | | └─ sticky (en la x de OTROS)
         | | |   | | └─── SGID   (en la x del GRUPO)
         | | └─────────── SUID   (en la x del DUEÑO)
```

Mayúscula o minúscula: `s` frente a `S`, `t` frente a `T`

Se ve	Significa
`s` / `t` (minúscula)	el bit especial **y también** la `x` correspondiente
`S` / `T` (mayúscula)	el bit especial **pero SIN** `x` (suele ser un **error** o algo raro)

```
$ touch e1 e2 e3 e4
$ chmod 4755 e1
$ chmod 4644 e2
$ chmod 1777 e3
$ chmod 1770 e4
$ stat -c '%a  %A  %n' e1 e2 e3 e4
4755  -rwsr-xr-x  e1
4644  -rwSr--r--  e2
1777  -rwxrwxrwt  e3
1770  -rwxrwx--T  e4
```

- `4755` → `rw**s**r-xr-x`: SUID **con** `x` → minúscula.
- `4644` → `rw**S**r--r--`: SUID **sin** `x` → **mayúscula** (¡un SUID que no se puede ejecutar! no sirve para nada).

- `1777` → `rwxrwxrw**t**`: sticky con `x` de otros.
- `1770` → `rwxrwx--**T**`: sticky **sin** `x` de otros (como tu `crontabs`: `drwx-wx--T`).

10.2 · SUID: ejecutar "como el dueño" 🪄

¿Para qué sirve? El caso de `passwd`

Tu contraseña vive en `/etc/shadow`, que **solo** `root` **puede modificar** (`-rw-r-----`). Pero tú puedes cambiar **tu** contraseña con `passwd` sin ser `root`. ¿Cómo? Porque `passwd` es un programa **SUID** `root`: cuando lo ejecutas, **durante esa ejecución actúa como** `root`.

```
$ ls -l /usr/bin/passwd /usr/bin/sudo /usr/bin/su /usr/bin/mount /usr/bin/chsh /usr/bin/newgrp
-rwsr-xr-x 1 root root  44760 May 30  2024 /usr/bin/chsh
-rwsr-xr-x 1 root root  51584 Aug 19 16:47 /usr/bin/mount
-rwsr-xr-x 1 root root  40664 May 30  2024 /usr/bin/newgrp
-rwsr-xr-x 1 root root  64152 May 30  2024 /usr/bin/passwd
-rwsr-xr-x 1 root root  55680 Aug 19 16:47 /usr/bin/su
-rwsr-xr-x 1 root root 282032 Sep 21 18:37 /usr/bin/sudo
$ ls -l /etc/shadow
-rw-r----- 1 root shadow 1485 Oct 16 17:08 /etc/shadow
```

Todos son `-rwsr-xr-x` y de `root`: la `s` en lugar de la `x` del dueño.

Real UID y *effective* UID

Un proceso tiene **dos identidades**:

	Qué es	Sirve para
UID real (*ruid*)	**quién lo lanzó**	saber quién es "de verdad"
UID efectivo (*euid*)	**con qué identidad actúa**	**comprobar los permisos**

Normalmente son iguales. Con **SUID**, el euid pasa a ser el **dueño del fichero**. Lo vemos **con tus propios ojos** usando una copia del programa `id`:

```
$ cp /usr/bin/id id-suid
$ cp /usr/bin/whoami whoami-suid
$ sudo chown root id-suid whoami-suid
$ sudo chmod u+s id-suid whoami-suid
$ ls -l id-suid whoami-suid
-rwsr-xr-x 1 root alumno 39432 Oct 16 17:08 id-suid
-rwsr-xr-x 1 root alumno 35336 Oct 16 17:08 whoami-suid
$ id
uid=1001(alumno) gid=1001(alumno) groups=1001(alumno)
$ ./id-suid
uid=1001(alumno) gid=1001(alumno) euid=0(root) groups=1001(alumno)
$ whoami
alumno
$ ./whoami-suid
root
```

- `id` normal: `uid=1001(alumno)`.
- `./id-suid`: `uid=1001(alumno)` **y** `euid=0(root)`: **tú sigues siendo** `alumno` (real), pero **el proceso actúa como** `root` (efectivo).
- `./whoami-suid` imprime `root` (muestra el usuario *efectivo*).

Sin SUID, el mismo programa copiado se ejecuta como tú:

```
$ cp /usr/bin/id id-normal
$ ./id-normal
uid=1001(alumno) gid=1001(alumno) groups=1001(alumno)
```

⚠️ Linux ignora el SUID en los scripts

Un script SUID sería muy peligroso (el intérprete lee un fichero que alguien podría cambiar en medio…), así que **el kernel de Linux ignora el bit SUID en los ficheros interpretados** (los que empiezan por `#!`). Solo vale en **binarios compilados**:

```
$ printf '#!/bin/bash\nid\n' > suid-script.sh
$ sudo chown root suid-script.sh
$ sudo chmod 4755 suid-script.sh
$ ls -l suid-script.sh
-rwsr-xr-x 1 root alumno 15 Oct 16 17:08 suid-script.sh
$ ./suid-script.sh
uid=1001(alumno) gid=1001(alumno) groups=1001(alumno)
```

Pese a ser `-rwsr-xr-x` y de `root`, el script **no** gana privilegios (no hay `euid=0`). Si necesitas privilegios en un script, usa `sudo` con una regla en `/etc/sudoers`, no SUID.

🚨 El SUID es poderoso, así que es un objetivo

Un programa SUID `root` con un fallo es una **puerta a convertirse en** `root`. Por eso:

- El sistema tiene **pocos** SUID, todos **auditados**.
- **Nunca** pongas SUID a un editor, un intérprete (`bash`, `python`), `find`, `cp`… equivaldría a regalar `root`.
- **Audita** tu sistema periódicamente (apartado 10.7).

```
$ find /usr -perm -4000 -type f 2>/dev/null | sort
/usr/bin/chfn
/usr/bin/chsh
/usr/bin/gpasswd
/usr/bin/mount
/usr/bin/newgrp
/usr/bin/passwd
/usr/bin/su
/usr/bin/sudo
/usr/bin/umount
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/polkit-1/polkit-agent-helper-1
```

(El resultado varía de un sistema a otro; en el tuyo habrá más. Compáralo con una lista "buena" conocida y **desconfía de cualquier recién llegado**.)

10.3 · SGID en ficheros: ejecutar "con el grupo del fichero"

Igual que el SUID, pero con el **grupo**: el proceso actúa con el **GID efectivo** = grupo del fichero.

```
$ cp /usr/bin/id id-sgid
$ sudo chgrp equipo id-sgid
$ sudo chmod g+s id-sgid
$ ls -l id-sgid
-rwxr-sr-x 1 alumno equipo 39432 Oct 16 17:08 id-sgid
$ ./id-sgid
uid=1001(alumno) gid=1001(alumno) egid=1005(equipo) groups=1005(equipo),1001(alumno)
```

Aparece `egid=1005(equipo)` (el **grupo efectivo**) mientras que tu `gid` real sigue siendo `alumno`. Un ejemplo real es `/usr/bin/crontab`, que es `-rwxr-sr-x root crontab`: gracias al SGID, `crontab` **escribe en la carpeta de** `cron` (que solo permite el grupo `crontab`, ¡tu captura!) sin que tú pertenezcas a ese grupo.

```
$ ls -l /usr/bin/expiry
-rwxr-sr-x 1 root shadow 27152 May 30  2024 /usr/bin/expiry
```

10.4 · SGID en carpetas: el truco de los equipos 👥

Esta es **la** utilidad práctica del SGID. En una carpeta con SGID:

📌 **Todo fichero o carpeta creado dentro hereda el GRUPO de la carpeta** (en vez del grupo principal de quien lo crea). Y las **subcarpetas** heredan además el propio bit SGID.

Sin SGID (el problema)

```
$ mkdir sin-sgid
$ sudo chgrp equipo sin-sgid
$ chmod 770 sin-sgid
$ sudo -u ana sh -c 'umask 002; touch sin-sgid/de-ana.txt'
$ sudo -u beto sh -c 'umask 002; touch sin-sgid/de-beto.txt'
$ ls -l sin-sgid
total 0
-rw-rw-r-- 1 ana  ana  0 Oct 16 17:08 de-ana.txt
-rw-rw-r-- 1 beto beto 0 Oct 16 17:08 de-beto.txt
```

Cada uno creó su fichero con **su** grupo principal (`ana:ana`, `beto:beto`): los demás miembros de `equipo` **no podrán editarlos**. (El `umask 002` nos da `rw-rw-r--` como en una sesión normal de Ubuntu.)

⚠️ Antes de nada: el SGID que desaparece en silencio

Un detalle que desconcierta a todo el mundo. Tú (`alumno`) **no eres miembro** de `equipo`. Mira qué pasa si intentas poner el SGID **sin** `sudo`:

```
$ mkdir prueba-sgid
$ sudo chgrp equipo prueba-sgid
$ chmod 2770 prueba-sgid
$ ls -ld prueba-sgid
drwxrwx--- 2 alumno equipo 4096 Oct 16 17:08 prueba-sgid
$ sudo chmod 2770 prueba-sgid
$ ls -ld prueba-sgid
drwxrws--- 2 alumno equipo 4096 Oct 16 17:08 prueba-sgid
```

El primer `chmod 2770` **no da error, pero no pone la** `s` (`drwxrwx---`). Con `sudo` sí (`drwxrws---`). Es una protección del kernel: *si no perteneces al grupo de un fichero o carpeta, tu* `chmod` *no puede activar el SGID* (si no, cualquiera podría "colarse" con los privilegios de un grupo ajeno). **Si no ves la** `s` **tras el** `chmod`**, esta es la causa.** En la vida real lo normal es que quien prepara la carpeta sea `root` (con `sudo`) o miembro del grupo.

Con SGID (la solución)

```
$ mkdir con-sgid
$ sudo chgrp equipo con-sgid
$ sudo chmod 2770 con-sgid
$ ls -ld con-sgid
drwxrws--- 2 alumno equipo 4096 Oct 16 17:08 con-sgid
$ sudo -u ana sh -c 'umask 002; touch con-sgid/de-ana.txt; mkdir con-sgid/sub'
$ sudo -u beto sh -c 'umask 002; touch con-sgid/de-beto.txt'
$ ls -l con-sgid
total 4
-rw-rw-r-- 1 ana  equipo    0 Oct 16 17:08 de-ana.txt
-rw-rw-r-- 1 beto equipo    0 Oct 16 17:08 de-beto.txt
drwxrwsr-x 2 ana  equipo 4096 Oct 16 17:08 sub
```

¡Ahora **todo es del grupo** `equipo`, y la subcarpeta `sub` heredó incluso la `s` (`drwxrwsr-x`). Comprueba que **cada uno puede editar lo del otro**:

```
$ sudo -u beto sh -c 'echo "editado por beto" >> con-sgid/de-ana.txt'
$ cat con-sgid/de-ana.txt
editado por beto
$ sudo -u carla ls con-sgid
ls: cannot open directory 'con-sgid': Permission denied
```

`beto` edita el fichero de `ana` (comparten grupo y el fichero es `rw-rw----`); `carla` (de otro grupo) ni entra.

> ⭐ **Receta de carpeta de equipo:** `sudo chgrp equipo carpeta` + `sudo chmod 2770 carpeta` (o `2775` si otros deben leer) + `umask 002` para los miembros. Para que **lo nuevo** sea editable por el grupo necesitas **las dos cosas**: SGID (el grupo correcto) y `umask 002` (el permiso `w` de grupo).

🔎 Trampa: `mv` frente a `cp`

El grupo se hereda al **crear**. Pero `mv` no crea nada, **mueve**, y el fichero **conserva su grupo**:

```
$ echo "creado fuera" > fuera.txt
$ echo "copiado" > origen2.txt
$ cp origen2.txt con-sgid/copiado.txt
$ mv fuera.txt con-sgid/movido.txt
$ ls -l con-sgid/copiado.txt con-sgid/movido.txt
-rw-rw-r-- 1 alumno equipo  8 Oct 16 17:08 con-sgid/copiado.txt
-rw-rw-r-- 1 alumno alumno 13 Oct 16 17:08 con-sgid/movido.txt
```

El copiado (`cp`) es del grupo `equipo` (heredado); el movido (`mv`) conserva `alumno`. Si pasa en tu equipo, arréglalo con `chgrp`.

10.5 · *Sticky*: "pegajoso" 🍯 (carpetas públicas seguras)

El problema

Una carpeta con `rwxrwxrwx` (`777`) deja que **cualquiera** cree… y que **cualquiera borre lo de los demás** (borrar depende de la carpeta, ¡capítulo 3!). En `/tmp` eso sería un desastre.

```
$ mkdir abierta-sin
$ chmod 777 abierta-sin
$ sudo -u ana touch abierta-sin/ana.txt
$ sudo -u beto rm -f abierta-sin/ana.txt
$ ls abierta-sin
```

`beto` borró el fichero de `ana`. 😱

La solución: el bit *sticky*

Con el bit **sticky** (`1777`), en esa carpeta solo pueden **borrar o renombrar** un fichero:

1. su **dueño**,
2. el **dueño de la carpeta**,
3. `root`.

```
$ mkdir abierta-con
$ chmod 1777 abierta-con
$ ls -ld abierta-con
drwxrwxrwt 2 alumno alumno 4096 Oct 16 17:08 abierta-con
$ sudo -u ana touch abierta-con/ana.txt
$ sudo -u beto rm -f abierta-con/ana.txt
rm: cannot remove 'abierta-con/ana.txt': Operation not permitted
$ sudo -u beto mv abierta-con/ana.txt abierta-con/beto.txt
mv: cannot move 'abierta-con/ana.txt' to 'abierta-con/beto.txt': Operation not permitted
$ ls abierta-con
ana.txt
$ sudo -u ana rm -f abierta-con/ana.txt
$ ls abierta-con
```

- `beto` **no puede** borrar ni renombrar el fichero de `ana` (`Operation not permitted`).
- `ana` **sí puede** borrar el suyo.
- Y recuerda: el **dueño de la carpeta** (tú, `alumno`) y `root` también podrían.

Por eso `/tmp`, `/var/tmp` y `/dev/shm` son `1777`:

```
$ ls -ld /tmp /var/tmp /dev/shm
drwxrwxrwt  3 root root   60 Oct 16 15:58 /dev/shm
drwxrwxrwt 14 root root 4096 Oct 16 17:08 /tmp
drwxrwxrwt  3 root root 4096 Oct 16 17:06 /var/tmp
```

El buzón de tu captura, recreado: `1730`

`drwx-wx--T` es el `crontabs`: grupo con `-wx` (deposita y entra, sin listar) + sticky (nadie borra lo del otro). Lo reproducimos con el grupo `equipo`:

```
$ mkdir buzon-seguro
$ sudo chgrp equipo buzon-seguro
$ chmod 1730 buzon-seguro
$ ls -ld buzon-seguro
drwx-wx--T 2 alumno equipo 4096 Oct 16 17:08 buzon-seguro
$ sudo -u ana touch buzon-seguro/de-ana
$ sudo -u beto touch buzon-seguro/de-beto
$ sudo -u beto rm -f buzon-seguro/de-ana
rm: cannot remove 'buzon-seguro/de-ana': Operation not permitted
$ sudo -u ana ls buzon-seguro
ls: cannot open directory 'buzon-seguro': Permission denied
$ ls buzon-seguro
de-ana  de-beto
```

Cada miembro **deposita**; ninguno **lista** ni **borra lo del otro**. Eso es exactamente `/var/spool/cron/crontabs`.

10.6 · Poner y quitar los bits especiales

Quiero…	Simbólico	Octal (4 cifras)
SUID	`chmod u+s f`	`chmod 4755 f`
SGID	`chmod g+s f`	`chmod 2755 f` / `2775 carpeta`
Sticky	`chmod +t carpeta` (o `o+t`)	`chmod 1777 carpeta`
Varios a la vez	`chmod u+s,g+s f`	`chmod 6755 f` (4+2)
Quitar SUID	`chmod u-s f`	`chmod 0755 f` (en ficheros)
Quitar SGID	`chmod g-s f`	`chmod 0755 f` (en ficheros); en **carpetas**: `g-s` o `00755`
Quitar sticky	`chmod -t carpeta`	`chmod 0777 carpeta`

Cosas a recordar (ya las vimos antes):

- `chown` / `chgrp` **borra el SUID/SGID** de un fichero (capítulo 9): primero `chown`, luego `chmod`.
- **Si no perteneces al grupo** del fichero o carpeta, tu `chmod` **no puede poner el SGID** (se descarta sin avisar): usa `sudo` (apartado 10.4).
- `chmod 755 f` **quita** los bits especiales de un **fichero**, pero `chmod 755 carpeta` **no** quita el SGID de una **carpeta** (capítulo 7).
- **Modificar el contenido** de un fichero SUID/SGID siendo un usuario sin privilegios **también borra** el SUID/SGID (otra protección del kernel). Lo comprobamos:

```
$ cp /usr/bin/id id-modificado
$ sudo chown ana id-modificado
$ sudo chmod 4777 id-modificado
$ ls -l id-modificado
-rwsrwxrwx 1 ana alumno 39432 Oct 16 17:08 id-modificado
$ sudo -u carla sh -c 'echo "x" >> id-modificado'
$ ls -l id-modificado
-rwxrwxrwx 1 ana alumno 39434 Oct 16 17:08 id-modificado
```

(`carla` consigue **escribir**, pero al hacerlo el kernel quita el SUID: `rwsrwxrwx` → `rwxrwxrwx`.)

10.7 · 🔎 Auditar: encontrar SUID, SGID y sticky con `find -perm`

El examen LPIC-1 (objetivo **110.1**) pide saber **buscar ficheros con SUID/SGID**. La herramienta es `find -perm`, que tiene **tres formas**, ¡y es de las cosas que más se confunden!:

Forma	Significa	Ejemplo	Se lee
`-perm 644`	permisos **exactamente** `644`	`find . -perm 644`	ni más ni menos
`-perm -644` (con `-`)	**TODOS** esos bits puestos (y quizá más)	`find . -perm -644`	"al menos `rw-r--r--`"
`-perm /644` (con `/`)	**ALGUNO** de esos bits puesto	`find . -perm /111`	"alguien tiene `x`"

Lo vemos en directo con unos ficheros de prueba:

```
$ mkdir auditoria
$ cd auditoria
$ touch a b c d e
$ chmod 644 a
$ chmod 664 b
$ chmod 755 c
$ chmod 4755 d
$ chmod 2755 e
$ for f in a b c d e; do stat -c '%a  %A  %n' $f; done
644  -rw-r--r--  a
664  -rw-rw-r--  b
755  -rwxr-xr-x  c
4755  -rwsr-xr-x  d
2755  -rwxr-sr-x  e
$ cd ..
```

Ahora las búsquedas (cada una con su pregunta):

```
$ cd auditoria
$ echo "--- exactamente 644:";               find . -type f -perm 644 | sort
--- exactamente 644:
./a
$ echo "--- al menos rw-r--r-- (-644):";      find . -type f -perm -644 | sort
--- al menos rw-r--r-- (-644):
./a
./b
./c
./d
./e
$ echo "--- alguien con x (/111):";           find . -type f -perm /111 | sort
--- alguien con x (/111):
./c
./d
./e
$ echo "--- SUID (-4000):";                   find . -type f -perm -4000 | sort
--- SUID (-4000):
./d
$ echo "--- SGID (-2000):";                   find . -type f -perm -2000 | sort
--- SGID (-2000):
./e
$ echo "--- SUID o SGID (/6000):";            find . -type f -perm /6000 | sort
--- SUID o SGID (/6000):
./d
./e
$ cd ..
```

Las tres recetas de auditoría que debes **saber de memoria**:

Quiero encontrar…	Comando
Ficheros con **SUID**	`find / -perm -4000 -type f 2>/dev/null`
Ficheros con **SGID**	`find / -perm -2000 -type f 2>/dev/null`
SUID o SGID	`find / -perm /6000 -type f 2>/dev/null` (o `\( -perm -4000 -o -perm -2000 \)`)

Quiero encontrar…	Comando
Carpetas **escribibles por todos y SIN sticky** (peligrosas)	`find / -xdev -type d -perm -0002 ! -perm -1000 2>/dev/null`
Ficheros **escribibles por cualquiera**	`find / -xdev -type f -perm -0002 2>/dev/null`
Ficheros **sin dueño**	`find / -nouser -o -nogroup 2>/dev/null`

> 💡 El `2>/dev/null` tira a la basura los *"Permission denied"* de las carpetas donde no puedes entrar (con `sudo` no hacen falta). Y `-xdev` impide que `find` salte a otros sistemas de ficheros (`/proc`, `/sys`…).

```
$ find /tmp /var/tmp -xdev -type d -perm -0002 ! -perm -1000 2>/dev/null
/tmp/exp
$ echo "(si no sale nada, ninguna carpeta pública sin sticky)"
(si no sale nada, ninguna carpeta pública sin sticky)
```

10.8 · 📋 Resumen para el examen

```
SUID  4   rws r-x r-x   4755   ejecuta con los privilegios del DUEÑO     (passwd)
SGID  2   rwx r-s r-x   2755   ejecuta con los privilegios del GRUPO     (crontab)
SGID  2   rwx rws r-x   2775   carpeta: lo nuevo hereda el GRUPO         (equipo)
STICKY 1  rwx rwx rwt   1777   carpeta: cada uno borra solo lo suyo      (/tmp)
```

Pregunta típica	Respuesta
¿Qué hace el SUID?	El programa se ejecuta con los permisos de **su dueño**, no de quien lo lanza
¿Cómo se ve?	`s` en lugar de la `x` del dueño (`rws`)
¿Qué significa `S` mayúscula?	Bit especial **sin** `x` debajo
¿Para qué sirve el SGID en una carpeta?	Que lo nuevo **herede el grupo** de la carpeta
¿Para qué sirve el *sticky*?	Que solo el dueño (o el de la carpeta / root) **borre** sus ficheros
¿Dónde se usa el *sticky*?	`/tmp`, `/var/tmp`
¿Cómo busco los SUID?	`find / -perm -4000`
¿Funciona el SUID en scripts?	**No** (Linux lo ignora)

10.9 · 🏋 Ejercicios

🟢 Ejercicio 10.1 · Leer un `s`

```
-rwsr-xr-x 1 root root 64152 may 30  2024 /usr/bin/passwd
```

¿Qué hace la `s`? ¿Cuál es su octal?

> 💡 **Solución**
>
> - **Tipo:** `-` → fichero normal
> - **Dueño (`root`):** `rws` → leer, escribir, ejecutar
> - **Grupo (`root`):** `r-x` → leer, ejecutar
> - **Otros (el resto del mundo):** `r-x` → leer, ejecutar
> - **En octal:** `4755`
> - **Bits especiales: SUID** (bit especial de usuario) (se explican en el capítulo 10)
>
> Cuando **cualquier** usuario lo ejecuta, el programa corre **como** `root` (su dueño): por eso puede escribir en `/etc/shadow`.

🟢 Ejercicio 10.2 · Leer un `t`

```
drwxrwxrwt 18 root root 4096 oct 16 11:28 /tmp
```

💡 Solución

- **Tipo:** `d` → directorio (carpeta)
- **Dueño (`root`):** `rwx` → listar, crear/borrar/renombrar dentro, entrar
- **Grupo (`root`):** `rwx` → listar, crear/borrar/renombrar dentro, entrar
- **Otros (el resto del mundo):** `rwt` → listar, crear/borrar/renombrar dentro, entrar
- **En octal:** `1777`
- **Bits especiales: sticky** (bit pegajoso) (se explican en el capítulo 10)

La `t` es el **sticky**: todos pueden crear en `/tmp`, pero cada uno solo borra lo suyo.

🟢 Ejercicio 10.3 · Leer un `T`

```
drwx-wx--T 2 root crontab 4096 oct  7 20:42 /var/spool/cron/crontabs
```

💡 Solución

- **Tipo:** `d` → directorio (carpeta)
- **Dueño (`root`):** `rwx` → listar, crear/borrar/renombrar dentro, entrar
- **Grupo (`crontab`):** `-wx` → crear/borrar/renombrar dentro, entrar
- **Otros (el resto del mundo):** `--T` → **nada**
- **En octal:** `1730`
- **Bits especiales: sticky** (bit pegajoso) (se explican en el capítulo 10)

La `T` mayúscula indica **sticky sin `x` para otros** (otros no tienen ningún permiso: `---`, y encima el sticky).

🟡 Ejercicio 10.4 · Poner el SGID a una carpeta de equipo

Crea `mi-equipo`, del grupo `equipo`, de forma que lo que `ana` cree dentro herede el grupo `equipo`. (Recuerda que tú no eres de `equipo`.)

💡 Solución

```
$ mkdir mi-equipo
$ sudo chgrp equipo mi-equipo
$ sudo chmod 2775 mi-equipo
$ ls -ld mi-equipo
drwxrwsr-x 2 alumno equipo 4096 Oct 16 17:08 mi-equipo
$ sudo -u ana touch mi-equipo/nota-de-ana.txt
$ ls -l mi-equipo
total 0
-rw-r--r-- 1 ana equipo 0 Oct 16 17:08 nota-de-ana.txt
```

🟡 Ejercicio 10.5 · Un SUID de juguete

Crea `quien-soy` (copia de `whoami`), propiedad de `root`, que al ejecutarlo **tú** diga `root`.

💡 Solución

```
$ cp /usr/bin/whoami quien-soy
$ sudo chown root quien-soy
$ sudo chmod u+s quien-soy
$ ./quien-soy
root
```

Recuerda el orden: `chown` **primero**, `chmod u+s` **después**.

🟡 Ejercicio 10.6 · Sticky en tu carpeta compartida

Haz `compartida` de modo que **cualquiera pueda crear** pero **solo cada uno borre lo suyo**.

💡 Solución

```
$ mkdir compartida
$ chmod 1777 compartida
$ ls -ld compartida
drwxrwxrwt 2 alumno alumno 4096 Oct 16 17:08 compartida
$ sudo -u ana touch compartida/a.txt
$ sudo -u carla rm -f compartida/a.txt
rm: cannot remove 'compartida/a.txt': Operation not permitted
$ ls compartida
a.txt
```

🟡 Ejercicio 10.7 · Octal con `s` y `t`

Pasa a octal: `rwsr-xr-x` · `rwxr-sr-x` · `rwxrwxrwt` · `rwx--S---`. Compruébalo con `permisos`.

💡 Solución

```
$ permisos rwsr-xr-x | grep Octal
  Octal     4755
$ permisos rwxr-sr-x | grep Octal
  Octal     2755
$ permisos rwxrwxrwt | grep Octal
  Octal     1777
$ permisos rwx--S--- | grep Octal
  Octal     2700
```

`4755`, `2755`, `1777`, `2700` (la `S` mayúscula del grupo = SGID sin `x`).

🔴 Ejercicio 10.8 · ¿Por qué no funciona el SUID en mi script?

Un compañero ha hecho `chmod 4755 backup.sh` (script de `root`) y dice que "no tiene privilegios". ¿Por qué? ¿Qué le recomendarías?

💡 Solución

Porque **Linux ignora el SUID/SGID en los scripts** (ficheros `#!`). Opciones: (1) usar `sudo` con una regla concreta en `sudoers`; (2) un pequeño programa compilado SUID que lance el script (¡con cuidado!); (3) usar una unidad `systemd` / `cron` que corra como `root`.

🔴 Ejercicio 10.9 · Carpetas públicas peligrosas

¿Qué comando busca carpetas **escribibles por todos y sin sticky**? Pruébalo creando una a propósito.

💡 Solución

```
$ mkdir -p peligro-publica
$ chmod 777 peligro-publica
$ find . -type d -perm -0002 ! -perm -1000
./abierta-sin
./peligro-publica
```

`-perm -0002` = "`w` para otros"; `! -perm -1000` = "**sin** sticky". (Esta salida también incluye `abierta-sin`, que dejamos así antes.) Soluciones: `chmod +t carpeta`.

🔴 Ejercicio 10.10 · Heredar el grupo… pero no los permisos

`ana` crea `todo.txt` en una carpeta `2775` del grupo `equipo` con `umask 022`. ¿Qué permisos tiene `todo.txt` y puede `beto` editarlo?

💡 **Solución**

```
$ mkdir equipo2
$ sudo chgrp equipo equipo2
$ sudo chmod 2775 equipo2
$ sudo -u ana sh -c 'umask 022; touch equipo2/todo.txt'
$ ls -l equipo2/todo.txt
-rw-r--r-- 1 ana equipo 0 Oct 16 17:08 equipo2/todo.txt
$ sudo -u beto sh -c 'echo x >> equipo2/todo.txt'
sh: 1: cannot create equipo2/todo.txt: Permission denied
```

Hereda el **grupo** (`equipo`), pero con `umask 022` los permisos son `rw-r--r--`: `beto` **no** puede editarlo. Para el trabajo en equipo hacen falta **las dos cosas**: SGID + `umask 002`.

🔴 Ejercicio 10.11 · Quitar bits especiales

Tienes `f` con `6755`. Quita **solo el SGID** sin tocar el SUID, con símbolos.

💡 **Solución**

```
$ touch f6755
$ chmod 6755 f6755
$ stat -c '%a %A' f6755
6755 -rwsr-sr-x
$ chmod g-s f6755
$ stat -c '%a %A' f6755
4755 -rwsr-xr-x
```

• Ejercicio 10.12 · Auditoría

Encuentra todos los ficheros con **SUID o SGID** dentro de `/usr/bin`, ordenados, con una sola orden.

💡 **Solución**

```
$ find /usr/bin -type f -perm /6000 2>/dev/null | sort
/usr/bin/chage
/usr/bin/chfn
/usr/bin/chsh
/usr/bin/expiry
/usr/bin/gpasswd
/usr/bin/mount
/usr/bin/newgrp
/usr/bin/passwd
/usr/bin/su
/usr/bin/sudo
/usr/bin/umount
```

(`-perm /6000` = "SUID **o** SGID". Otra forma: `\( -perm -4000 -o -perm -2000 \)`.)

• Ejercicio 10.13 · `-perm 644`, `-perm -644` y `-perm /644`

Con los ficheros de `auditoria/` (`644`, `664`, `755`, `4755`, `2755`), ¿cuáles encuentra cada una?

💡 **Solución**

```
$ cd auditoria
$ echo "exacto 644:";   find . -type f -perm 644  | sort
exacto 644:
./a
$ echo "-644 (todos):";  find . -type f -perm -644 | sort
-644 (todos):
./a
./b
./c
./d
./e
$ echo "/644 (alguno):"; find . -type f -perm /644 | sort
/644 (alguno):
./a
./b
./c
./d
./e
$ cd ..
```

- `-perm 644` → solo `a` (**exactamente**).
- `-perm -644` → `a`, `b`, `c`, `d`, `e`: **todos** tienen **al menos** `rw-r--r--`.
- `-perm /644` → todos: cualquiera tiene **alguno** de esos bits.

• Ejercicio 10.14 · Examen: ¿qué es `drwxrwsr-x`?

a) Carpeta con SUID b) Carpeta con SGID c) Carpeta con sticky d) Fichero ejecutable

💡 Solución

b) Carpeta (`d`) con **SGID** (la `s` en la `x` del **grupo**). Octal `2775`.

• Ejercicio 10.15 · Examen: ¿cómo busco SUID?

¿Qué comando muestra todos los ficheros con el bit SUID del sistema? a) `find / -perm 4000` b) `find / -perm -4000` c) `find / -type s` d) `ls -l | grep s`

💡 Solución

b) `find / -perm -4000`. (a) busca **exactamente** `4000` (ni `4755` valdría). `-type s` busca **sockets**.

• Ejercicio 10.16 · Examen: el sticky

¿Qué hace el bit sticky en un directorio? a) Impide entrar b) Hace que los ficheros hereden el grupo c) Solo el dueño del fichero (o del directorio, o root) puede borrarlo o renombrarlo d) Ejecuta con permisos de root

💡 Solución

c). (b es el SGID; d es el SUID.)

Capítulo 11 · Los permisos en la vida real: el sistema, los comandos de cada día y los enlaces

> 🎯 **Objetivo:** reconocer **por qué** los ficheros y carpetas de un Linux real tienen los permisos que tienen (`/etc/shadow`, `/home`, `/tmp`, `/var/mail` …), saber **qué permisos pide cada comando** (`cp`, `mv`, `rm`, `ln` …), entender cómo los **editores** y `sed -i` "se saltan" el `w` del fichero, y qué pasa con los permisos de los **enlaces**.
>
> 📘 **LPIC-1:** 104.5 · **104.6** (enlaces duros y simbólicos) · 107.1 · 110.1.
>
> 🧪 **Antes de empezar:** `cd ~/lab-permisos`.

11.1 · El mapa de permisos de un Ubuntu

Si ahora mismo ejecutas `ls -ld` sobre las carpetas y ficheros clave de **tu** sistema, verás esto (este es el de **nuestro** laboratorio, un Ubuntu 24.04 real):

```
$ ls -ld / /etc /home /root /tmp /usr/bin /var /var/log /var/mail /var/tmp
drwxr-xr-x 22 root root  4096 Oct 16 15:58 /
drwxr-xr-x 81 root root  4096 Oct 16 17:08 /etc
drwxr-xr-x  9 root root  4096 Oct 16 17:08 /home
drwx------ 16 root root  4096 Oct 16 16:02 /root
drwxrwxrwt 14 root root  4096 Oct 16 17:08 /tmp
drwxr-xr-x  2 root root 24576 Oct  3 05:42 /usr/bin
drwxr-xr-x 11 root root  4096 Sep 17 02:29 /var
drwxr-xr-x  7 root root  4096 Oct  3 05:36 /var/log
drwxrwsr-x  2 root mail  4096 Sep 17 02:20 /var/mail
drwxrwxrwt  3 root root  4096 Oct 16 17:06 /var/tmp
```

Y los ficheros de cuentas y de configuración:

```
$ ls -l /etc/passwd /etc/group /etc/shadow /etc/gshadow /etc/sudoers /etc/hosts /etc/fstab /etc/login.defs
-rw-r--r-- 1 root root      37 Sep 17 02:20 /etc/fstab
-rw-r--r-- 1 root root    1062 Oct 16 17:08 /etc/group
-rw-r----- 1 root shadow   933 Oct 16 17:08 /etc/gshadow
-rw-r--r-- 1 root root      49 Oct 16 15:58 /etc/hosts
-rw-r--r-- 1 root root   12345 Feb 22  2024 /etc/login.defs
-rw-r--r-- 1 root root    3558 Oct 16 17:08 /etc/passwd
-rw-r----- 1 root shadow  1485 Oct 16 17:08 /etc/shadow
-r--r----- 1 root root    1831 Oct  3 05:42 /etc/sudoers
```

Cada uno responde a una **idea de seguridad**. Esta es la tabla que debes llevar en la cabeza:

Ruta	Permisos	Dueño:grupo	Por qué es así
`/`	`drwxr-xr-x` (755)	root:root	cualquiera puede **ver** la raíz, solo `root` la modifica
`/etc`	`drwxr-xr-x`	root:root	la configuración se **lee** (todos) pero solo la cambia `root`
`/root`	`drwx------` (700)	root:root	la carpeta personal de `root` es **privada**
`/home/usuario`	`drwxr-x---` (750)	usuario:usuario	cada usuario protege la suya (Ubuntu 22.04+ → `HOME_MODE 0750`)
`/tmp`, `/var/tmp`	`drwxrwxrwt` (**1777**)	root:root	todos escriben, cada uno **solo borra lo suyo** (sticky)
`/var/mail`	`drwxrwsr-x` (**2775**)	root:**mail**	buzones: lo creado **hereda el grupo** `mail` (SGID)

Ruta	Permisos	Dueño:grupo	Por qué es así
`/var/log`	`drwxr-xr-x`	root:root	los **registros** se guardan aquí; los ficheros de dentro suelen ser `640` (`syslog:adm`)
`/etc/passwd`	`-rw-r--r--` (644)	root:root	hay que **leerlo** (convertir UID en nombre)
`/etc/shadow`	`-rw-r-----` (**640**)	root:**shadow**	**contraseñas cifradas**: solo `root` y el grupo `shadow`
`/etc/group`	`-rw-r--r--`	root:root	igual que `passwd`
`/etc/gshadow`	`-rw-r-----`	root:shadow	contraseñas de grupos
`/etc/sudoers`	`-r--r-----` (**440**)	root:root	¡solo lectura **incluso para root**! Se edita con `visudo`
`/usr/bin/passwd`, `/usr/bin/sudo`, `/bin/su`	`-rwsr-xr-x` (**4755**)	root:root	**SUID** para poder actuar como root (capítulo 10)
`/dev/null`	`crw-rw-rw-`	root:root	tirar datos: cualquiera
`~/.ssh/`	`drwx------` (700)	usuario	claves SSH: **privado**
`~/.ssh/id_ed25519` (clave **privada**)	`-rw-------` (**600**)	usuario	si está abierta, `ssh` **la rechaza**

> 🧠 **Pauta general que se repite:** *lo que se puede consultar → lectura para todos (644/755); lo que contiene secretos → 600/640/700; lo que todos usan para escribir → sticky.*

11.2 · `/home`: ¿por qué `ana` no puede ver tu carpeta?

Desde Ubuntu 21.04 las carpetas personales se crean `750`; el valor sale de `HOME_MODE` en `/etc/login.defs`:

```
$ grep -n HOME_MODE /etc/login.defs
153:# HOME_MODE is used by useradd(8) and newusers(8) to set the mode for new
155:# If HOME_MODE is not set, the value of UMASK is used to create the mode.
156:HOME_MODE   0750
$ ls -ld /home/ana /home/beto /home/carla
drwxr-x--- 2 ana   ana   4096 Oct 16 17:08 /home/ana
drwxr-x--- 2 beto  beto  4096 Oct 16 17:08 /home/beto
drwxr-x--- 2 carla carla 4096 Oct 16 17:08 /home/carla
$ sudo -u carla ls /home/ana
ls: cannot open directory '/home/ana': Permission denied
```

`carla` no puede ni **listar** la carpeta de `ana` (`---` para otros). (Tu propia carpeta aparece con `751` porque el script del laboratorio le añadió `o+x`, como vimos en el capítulo 3.)

11.3 · Una carpeta con SGID de verdad: `/var/mail`

`/var/mail` es `drwxrwsr-x root mail`. **Está en el sistema desde siempre**: el correo de cada usuario es un fichero ahí; el grupo `mail` lo **hereda** cada fichero nuevo gracias al SGID, y así el programa de correo (con ese grupo) puede escribir en todos los buzones sin ser `root`.

```
$ ls -ld /var/mail
drwxrwsr-x 2 root mail 4096 Sep 17 02:20 /var/mail
$ find / -xdev -type d -perm -2000 -path '/var/*' 2>/dev/null | sort
/var/local
/var/log/journal
/var/log/redis
/var/mail
```

(Cuántos aparecen depende de lo que tengas instalado: `/var/mail`, `/var/local`, los logs del diario del sistema…)

11.4 · ¿Qué permisos necesita cada comando? La tabla maestra

Resumimos todo lo aprendido (capítulos 2 y 3) en **una sola tabla**. `F` = el fichero afectado, `D` = su carpeta, `D2` = la carpeta destino:

Quiero…	Comando	Permiso en el fichero	Permiso en la carpeta
Ver un fichero	`cat F`	`r`	`x` en `D`
Editar el contenido	`echo x >> F`	`w`	`x` en `D`
Ejecutarlo	`./F`	`x` (+ `r` si es script)	`x` en `D`
Copiar	`cp F D2/G`	`r` en `F`	`x` en `D`; `w`+`x` en `D2`
Mover/renombrar (misma carpeta)	`mv F G`	*ninguno*	`w`+`x` en `D`
Mover a otra carpeta	`mv F D2/`	*ninguno*	`w`+`x` en `D` **y** `D2`
Borrar	`rm F`	*ninguno*	`w`+`x` en `D` (y *sticky* si lo hay)
Crear	`touch G`	—	`w`+`x` en `D`
Crear un **enlace duro**	`ln F L`	ser dueño, o `r`+`w` (protección)	`w`+`x` en la carpeta de `L`
Crear un **enlace simbólico**	`ln -s F L`	*ninguno*	`w`+`x` en la carpeta de `L`
Listar	`ls D`	—	`r` (y `x` para `ls -l`)
Entrar	`cd D`	—	`x`
Cambiar permisos	`chmod …`	**ser el dueño**	`x` en `D`
Cambiar dueño	`chown …`	**ser** `root`	`x` en `D`

Tres experimentos que lo confirman:

① `cp` necesita poder LEER el origen:

```
$ sudo -u ana cp datos/secreto.txt /tmp/lab-copia-ana.txt
cp: cannot open 'datos/secreto.txt' for reading: Permission denied
$ sudo -u ana cp datos/agenda.txt /tmp/lab-copia-ana.txt
$ ls -l /tmp/lab-copia-ana.txt
-rw-r--r-- 1 ana ana 56 Oct 16 17:08 /tmp/lab-copia-ana.txt
$ rm -f /tmp/lab-copia-ana.txt
rm: cannot remove '/tmp/lab-copia-ana.txt': Operation not permitted
$ sudo rm -f /tmp/lab-copia-ana.txt
```

`secreto.txt` (`600`) no se puede leer → `cp` falla antes de crear nada. `agenda.txt` (`644`) sí. Fíjate en dos cosas: la copia pasa a ser de `ana` (quien la creó), aunque el original sea tuyo; y **tú no puedes borrarla** de `/tmp` (`Operation not permitted`): `/tmp` tiene el bit *sticky* y el fichero es de `ana`. Por eso hemos tenido que usar `sudo` para limpiar.

② `mv` y `rm` mandan por la carpeta, no por el fichero:

```
$ sudo touch fichero-de-root.txt
$ ls -l fichero-de-root.txt
-rw-r--r-- 1 root root 0 Oct 16 17:08 fichero-de-root.txt
$ mv fichero-de-root.txt renombrado.txt
$ rm -f renombrado.txt
$ ls fichero-de-root.txt renombrado.txt
ls: cannot access 'fichero-de-root.txt': No such file or directory
ls: cannot access 'renombrado.txt': No such file or directory
```

Un fichero que es de `root` (¡y que no puedes ni modificar!) **lo renombras y lo borras sin problema**, porque la **carpeta** es tuya.

③ Para crear en una carpeta hace falta `w` y `x`:

```
$ mkdir solo-lectura
$ chmod 555 solo-lectura
$ sudo -u ana touch solo-lectura/x
touch: cannot touch 'solo-lectura/x': Permission denied
$ touch solo-lectura/x
touch: cannot touch 'solo-lectura/x': Permission denied
$ chmod 755 solo-lectura
```

Sin `w`, **ni siquiera tú** (el dueño) puedes crear dentro.

11.5 · ⚠️ El `w` del fichero no te protege de `sed -i` ni de los editores

Esta es una de las cosas que más desconcierta. Un fichero `r--r--r--` (solo lectura) en una carpeta tuya:

```
$ echo "hola" > soloplantilla.txt
$ chmod 444 soloplantilla.txt
$ echo "añado" >> soloplantilla.txt
bash: soloplantilla.txt: Permission denied
$ sed -i 's/hola/adiós/' soloplantilla.txt
$ cat soloplantilla.txt
adiós
$ ls -l soloplantilla.txt
-r--r--r-- 1 alumno alumno 7 Oct 16 17:08 soloplantilla.txt
```

- `echo … >>` → `Permission denied` (abre el fichero para escribir, y no hay `w`).
- `sed -i` → **funciona**: no abre el fichero para escribir; **crea uno nuevo, lo rellena, y lo renombra** encima del viejo. Eso depende de la **carpeta** (tienes `w`+`x`), no del fichero.

Lo mismo hacen `vim` y otros editores al guardar (escriben un fichero nuevo y lo renombran). Consecuencias:

> 🎓 **Para proteger de verdad un fichero**, `chmod 444` **no basta** si la carpeta es escribible por quien no debe. Hay que proteger **la carpeta**, o usar el atributo inmutable (`chattr +i`, capítulo 12).

Y el *efecto colateral*: el fichero nuevo es **tuyo**, aunque el viejo fuera de otro. Lo vemos con uno de `root`:

```
$ sudo cp datos/agenda.txt de-root.txt
$ ls -l de-root.txt
-rw-r--r-- 1 root root 56 Oct 16 17:08 de-root.txt
$ sed -i 's/lunes/martes/' de-root.txt
$ ls -l de-root.txt
-rw-r--r-- 1 alumno alumno 57 Oct 16 17:08 de-root.txt
$ cat de-root.txt
Reunión el martes a las 10:00
Llevar el informe impreso
```

Un fichero de `root` (`644`) en una carpeta **tuya** puede acabar **siendo tuyo** tras un `sed -i`. (No es un fallo de seguridad: **la carpeta es tuya**, tú decides qué hay en ella.)

11.6 · Enlaces duros y simbólicos: ¿y sus permisos?

(LPIC-1 **104.6**.) Dos clases de "acceso directo", con reglas distintas:

	Enlace duro (`ln f l`)	**Enlace simbólico (`ln -s f l`)**
Qué es	**otro nombre** para el **mismo** fichero (mismo *inodo*)	un **fichero pequeño** que contiene la **ruta** de otro
Permisos propios	**los mismos** (son el mismo fichero)	`lrwxrwxrwx` siempre, **no cuentan**
¿Qué permisos manda?	los del **único fichero**	los del **destino**
Si borras el original	el dato sigue (queda otro nombre)	el enlace queda **roto**

Enlaces duros: son el mismo fichero

```
$ echo "contenido compartido" > original.txt
$ ln original.txt segundo-nombre.txt
$ chmod 600 segundo-nombre.txt
$ ls -li original.txt segundo-nombre.txt
599865 -rw------- 2 alumno alumno 21 Oct 16 17:08 original.txt
599865 -rw------- 2 alumno alumno 21 Oct 16 17:08 segundo-nombre.txt
```

Mismo **número de inodo** (primera columna), mismos permisos (`600`) y el **2** (número de enlaces). Cambiar permisos por un nombre los cambia por **el otro**: es un solo fichero con dos nombres.

🛡️ Protección de enlaces duros

Si cualquiera pudiera hacer un enlace duro a `/etc/passwd`, podría **conservar el fichero** aunque se "actualizase" o hacer trampas con SUID. Por eso Linux moderno tiene `fs.protected_hardlinks = 1`: solo puedes enlazar ficheros que **son tuyos** o que puedes **leer y escribir**:

```
$ cat /proc/sys/fs/protected_hardlinks
1
$ ln /etc/passwd mi-passwd
ln: failed to create hard link 'mi-passwd' => '/etc/passwd': Operation not permitted
$ sudo -u ana ln datos/agenda.txt /tmp/lab-enlace-ana
ln: failed to create hard link '/tmp/lab-enlace-ana' => 'datos/agenda.txt': Operation not permitted
```

Enlaces simbólicos: mandan los permisos del destino

```
$ ln -s datos/secreto.txt enlace-secreto
$ ls -l enlace-secreto
lrwxrwxrwx 1 alumno alumno 17 Oct 16 17:08 enlace-secreto -> datos/secreto.txt
$ cat enlace-secreto
La contraseña del wifi es: patata-frita-42
$ sudo -u carla cat enlace-secreto
cat: enlace-secreto: Permission denied
$ ln -s no-existe.txt enlace-roto
$ cat enlace-roto
cat: enlace-roto: No such file or directory
```

- El enlace es `lrwxrwxrwx` (¡777!): **no significa nada**.
- `carla` no puede leer `enlace-secreto` porque **el destino** (`secreto.txt` , `600`) no se lo permite.
- `enlace-roto` apunta a algo que no existe: `No such file or directory`.

11.7 · Las claves SSH: el caso donde los permisos "se hacen valer" 🔑

`ssh` **se niega a usar** una clave privada que otros puedan leer. Estas son las medidas habituales:

Fichero	Permisos	Si no…
`~/.ssh/`	`700`	`ssh` o `sshd` pueden ignorar el contenido
`~/.ssh/id_ed25519` (clave **privada**)	`600`	*"UNPROTECTED PRIVATE KEY FILE"*
`~/.ssh/id_ed25519.pub` (clave **pública**)	`644`	—
`~/.ssh/authorized_keys`	`600` (o `644`)	el servidor **rechaza** las claves
`~/.ssh/config`	`600`	`ssh` puede quejarse

Si dejas una clave privada con `644` (legible por el grupo y los demás), esto es lo que ocurre al usarla (salida **típica**; no se ejecutó en el laboratorio de la guía porque no lleva cliente `ssh`):

```
$ ls -l ~/.ssh/id_ed25519
-rw-r--r-- 1 alumno alumno 411 oct 16 12:00 /home/alumno/.ssh/id_ed25519
$ ssh servidor
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@         WARNING: UNPROTECTED PRIVATE KEY FILE!          @
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
Permissions 0644 for '/home/alumno/.ssh/id_ed25519' are too open.
It is required that your private key files are NOT accessible by others.
This private key will be ignored.
```

La solución: `chmod 600 ~/.ssh/id_ed25519` y `chmod 700 ~/.ssh`. Es una de las situaciones en las que **más veces** te toparás con permisos "en la vida real".

11.8 · Servicios y usuarios "de sistema"

Los **programas que corren solos** (servidor web, base de datos, cron...) **no usan tu usuario**: cada uno tiene el suyo, **sin contraseña ni shell**, para limitar el daño si lo atacan:

Servicio	Usuario típico	Los ficheros que escribe
Apache / Nginx	`www-data`	`/var/www`, `/var/log/apache2`
MySQL / MariaDB	`mysql`	`/var/lib/mysql` (modo `700`)
Registros (`rsyslog`)	`syslog` (grupo `adm`)	`/var/log/syslog` (`640`)

Si un servicio **no puede leer o escribir** algo, el síntoma es "Permission denied" en su **log**, y el remedio es **darle a ese usuario** (con `chown`/`chgrp`/SGID) lo que necesite, **no** hacer `chmod 777`.

11.9 · 🏋️ Ejercicios

🟢 Ejercicio 11.1 · Por qué `/etc/shadow` es 640

¿Quién puede leer `/etc/shadow`? ¿Quién puede escribirlo?

> 💡 **Solución**
>
> ```
> $ ls -l /etc/shadow
> -rw-r----- 1 root shadow 1485 Oct 16 17:08 /etc/shadow
> $ sudo -u carla cat /etc/shadow
> cat: /etc/shadow: Permission denied
> ```
>
> Dueño `root` (`rw-`): lee y escribe. Grupo `shadow` (`r--`): lee. Otros: nada. `carla` → `Permission denied`.

🟢 Ejercicio 11.2 · Tu `/home`

¿Quién puede listar tu carpeta personal? ¿Por qué `ana` no puede?

> 💡 **Solución**
>
> ```
> $ ls -ld ~
> drwxr-x--x 3 alumno alumno 4096 Oct 16 17:08 /home/alumno
> $ sudo -u ana ls ~
> ls: cannot open directory '/home/alumno': Permission denied
> ```
>
> Tu carpeta es `drwxr-x--x`: `ana` es "otros" con solo `--x` (puede **atravesarla**, no **listarla**).

🟡 Ejercicio 11.3 · Un `cp` que falla

`ana` quiere copiar `datos/secreto.txt` a `/tmp`. ¿Falla en la **lectura** o en la **escritura**? ¿Qué mensaje da?

> 💡 **Solución**

```
$ sudo -u ana cp datos/secreto.txt /tmp/lab-no.txt
cp: cannot open 'datos/secreto.txt' for reading: Permission denied
```

Falla en la **lectura** del origen (`cannot open … for reading`). Si fallara el destino diría `cannot create regular file` .

🟡 Ejercicio 11.4 · Renombrar un fichero ajeno

Renombra `fichero-de-root.txt` (de `root` , no modificable) a `tuyo.txt` . ¿Por qué se puede?

💡 Solución

```
$ sudo touch fichero-de-root.txt
$ mv fichero-de-root.txt tuyo.txt
$ ls -l tuyo.txt
-rw-r--r-- 1 root root 0 Oct 16 17:08 tuyo.txt
```

Renombrar es modificar **la carpeta** (que es tuya). Siguen siendo de `root` .

🟡 Ejercicio 11.5 · El mismo fichero, dos nombres

Haz un enlace duro de `datos/notas.txt` llamado `notas-bis.txt` . ¿Cuántos enlaces tiene? Cambia los permisos de uno y mira el otro.

💡 Solución

```
$ ln datos/notas.txt notas-bis.txt
$ ls -li datos/notas.txt notas-bis.txt
599861 -rw-r--r-- 2 alumno alumno 39 Oct 12 09:30 datos/notas.txt
599861 -rw-r--r-- 2 alumno alumno 39 Oct 12 09:30 notas-bis.txt
$ chmod 600 notas-bis.txt
$ ls -li datos/notas.txt notas-bis.txt
599861 -rw------- 2 alumno alumno 39 Oct 12 09:30 datos/notas.txt
599861 -rw------- 2 alumno alumno 39 Oct 12 09:30 notas-bis.txt
```

Mismo inodo, `2` enlaces, y el cambio de permisos se ve en **ambos** nombres.

🟡 Ejercicio 11.6 · ¿Quién manda en un enlace simbólico?

Crea `acceso-notas` → `datos/secreto.txt` . ¿Quién puede leerlo a través del enlace?

💡 Solución

```
$ ln -s datos/secreto.txt acceso-notas
$ ls -l acceso-notas
lrwxrwxrwx 1 alumno alumno 17 Oct 16 17:08 acceso-notas -> datos/secreto.txt
$ cat acceso-notas
La contraseña del wifi es: patata-frita-42
$ sudo -u ana cat acceso-notas
cat: acceso-notas: Permission denied
```

Solo el que puede leer **el destino** (`600`): tú. El `lrwxrwxrwx` del enlace es decorativo.

🔴 Ejercicio 11.7 · `chmod 444` no basta

Tienes `config.txt` `444` en una carpeta tuya. Demuestra que alguien con `w` en la **carpeta** puede cambiarlo igualmente. ¿Cómo protegerías el fichero de verdad?

💡 Solución

```
$ echo "valor=1" > config.txt
$ chmod 444 config.txt
$ sed -i 's/1/2/' config.txt
$ cat config.txt
valor=2
```

Se cambia porque `sed -i` **reemplaza** el fichero (permiso de la **carpeta**). Para protegerlo: quitar la `w` de la **carpeta** a quien no deba, o (`root`) `chattr +i config.txt` (capítulo 12).

🔴 Ejercicio 11.8 · Hard link a un fichero ajeno

¿Puede `ana` hacer un enlace duro (`ln`) a `datos/secreto.txt`? ¿Y a un fichero legible por todos como `datos/agenda.txt`? (`protected_hardlinks`)

💡 Solución

```
$ sudo -u ana ln datos/secreto.txt /tmp/lab-h1
ln: failed to create hard link '/tmp/lab-h1' => 'datos/secreto.txt': Operation not permitted
$ sudo -u ana ln datos/agenda.txt /tmp/lab-h2
ln: failed to create hard link '/tmp/lab-h2' => 'datos/agenda.txt': Operation not permitted
```

Ambos fallan: con `protected_hardlinks=1` solo puedes enlazar lo que **posees** o puedes **leer y escribir** a la vez. `agenda.txt` es de lectura para `ana`, pero no escritura.

• Ejercicio 11.9 · Examen: `/etc/shadow`

¿Qué permisos tiene normalmente `/etc/shadow` *?* a) `-rw-r--r--` b) `-rw-r-----` c) `-rw-------` d) `-rwxr-xr-x`

💡 Solución

b) `640`, con dueño `root` y grupo `shadow` en Debian/Ubuntu (en otras distribuciones es `000` o `600`).

• Ejercicio 11.10 · Examen: `/tmp`

¿Por qué los permisos de `/tmp` *son* `drwxrwxrwt` *?* a) Para que cualquiera pueda crear ficheros, pero solo el dueño pueda borrar los suyos b) Para que se ejecute como root c) Para heredar el grupo d) Para cifrar los ficheros

💡 Solución

a) `rwx` para todos + bit **sticky** (`t`).

• Ejercicio 11.11 · Examen: enlace simbólico

¿Qué permisos aparecen siempre en un enlace simbólico en `ls -l` *?* a) `lrwxrwxrwx` b) los mismos del destino c) `lrw-r--r--` d) `l---------`

💡 Solución

a) `lrwxrwxrwx`, y no sirven para nada: mandan los del destino.

Capítulo 12 · Más allá de `rwx`: atributos, ACL, *capabilities* y montajes

> 🎯 **Objetivo:** saber que `rwx` **no es la única barrera**. Conocer los **atributos de fichero** (`chattr +i` te protege hasta de `root`), las **opciones de montaje** (`nosuid`, `noexec`, `ro`), las **capabilities** (el poder de `root` troceado), las **ACL** (permisos para usuarios concretos) y el **MAC** (AppArmor/SELinux). Y tener un **mapa mental** de todas las capas.
>
> 📘 **LPIC-1:** 104.5 y 110.1 (conceptos); algunas partes (ACL, *capabilities*) son **extra** y se profundizan en LPIC-2, pero **las encontrarás en la vida real**.
>
> 🧪 **Antes de empezar:** `cd ~/lab-permisos`.

12.1 · Los límites de `rwx`

Los permisos de usuario/grupo/otros son simples y potentes, pero tienen huecos:

Problema	Solución
*"Quiero que **proteger un fichero hasta de** `root` (o de un `rm -rf` accidental)"*	**atributos** (`chattr +i`)
*"Quiero dar acceso a **un usuario concreto** que no es dueño ni del grupo"*	**ACL** (`setfacl`)
*"Quiero que un programa tenga **solo una parte** del poder de `root`"*	**capabilities** (`setcap`)
*"Quiero que **esta partición** no pueda ejecutar programas ni SUID"*	**opciones de montaje** (`noexec`, `nosuid`)
"Quiero confinar a un servicio aunque tenga `rwx`"	**MAC**: AppArmor / SELinux

12.2 · Atributos de fichero: `lsattr` y `chattr` 🔒

En los sistemas de ficheros **ext2/3/4** (y otros), cada fichero tiene, además de los permisos, unos **atributos** (banderas). Se ven con `lsattr` y se cambian con `chattr`:

```
$ echo "contenido" > importante.txt
$ lsattr importante.txt
--------------e------- importante.txt
```

Los 20 caracteres son banderas; la `e` (*extents*) es normal en ext4. Las **que debes conocer**:

Flag	Nombre	Efecto
`i`	**inmutable**	el fichero **no se puede modificar, borrar, renombrar, enlazar ni cambiar de permisos**, **ni por** `root`
`a`	**solo-añadir** (*append only*)	solo se puede **añadir** al final (`>>`); nunca sobrescribir ni borrar
`A`	sin actualizar `atime`	menos escrituras en disco
`d`	no volcar	`dump` lo ignora
`s`	borrado seguro	rellena de ceros al borrar (no siempre soportado)
`S`	síncrono	las escrituras van directas al disco

> 🛑 Solo `root` puede poner o quitar `i` y `a` (necesita la capacidad `CAP_LINUX_IMMUTABLE`). Un usuario normal: `Operation not permitted`.

① Inmutable (`+i`): protegido hasta de `root`

```
$ chattr +i importante.txt
chattr: Operation not permitted while setting flags on importante.txt
$ sudo chattr +i importante.txt
$ lsattr importante.txt
----i---------e------- importante.txt
$ echo "cambio" >> importante.txt
bash: importante.txt: Operation not permitted
$ rm -f importante.txt
rm: cannot remove 'importante.txt': Operation not permitted
$ mv importante.txt otro.txt
mv: cannot move 'importante.txt' to 'otro.txt': Operation not permitted
$ chmod 777 importante.txt
chmod: changing permissions of 'importante.txt': Operation not permitted
$ ln importante.txt enlace-duro.txt
ln: failed to create hard link 'enlace-duro.txt' => 'importante.txt': Operation not permitted
$ sudo rm -f importante.txt
rm: cannot remove 'importante.txt': Operation not permitted
$ sudo sh -c 'echo "root tampoco" >> importante.txt'
sh: 1: cannot create importante.txt: Operation not permitted
```

Ni escribir, ni borrar, ni renombrar, ni cambiar permisos, ni hacer un enlace duro... **ni siquiera** `root` hasta que quite el atributo:

```
$ sudo chattr -i importante.txt
$ lsattr importante.txt
--------------e------- importante.txt
$ rm -f importante.txt
$ ls importante.txt
ls: cannot access 'importante.txt': No such file or directory
```

> 🎓 **Usos reales:** proteger `/etc/resolv.conf`, `/etc/passwd` en servidores críticos o ficheros que un malware tocaría. Y **¡cuidado!** también es una trampa: *"no puedo borrar este fichero ni siendo* `root` *"* → `lsattr` y busca la `i`.

② Solo-añadir (`+a`): ideal para registros

```
$ echo "línea 1" > registro.log
$ sudo chattr +a registro.log
$ lsattr registro.log
-----a--------e------- registro.log
$ echo "línea 2" >> registro.log
$ echo "sobrescribir" > registro.log
bash: registro.log: Operation not permitted
$ rm -f registro.log
rm: cannot remove 'registro.log': Operation not permitted
$ cat registro.log
línea 1
línea 2
$ sudo chattr -a registro.log
```

Se puede **añadir** (`>>`), pero ni **sobrescribir** (`>`) ni **borrar**. Un atacante con acceso no puede "limpiar" las huellas del log.

12.3 · Opciones de montaje: la barrera de la partición

Los permisos de un fichero **no valen** si la **partición** donde vive está montada con restricciones. Se declaran al montarla (en `/etc/fstab`) y mandan **por encima** de los permisos:

Opción	Efecto sobre todo lo que hay en esa partición
`ro`	solo lectura: nadie escribe (ni `root`)
`noexec`	**no se puede ejecutar** nada (aunque tenga `x`)
`nosuid`	**se ignoran** SUID y SGID (¡un `4755` deja de funcionar!)

Opción	Efecto sobre todo lo que hay en esa partición
`nodev`	se ignoran los ficheros de dispositivos (`b` , `c`)
`noatime`	no actualiza la fecha de acceso

Se miran con `findmnt` o `mount`. Esto es lo que tiene el sistema de ficheros del laboratorio:

```
$ findmnt -no OPTIONS / | tr ',' '\n'
rw
relatime
discard
no_prefetch_block_bitmaps
resv_strict
resuid=65534
resgid=65534
```

Una **partición** `/tmp` **endurecida** es típica de un servidor: `tmpfs /tmp tmpfs defaults,noexec,nosuid,nodev 0 0` en `/etc/fstab`. Con eso, aunque alguien consiga subir un programa a `/tmp`, **no se podrá ejecutar**.

> 🎓 **Examen:** un SUID `4755` en una partición montada con `nosuid` **no tiene efecto**. Y una unidad USB con ficheros `x` pero montada `noexec` **no deja** ejecutarlos (el famoso *"Permission denied"* al lanzar un script de un pendrive: ¡no es culpa de `chmod` !).

12.4 · *Capabilities*: el poder de `root` en trocitos 🧩

Históricamente "ser `root` " era **todo o nada**. Linux lo trocea en unas **40 capacidades** (*capabilities*), como `CAP_NET_BIND_SERVICE` (usar puertos < 1024), `CAP_CHOWN` (cambiar dueños), `CAP_DAC_OVERRIDE` (saltarse los permisos de lectura/escritura/ejecución) o `CAP_LINUX_IMMUTABLE` (usar `chattr`). Un programa puede recibir **solo las que necesita**, en vez de ser SUID `root` :

```
$ cp /usr/bin/cat micat
$ getcap micat
$ cat /etc/shadow
cat: /etc/shadow: Permission denied
$ ./micat /etc/shadow
./micat: /etc/shadow: Permission denied
```

(Las dos fallan adrede: ni `cat` ni nuestra copia `micat` pueden leer `/etc/shadow`, que es `root:shadow 640` ; `getcap` no muestra nada: aún no tiene ninguna capacidad.) Ahora **damos una capacidad concreta** al *programa* `micat` :

```
$ sudo setcap cap_dac_read_search+ep micat
$ getcap micat
micat cap_dac_read_search=ep
$ ./micat /etc/shadow | head -n 1 | cut -c1-5
root:
$ sudo setcap -r micat
$ getcap micat
$ ./micat /etc/shadow
./micat: /etc/shadow: Permission denied
```

Con `cap_dac_read_search` el programa puede **leer** cualquier fichero aunque no tenga `r` (aquí solo imprimimos `root:` de la primera línea para no mostrar contraseñas ajenas). Con `setcap -r` se la quitamos y vuelve el `Permission denied` .

> ⚠️ **Es potente y peligroso** (¡un `cat` con esa capacidad lee `/etc/shadow` !). Por eso **auditar** capacidades es tan importante como auditar SUID: `getcap -r / 2>/dev/null` . Un ejemplo típico y legítimo: `ping` tiene `cap_net_raw=ep` en lugar de ser SUID.
>
> En un `ls -l` **no** se ven las capacidades (no hay ninguna marca): hay que preguntar con `getcap` .

12.5 · ACL: permisos "a la carta" por usuario *(extra, no ejecutado en el laboratorio)*

> 📝 **Nota honesta:** en el entorno donde se generaron las salidas de esta guía **no estaban instaladas** las herramientas `getfacl` / `setfacl`, así que **esta sección no se ejecutó**. Los comandos y la salida son los **habituales**; compruébalos en tu máquina (`sudo apt install acl` si te falta el paquete). Las ACL **no** son parte del examen LPIC-1.

Con `rwx` solo hay **un** dueño y **un** grupo. Una **ACL** (*Access Control List*) permite añadir **más usuarios y grupos con permisos propios**:

```
$ touch informe.txt
$ setfacl -m u:ana:rw informe.txt        # a ana, lectura y escritura
$ setfacl -m g:soporte:r informe.txt     # al grupo soporte, solo lectura
$ getfacl informe.txt
# file: informe.txt
# owner: alumno
# group: alumno
user::rw-
user:ana:rw-
group::r--
group:soporte:r--
mask::rw-
other::r--

$ ls -l informe.txt
-rw-rw-r--+ 1 alumno alumno 0 oct 16 12:00 informe.txt
```

Fíjate en el `+` al final de los permisos: es la señal de que el fichero tiene ACL. Qué significa cada línea de `getfacl`:

Línea	Significa
`user::rw-`	permisos del **dueño**
`user:ana:rw-`	permisos de **ana** (entrada extra)
`group::r--`	permisos del **grupo propietario**
`group:soporte:r--`	permisos del **grupo** `soporte` (entrada extra)
`mask::rw-`	el **máximo** que pueden tener las entradas extra y el grupo
`other::r--`	los demás

La **máscara** es el truco: `chmod` sobre un fichero con ACL **cambia la máscara** (y por eso `ls -l` muestra la máscara donde antes estaba el grupo).

Los comandos esenciales:

Quiero…	Comando
Ver la ACL	`getfacl fichero`
Dar permisos a un usuario	`setfacl -m u:ana:rw fichero`
Dar permisos a un grupo	`setfacl -m g:soporte:rx fichero`
Quitar la entrada de ana	`setfacl -x u:ana fichero`
Quitar toda la ACL	`setfacl -b fichero`
Recursivo	`setfacl -R -m u:ana:rX carpeta`
ACL por defecto (hereda lo que se cree dentro de una carpeta)	`setfacl -d -m g:equipo:rwx carpeta`

> 💡 Las ACL **por defecto** hacen en una carpeta lo que el SGID + `umask` pero **mucho mejor**: puedes decidir qué permisos reciben automáticamente **todos** los ficheros nuevos, para **cada** usuario/grupo.

12.6 · MAC: AppArmor y SELinux (cuando `rwx` no basta)

Todo lo anterior es **DAC** (*Discretionary Access Control*): **el dueño decide**. Linux añade **MAC** (*Mandatory Access Control*): **una política del sistema** que se aplica **aunque** `rwx` **diga que sí**:

	AppArmor	SELinux
Distribuciones	**Ubuntu**, Debian, openSUSE	RHEL, Fedora, CentOS, Rocky
Cómo funciona	**perfiles por programa** (qué rutas puede tocar)	**etiquetas** en cada fichero/proceso
Estado	`sudo aa-status`	`getenforce` / `sestatus`
Se nota en `ls -l`	no	con `ls -lZ` (y un `.` al final de los permisos)
Dónde mirar los bloqueos	`journalctl -k | grep -i apparmor`	`ausearch -m avc` / `/var/log/audit/audit.log`

> 🧠 **Si un servicio tiene `rwx` correcto y aun así falla con *"Permission denied"*, sospecha del MAC** (AppArmor/SELinux). Es la causa clásica de *"he puesto 777 y sigue sin ir"*. La solución **no** es abrir permisos: es ajustar la política.

12.7 · 🗺️ El mapa completo: todas las "puertas" por las que pasa un acceso

Cuando un programa intenta abrir un fichero, el kernel pasa por **varias comprobaciones**. Si **cualquiera** dice que no, `Permission denied`:

```
┌─ 1. RUTA ──────────────── ¿hay "x" en TODAS las carpetas del camino?      (cap. 3)
│
├─ 2. MONTAJE ────────────── ¿partición "ro", "noexec", "nosuid"?           (12.3)
│
├─ 3. ATRIBUTOS ──────────── ¿"i" inmutable, "a" solo-añadir?               (12.2)
│
├─ 4. PERMISOS DAC ──────── dueño / grupo / otros (+ ACL)                   (cap. 4)
│       └─ y los bits especiales SUID / SGID / sticky                       (cap. 10)
│
├─ 5. CAPABILITIES ──────── ¿el proceso tiene CAP_DAC_OVERRIDE, etc.?       (12.4)
│
└─ 6. MAC ──────────────── ¿AppArmor / SELinux lo permite?                  (12.6)
```

Este mapa será tu **lista de comprobación** en el capítulo 13, cuando toque diagnosticar un *"Permission denied"*.

12.8 · 🏋️ Ejercicios

🟢 Ejercicio 12.1 · El fichero que ni `root` puede borrar

Crea `sagrado.txt`, hazlo inmutable y demuestra que `root` no lo puede borrar. Después recupéralo.

> 💡 **Solución**

```
$ echo "no tocar" > sagrado.txt
$ sudo chattr +i sagrado.txt
$ lsattr sagrado.txt
----i---------e------- sagrado.txt
$ sudo rm -f sagrado.txt
rm: cannot remove 'sagrado.txt': Operation not permitted
$ sudo chattr -i sagrado.txt
$ rm -f sagrado.txt
$ ls sagrado.txt
ls: cannot access 'sagrado.txt': No such file or directory
```

🟢 Ejercicio 12.2 · ¿Qué atributos tiene?

¿Cómo ves los atributos de `datos/agenda.txt`? ¿Qué significa la `e`?

💡 Solución

```
$ lsattr datos/agenda.txt
--------------e------- datos/agenda.txt
```

La `e` quiere decir que usa *extents* (la forma moderna de almacenar ficheros en ext4). Es normal.

🟡 Ejercicio 12.3 · Un log a prueba de manipulaciones

Crea `auditoria.log` de modo que se pueda **añadir** pero **no sobrescribir ni borrar**. Pruébalo.

💡 Solución

```
$ echo "inicio" > auditoria.log
$ sudo chattr +a auditoria.log
$ echo "evento 1" >> auditoria.log
$ echo "borrado" > auditoria.log
bash: auditoria.log: Operation not permitted
$ rm -f auditoria.log
rm: cannot remove 'auditoria.log': Operation not permitted
$ cat auditoria.log
inicio
evento 1
$ sudo chattr -a auditoria.log
```

🟡 Ejercicio 12.4 · No me deja borrar un fichero... ¡siendo root!

Un compañero dice que `sudo rm fichero` da `Operation not permitted`. ¿Cuál es tu primer comando para investigar?

💡 Solución

`lsattr fichero`: seguro que tiene la `i` (inmutable) o la `a`. Se quita con `sudo chattr -i fichero`. (Otras causas: partición montada `ro`; el fichero es el **punto de montaje**; un fichero "en uso" no impide borrar en Linux.)

🟡 Ejercicio 12.5 · Un USB que no ejecuta

Copias un script a un USB, le haces `chmod +x`, y al lanzarlo: `Permission denied`. ¿Por qué? ¿Cómo lo compruebas?

💡 Solución

El pendrive probablemente está montado con `noexec`. Se comprueba con `findmnt /media/tu-usb` o `mount | grep usb`. Soluciones: ejecutarlo con `bash script.sh` (el intérprete sí se ejecuta) o copiarlo a tu carpeta.

🔴 Ejercicio 12.6 · Una capacidad sin SUID

Explica por qué dar `cap_net_bind_service` a un servidor web es **mejor** que hacerlo SUID `root`.

> **💡 Solución**
>
> Con SUID `root`, el servidor tiene **todo** el poder de `root` mientras se ejecuta: un fallo → el atacante es `root`. Con `cap_net_bind_service` solo puede **abrir puertos bajos** y nada más (*principio de mínimo privilegio*). (`sudo setcap cap_net_bind_service=+ep /usr/bin/miservidor`.)

🔴 Ejercicio 12.7 · El SUID que no funciona en `/media`

Un programa SUID `root` copiado a `/media/usb` no da privilegios. ¿Por qué?

> **💡 Solución**
>
> Porque los medios extraíbles suelen montarse `nosuid`: el kernel ignora SUID/SGID de **todos** los ficheros de esa partición.

• Ejercicio 12.8 · Examen: `chattr`

*¿Qué comando impide que un fichero sea modificado o borrado, **incluso por** `root`?* a) `chmod 000 f` b) `chown root f` c) `chattr +i f` d) `chmod +t f`

> **💡 Solución**
>
> **c)** `chattr +i f` (atributo **i**nmutable). `chmod 000` no detiene a `root`.

• Ejercicio 12.9 · Examen: `+` al final

En `-rw-rw-r--+`, ¿qué indica el `+`? a) Tiene el sticky b) Tiene **ACL** c) Es un enlace d) Es ejecutable

> **💡 Solución**
>
> **b)**: el fichero tiene una **ACL** (lista de control de acceso). Se ve con `getfacl`.

• Ejercicio 12.10 · Examen: la `nosuid`

Un fichero `-rwsr-xr-x` de `root` no da privilegios a quien lo ejecuta. ¿Una posible causa? a) El grupo es incorrecto b) La partición está montada con `nosuid` c) Falta la `x` de otros d) Está en `/tmp`

> **💡 Solución**
>
> **b)**. (Y recuerda: si fuera un **script**, también se ignoraría; capítulo 10.)

Capítulo 13 · Diagnóstico: "¡Permission denied!" y qué hacer · Recetas

> 🎯 **Objetivo:** ante un fallo de permisos, **no adivinar ni tirar de** `chmod 777`, sino **diagnosticar con método**: leer bien el mensaje, localizar **qué elemento exacto** bloquea (`namei`, `diagnostica`, `strace`) y arreglarlo con **el cambio mínimo**. Más un **recetario** de situaciones típicas.
>
> 📘 **LPIC-1:** 104.5 · 110.1 · 103.1 (leer errores).
>
> 🧪 **Antes de empezar:** `cd ~/lab-permisos`. Instala las herramientas de la guía si no lo hiciste (capítulo 5): esta vez también necesitamos `diagnostica`.

13.1 · Primero, lee el mensaje: no todos los errores son "permisos"

Casi la mitad de los "problemas de permisos" **no** son de permisos. El texto del error te dice casi siempre por dónde ir:

Mensaje (inglés / español)	Qué significa	Primera comprobación
`Permission denied` / *Permiso denegado*	te **falta un permiso** (rwx), en el fichero o en el camino; o `noexec`/MAC	`ls -ld`, `namei -l`, `id`
`Operation not permitted` / *Operación no permitida*	acción **reservada** (no es cuestión de rwx): `chown` sin ser root, `chmod` sin ser dueño, **sticky**, **inmutable**	¿soy dueño/root? `lsattr`
`No such file or directory` / *No existe el fichero...*	la **ruta no existe**... o es un enlace roto... o el **intérprete** del *shebang* no existe	`ls`, `readlink -f`, `head -1 script`
`command not found` / *orden no encontrada*	no está en el `PATH` (¡no es de permisos!)	`echo $PATH`, `./script`
`Is a directory`	intentas leer/ejecutar una carpeta como un fichero	`ls -ld`
`Not a directory`	usas un fichero como si fuera carpeta (`cat f/x`)	`ls -ld`
`Read-only file system`	la partición está **montada** `ro`	`findmnt -T ruta`
`Text file busy`	intentas **escribir sobre un programa que se está ejecutando**	`fuser`, `lsof`
`cannot execute: required file not found` (bash 5.2, Ubuntu 24.04) o `bad interpreter` (bash antiguos)	la primera línea (`#!`) apunta a un programa que no existe (o lleva un `\r` de Windows)	`head -1 script | cat -A`
`cannot execute binary file: Exec format error`	no es un ejecutable válido (o es de otra arquitectura)	`file fichero`

Las **dos primeras** son las que importan para esta guía y se confunden mucho:

> 🔑 `Permission denied` = **"te falta un permiso"** (se arregla dándolo). `Operation not permitted` = **"esto no se puede hacer así, ni con permisos"** (hay que ser otro usuario o quitar una protección).

Demostraciones de los que no son de permisos

```
$ printf '#!/bin/bash\necho hola\n' > ok.sh
$ chmod +x ok.sh
$ ok.sh
bash: ok.sh: command not found
$ ./ok.sh
hola
$ ./datos
bash: ./datos: Is a directory
$ cat cerve.sh/x
cat: cerve.sh/x: Not a directory
$ cd cerve.sh
bash: cd: cerve.sh: Not a directory
```

- `ok.sh` a secas → `command not found`: el script **no** está en el `PATH` (el directorio actual no está). Con `./ok.sh` sí.
- `./datos` → `Is a directory`.
- `cat cerve.sh/x` y `cd cerve.sh` → `Not a directory` (`cerve.sh` es un fichero).

```
$ printf '#!/bin/noexiste\necho hola\n' > malo.sh
$ chmod +x malo.sh
$ ./malo.sh
bash: ./malo.sh: cannot execute: required file not found
$ printf '#!/bin/bash\r\necho hola\r\n' > windows.sh
$ chmod +x windows.sh
$ ./windows.sh
bash: ./windows.sh: cannot execute: required file not found
$ printf 'MZ\000\001' > falso.bin
$ chmod +x falso.bin
$ ./falso.bin
bash: ./falso.bin: cannot execute binary file: Exec format error
```

- `malo.sh` → `cannot execute: required file not found` (en Ubuntu 22.04 o anteriores verías `/bin/noexiste: bad interpreter: No such file or directory`): el fichero **sí existe**, pero **el programa de la primera línea** no.
- `windows.sh` → el **mismo error**: el script se guardó con finales de línea de Windows (`\r\n`) y el intérprete que se busca es `/bin/bash^M` (`^M` = el `\r` invisible). Solución: `sed -i 's/\r$//' windows.sh` (o `dos2unix`). Para verlo: `head -1 windows.sh | cat -A`.
- `falso.bin` → `Exec format error`: tiene `x`, pero su contenido no es un ejecutable.

> 🎓 Si el error **no** es de permisos, **cambiar permisos no lo arreglará**. Y *"el script existe pero dice que no existe"* casi siempre es el *shebang* o el `\r` de Windows.

13.2 · 🧭 El método de 7 pasos

Cuando algo falla con `Permission denied`, sigue **siempre** este orden (es el mapa del capítulo 12):

#	Pregunta	Herramienta
1	¿**Qué dice exactamente** el error y **sobre qué ruta**?	leer el mensaje completo
2	¿**Quién soy** (o quién es el proceso)? ¿qué **grupos** tengo?	`id`, `id usuario`
3	¿Puedo **llegar** (x en **todas** las carpetas del camino)?	`namei -l ruta`
4	¿Qué **permisos, dueño y grupo** tiene el objeto?	`ls -ld ruta`, `stat ruta`
5	¿**Qué trozo** me toca (dueño / grupo / otros) y qué le falta?	el algoritmo del capítulo 4 → `diagnostica`
6	¿Hay algo **por encima**? ACL, atributos, `noexec/ro`, capacidades, AppArmor/SELinux	`getfacl`, `lsattr`, `findmnt -T`, `getcap`, `aa-status`
7	**Arreglar con el cambio mínimo**: un `chmod` concreto, un `chgrp`, una ACL... **¡nunca `777`!**	`chmod`, `chgrp`, `usermod -aG` + volver a entrar

13.3 · 🛠️ Las herramientas

`namei -l`: ¿en qué tramo se cae?

```
$ namei -l /home/alumno/lab-permisos/datos/secreto.txt
f: /home/alumno/lab-permisos/datos/secreto.txt
drwxr-xr-x root   root   /
drwxr-xr-x root   root   home
drwxr-x--x alumno alumno alumno
drwxr-xr-x alumno alumno lab-permisos
drwxr-xr-x alumno alumno datos
-rw------- alumno alumno secreto.txt
```

Recorre **todos** los tramos y te enseña dueño, grupo y permisos de cada uno. Busca la carpeta que **no** deja pasar (te falta la `x`).

`diagnostica`: el algoritmo del capítulo 4, tramo a tramo

La herramienta `diagnostica` (del directorio `herramientas/`) aplica la lógica exacta del sistema para un usuario y te dice **qué tramo bloquea**. (Se instala igual que `permisos`; se ejecuta con `sudo` para poder mirar carpetas de otros.)

```
sudo install -m 755 ~/Guias-y-Herramientas/guias/permisos-linux-lpic1/herramientas/diagnostica
/usr/local/bin/diagnostica
```

Caso ①: `carla` quiere leer `secreto.txt`

```
$ sudo diagnostica -u carla /home/alumno/lab-permisos/datos/secreto.txt
Usuario: carla (uid 1004)   Grupos: carla soporte
Ruta:    /home/alumno/lab-permisos/datos/secreto.txt

  Tramo                                    Permisos    Dueño:Grupo     Aplica rwx  ¿Puede?
  /                                        drwxr-xr-x  root:root        otros  r-x  ✔ puede atravesarlo
  /home                                    drwxr-xr-x  root:root        otros  r-x  ✔ puede atravesarlo
  /home/alumno                             drwxr-x--x  alumno:alumno    otros  --x  ✔ puede atravesarlo
  /home/alumno/lab-permisos                drwxr-xr-x  alumno:alumno    otros  r-x  ✔ puede atravesarlo
  /home/alumno/lab-permisos/datos          drwxr-xr-x  alumno:alumno    otros  r-x  ✔ puede atravesarlo
  /home/alumno/lab-permisos/datos/secreto.txt  -rw-------  alumno:alumno    otros  ---  (el objeto final: ver
abajo)

Camino libre hasta el objeto. Sobre /home/alumno/lab-permisos/datos/secreto.txt, carla puede:
   ✘ leer (r)
   ✘ escribir (w)
   ✘ ejecutar (x)

Sin avisos de ACL, atributos inmutables ni opciones de montaje restrictivas.
(Recuerda además el MAC —AppArmor/SELinux— y que los grupos nuevos exigen volver a iniciar sesión.)
```

Léelo: el **camino está libre** (cada carpeta le da la `x` a "otros"), pero el **objeto final** es `rw-------`: para `carla` ("otros") `---`: ✘ leer, ✘ escribir, ✘ ejecutar.

Caso ②: ahora cerramos tu carpeta personal y `ana` prueba a leer un fichero que es legible por todos

```
$ chmod o-x ~
$ sudo diagnostica -u ana /home/alumno/lab-permisos/datos/agenda.txt
Usuario: ana (uid 1002)   Grupos: ana equipo
Ruta:    /home/alumno/lab-permisos/datos/agenda.txt

  Tramo                                    Permisos    Dueño:Grupo     Aplica rwx  ¿Puede?
  /                                        drwxr-xr-x  root:root        otros  r-x  ✔ puede atravesarlo
  /home                                    drwxr-xr-x  root:root        otros  r-x  ✔ puede atravesarlo
  /home/alumno                             drwxr-x---  alumno:alumno    otros  ---  ✘ FALTA x para
atravesarlo
  /home/alumno/lab-permisos                drwxr-xr-x  alumno:alumno    otros  r-x  ✔ puede atravesarlo
  /home/alumno/lab-permisos/datos          drwxr-xr-x  alumno:alumno    otros  r-x  ✔ puede atravesarlo
  /home/alumno/lab-permisos/datos/agenda.txt  -rw-r--r--  alumno:alumno    otros  r--  (el objeto final: ver
abajo)

✘ BLOQUEO en: /home/alumno  (otros: ---)
  Sin poder atravesar ese tramo, ana no llega al objeto: NO puede hacer nada con él.
  Remedio: dar la x que falta en esa carpeta (chmod o+x / g+x, o añadir al usuario al grupo).

Sin avisos de ACL, atributos inmutables ni opciones de montaje restrictivas.
(Recuerda además el MAC —AppArmor/SELinux— y que los grupos nuevos exigen volver a iniciar sesión.)
$ chmod o+x ~
```

El fichero es `644`, ¡pero `ana` no puede leerlo! La herramienta señala el **bloqueo**: la carpeta `/home/alumno` (`drwxr-x---`): a "otros" **les falta la** `x`. Esto es el *"he puesto 644 y no se lee"* del 90 % de los casos reales. (Y ya hemos devuelto el `o+x`.)

Caso ③: un usuario del grupo

```
$ echo "informe" > datos/equipo.txt
$ sudo chgrp equipo datos/equipo.txt
$ chmod 640 datos/equipo.txt
$ sudo diagnostica -u beto datos/equipo.txt
Usuario: beto (uid 1003)   Grupos: beto equipo soporte
Ruta:    /home/alumno/lab-permisos/datos/equipo.txt

  Tramo                                    Permisos    Dueño:Grupo     Aplica rwx  ¿Puede?
  /                                        drwxr-xr-x  root:root        otros  r-x  ✔ puede atravesarlo
  /home                                    drwxr-xr-x  root:root        otros  r-x  ✔ puede atravesarlo
  /home/alumno                             drwxr-x--x  alumno:alumno    otros  --x  ✔ puede atravesarlo
  /home/alumno/lab-permisos                drwxr-xr-x  alumno:alumno    otros  r-x  ✔ puede atravesarlo
  /home/alumno/lab-permisos/datos          drwxr-xr-x  alumno:alumno    otros  r-x  ✔ puede atravesarlo
  /home/alumno/lab-permisos/datos/equipo.txt  -rw-r-----  alumno:equipo    grupo  r--  (el objeto final: ver
abajo)

Camino libre hasta el objeto. Sobre /home/alumno/lab-permisos/datos/equipo.txt, beto puede:
   ✔ leer (r)
   ✘ escribir (w)
   ✘ ejecutar (x)

Sin avisos de ACL, atributos inmutables ni opciones de montaje restrictivas.
(Recuerda además el MAC —AppArmor/SELinux— y que los grupos nuevos exigen volver a iniciar sesión.)
```

`beto` es de `equipo` → trozo **grupo** (`r--`) → puede **leer** pero no escribir ni ejecutar.

Caso ④: tú, con un fichero del sistema

```
$ diagnostica /etc/shadow
Usuario: alumno (uid 1001)   Grupos: alumno
Ruta:    /etc/shadow

  Tramo        Permisos    Dueño:Grupo     Aplica rwx  ¿Puede?
  /            drwxr-xr-x  root:root        otros  r-x  ✔ puede atravesarlo
  /etc         drwxr-xr-x  root:root        otros  r-x  ✔ puede atravesarlo
  /etc/shadow  -rw-r-----  root:shadow      otros  ---  (el objeto final: ver abajo)

Camino libre hasta el objeto. Sobre /etc/shadow, alumno puede:
   ✘ leer (r)
   ✘ escribir (w)
   ✘ ejecutar (x)

Sin avisos de ACL, atributos inmutables ni opciones de montaje restrictivas.
(Recuerda además el MAC —AppArmor/SELinux— y que los grupos nuevos exigen volver a iniciar sesión.)
```

(Aquí no hace falta `sudo`: `/etc/shadow` es visible para todos los usuarios; lo que está protegido es su **contenido**.)

> 🧠 La herramienta **no** evalúa ACL, atributos, `noexec`, capacidades ni AppArmor: los **lista como avisos** al final (ACL, inmutable, `ro`, `noexec`, `nosuid`). Pero en el 90 % de los casos el culpable está en la tabla de permisos.

`strace`: ver el fallo exacto, sin adivinar (nivel *pro*)

Cuando un programa dice *"Permission denied"* **sin decir qué fichero**, `strace` te lo cuenta: te enseña **cada llamada al sistema** y su resultado.

```
$ sudo -u carla strace -f -e trace=openat cat datos/secreto.txt 2>&1 | grep -E 'secreto|EACCES'
openat(AT_FDCWD, "datos/secreto.txt", O_RDONLY) = -1 EACCES (Permission denied)
cat: datos/secreto.txt: Permission denied
```

Ahí está: `openat(…, "datos/secreto.txt", O_RDONLY) = -1 EACCES (Permission denied)`: `EACCES` **es** `Permission denied`. (`strace` quizá haya que instalarlo: `sudo apt install strace`.) Con un servicio que falla (`strace -f -e trace=file -p PID`), el fichero culpable aparece en la última línea con `EACCES`.

13.4 · Casos de la vida real

🗒️ Caso A · "Mi script no se ejecuta"

Síntoma	Causa	Arreglo
`bash: ./s.sh: Permission denied`	falta la `x`	`chmod +x s.sh`
`s.sh: command not found`	no está en el `PATH`	`./s.sh` o ponerlo en `~/bin`
`cannot execute: required file not found` / `bad interpreter`	*shebang* mal o `\r` de Windows	`head -1 s.sh | cat -A`; `sed -i 's/\r$//' s.sh`
`Permission denied` aun con `x`	`noexec` en la partición; o falta `r`	`findmnt -T .`; `bash s.sh`

🗒️ Caso B · "Mi compañero no puede editar lo que yo creo en la carpeta compartida"

Causas típicas (¡las dos a la vez!): **(1)** el fichero nace con el **grupo equivocado** (falta **SGID** en la carpeta, capítulo 10) y **(2)** nace sin `w` de grupo (`umask 022` en vez de `002`, capítulo 8).

```
sudo chgrp equipo carpeta
sudo chmod 2775 carpeta          ← SGID: lo nuevo hereda el grupo
# y en la sesión de cada miembro:  umask 002
```

🗒️ Caso C · "El servidor web da *403 Forbidden*"

Su usuario (`www-data`) necesita `x` **en TODAS las carpetas del camino** y `r` **en el fichero**. Lo más típico es la **carpeta personal** cerrada (`750`) o una carpeta intermedia sin `x`. Diagnóstico: `sudo diagnostica -u www-data /home/pepe/web/index.html` (o `namei -l`).

🗒️ Caso D · "Después de `usermod -aG` sigue sin funcionar"

Los grupos se leen **al iniciar sesión** (capítulo 4): cierra la sesión y vuelve a entrar, o usa `newgrp grupo`. Comprueba con `id` (no `id usuario`).

🗒️ Caso E · "No puedo borrar este fichero"

Mensaje	Causa	Arreglo
`Permission denied`	no tienes `w` + `x` en la **carpeta**	`chmod u+wx carpeta`
`Operation not permitted` en `/tmp` o carpeta compartida	**sticky**: el fichero es de otro	pídeselo al dueño o usa `sudo`
`Operation not permitted` **incluso con** `sudo`	atributo **inmutable**	`lsattr f` → `sudo chattr -i f`
`Read-only file system`	partición `ro`	`findmnt -T ruta` → remontar

🗒️ Caso F · "Mis ficheros son de `root` y no puedo editarlos"

Los creó un comando con `sudo` (capítulo 9). Arreglo: `sudo chown -R "$USER": carpeta`. Para el futuro: **no uses `sudo` salvo cuando haga falta** y no lo mezcles con herramientas que crean ficheros en tu carpeta personal.

🗒️ Caso G · "He ejecutado `chmod -R 777` y ahora todo es un lío"

Respira. Se arregla **por tipos** (capítulo 7):

```
find carpeta -type d -exec chmod 755 {} +
find carpeta -type f -exec chmod 644 {} +
# y los scripts que deban ejecutarse:
chmod +x carpeta/scripts/*.sh
```

Si fue sobre carpetas del **sistema** (`/etc`, `/usr`…) ya no hay forma limpia de repararlo a mano: **reinstalar o restaurar copia**. Por eso nunca `chmod -R` en `/`.

13.5 · 📋 Recetario: "quiero que…" → "haz esto"

Quiero…	Receta
Un **fichero privado**	`chmod 600 f`
Una **carpeta privada**	`chmod 700 d`
Un **script ejecutable** por todos	`chmod 755 s.sh`
Publicar un árbol (web, documentación)	`chmod -R u=rwX,go=rX d`
Una **carpeta de equipo** (todos del grupo editan)	`sudo chgrp g d && sudo chmod 2770 d` + `umask 002`
Una **carpeta de equipo** que otros puedan **leer**	`sudo chmod 2775 d`
Una **carpeta pública** donde cada uno borra lo suyo	`chmod 1777 d`
Un **buzón** (depositar sin listar)	`sudo chgrp g d && chmod 1730 d`
Un **fichero solo de lectura** para todos	`chmod 444 f` (¡y la carpeta controlada!)
Un fichero **a prueba de borrado**, incluso de root	`sudo chattr +i f`
Un **log** al que solo se pueda añadir	`sudo chattr +a f`
Dar acceso a **un solo usuario** concreto	ACL: `setfacl -m u:ana:r f` (cap. 12)

Quiero…	Receta
Quitar la `x` a todos los ficheros (no a carpetas)	`find . -type f -exec chmod a-x {} +`
Copiar un fichero **conservando permisos y dueño**	`sudo cp -a origen destino` (o `rsync -a`)
Copiar un árbol y que **mande mi** `umask`	`cp -r origen destino` (sin `-p` / `-a`)
Clonar permisos de otro fichero	`chmod --reference=modelo f` y `chown --reference=modelo f`
Ver los permisos de todo un árbol en octal	`find d -exec stat -c '%a %U:%G %n' {} + | sort -k3`
Encontrar **peligros**	`find / -xdev -perm -0002 -type f`, `find / -perm -4000 -type f`
Que un usuario **vea qué hay pero no cambie** nada	dueño ≠ usuario; `chmod o=rX d`
Reparar ficheros que quedaron de `root` en `~`	`sudo chown -R "$USER": ~/carpeta`
Auditar qué puede hacer un usuario con una ruta	`sudo diagnostica -u usuario ruta`

13.6 · 🛠️ Retos de reparación

Cada reto empieza con un **bloque que prepara el desastre** (ejecútalo tal cual). Tu misión: **diagnosticar** (con el método de 7 pasos y las herramientas) y **arreglar con el cambio mínimo**. Las soluciones están desplegadas.

🟡 Reto 13.1 · "ana no llega a `proyecto-x/docs/leeme.txt`"

```
$ mkdir -p proyecto-x/docs
$ echo "documentación" > proyecto-x/docs/leeme.txt
$ chmod 755 proyecto-x
$ chmod 700 proyecto-x/docs
$ chmod 644 proyecto-x/docs/leeme.txt
$ sudo -u ana cat proyecto-x/docs/leeme.txt
cat: proyecto-x/docs/leeme.txt: Permission denied
```

💡 Solución

```
$ sudo diagnostica -u ana /home/alumno/lab-permisos/proyecto-x/docs/leeme.txt
Usuario: ana (uid 1002)   Grupos: ana equipo
Ruta:    /home/alumno/lab-permisos/proyecto-x/docs/leeme.txt

  Tramo                                          Permisos    Dueño:Grupo     Aplica rwx  ¿Puede?
  /                                              drwxr-xr-x  root:root        otros  r-x  ✔ puede
atravesarlo
  /home                                          drwxr-xr-x  root:root        otros  r-x  ✔ puede
atravesarlo
  /home/alumno                                   drwxr-x--x  alumno:alumno    otros  --x  ✔ puede
atravesarlo
  /home/alumno/lab-permisos                      drwxr-xr-x  alumno:alumno    otros  r-x  ✔ puede
atravesarlo
  /home/alumno/lab-permisos/proyecto-x           drwxr-xr-x  alumno:alumno    otros  r-x  ✔ puede
atravesarlo
  /home/alumno/lab-permisos/proyecto-x/docs      drwx------  alumno:alumno    otros  ---  ✘ FALTA x
para atravesarlo
  /home/alumno/lab-permisos/proyecto-x/docs/leeme.txt  -rw-r--r--  alumno:alumno    otros  r--  (el
objeto final: ver abajo)

✘ BLOQUEO en: /home/alumno/lab-permisos/proyecto-x/docs  (otros: ---)
  Sin poder atravesar ese tramo, ana no llega al objeto: NO puede hacer nada con él.
  Remedio: dar la x que falta en esa carpeta (chmod o+x / g+x, o añadir al usuario al grupo).

Sin avisos de ACL, atributos inmutables ni opciones de montaje restrictivas.
(Recuerda además el MAC —AppArmor/SELinux— y que los grupos nuevos exigen volver a iniciar sesión.)
$ chmod 755 proyecto-x/docs
$ sudo -u ana cat proyecto-x/docs/leeme.txt
documentación
```

El fichero es `644`, pero **la carpeta** `docs` **es** `700`: a `ana` ("otros") le falta la `x` ahí. Arreglo mínimo: `chmod 755 docs` (o solo `o+rx`).

🟡 Reto 13.2 · "`backup.sh` da Permission denied"

```
$ printf '#!/bin/bash\necho haciendo copia\n' > backup.sh
$ chmod 600 backup.sh
$ ./backup.sh
bash: ./backup.sh: Permission denied
```

💡 Solución

```
$ ls -l backup.sh
-rw------- 1 alumno alumno 32 Oct 16 17:08 backup.sh
$ chmod u+x backup.sh
$ ./backup.sh
haciendo copia
```

Le faltaba la `x` (y como es un script, tiene que mantener la `r`: `rwx------`, o sea `700`).

🔴 Reto 13.3 · "`beto` no puede guardar en `informes/` aunque es de equipo"

```
$ mkdir informes
$ chmod 750 informes
$ sudo chgrp equipo informes
$ sudo -u beto sh -c 'echo "datos" > informes/beto.txt'
sh: 1: cannot create informes/beto.txt: Permission denied
```

💡 Solución

```
$ sudo diagnostica -u beto /home/alumno/lab-permisos/informes
Usuario: beto (uid 1003)   Grupos: beto equipo soporte
Ruta:    /home/alumno/lab-permisos/informes

  Tramo                              Permisos    Dueño:Grupo     Aplica rwx  ¿Puede?
  /                                  drwxr-xr-x  root:root        otros  r-x  ✔ puede atravesarlo
  /home                              drwxr-xr-x  root:root        otros  r-x  ✔ puede atravesarlo
  /home/alumno                       drwxr-x--x  alumno:alumno    otros  --x  ✔ puede atravesarlo
  /home/alumno/lab-permisos          drwxr-xr-x  alumno:alumno    otros  r-x  ✔ puede atravesarlo
  /home/alumno/lab-permisos/informes  drwxr-x---  alumno:equipo    grupo  r-x  (el objeto final: ver
abajo)

Camino libre hasta el objeto. Sobre /home/alumno/lab-permisos/informes, beto puede:
   ✔ listar (r)
   ✘ crear/borrar dentro (w)
   ✔ entrar (x)

Sin avisos de ACL, atributos inmutables ni opciones de montaje restrictivas.
(Recuerda además el MAC —AppArmor/SELinux— y que los grupos nuevos exigen volver a iniciar sesión.)
$ chmod 770 informes
$ sudo -u beto sh -c 'echo "datos" > informes/beto.txt'
$ ls -l informes
total 4
-rw-r--r-- 1 beto beto 6 Oct 16 17:08 beto.txt
```

`beto` entra por el trozo **grupo** (`r-x`): puede entrar y listar, pero **no crear** (falta `w`). Arreglo: `chmod 770` (o `g+w`). Y para que lo que cree herede el grupo: `sudo chmod 2770` (capítulo 10).

🔴 Reto 13.4 · "No puedo borrar `viejo.log` ni con `sudo`"

```
$ echo "datos" > viejo.log
$ sudo chattr +i viejo.log
$ sudo rm -f viejo.log
rm: cannot remove 'viejo.log': Operation not permitted
```

💡 **Solución**

```
$ lsattr viejo.log
----i---------e------- viejo.log
$ sudo chattr -i viejo.log
$ sudo rm -f viejo.log
$ ls viejo.log
ls: cannot access 'viejo.log': No such file or directory
```

`Operation not permitted` **incluso siendo** `root` → atributo **inmutable** (`i`). Se ve con `lsattr` y se quita con `chattr -i` .

🔴 Reto 13.5 · "Diseña el reparto"

Prepara `intranet/` con dos carpetas: `publico/` (lo puede **leer cualquiera**) y `privado/` (**solo** `equipo` , que además puede escribir). `carla` (de `soporte`) no debe ver `privado` , pero sí `publico` .

💡 **Solución**

```
$ mkdir -p intranet/publico intranet/privado
$ echo "para todos" > intranet/publico/info.txt
$ echo "solo equipo" > intranet/privado/plan.txt
$ chmod 755 intranet
$ chmod 755 intranet/publico
$ chmod 644 intranet/publico/info.txt
$ sudo chgrp -R equipo intranet/privado
$ sudo chmod 2770 intranet/privado
$ sudo chmod 660 intranet/privado/plan.txt
$ sudo -u carla cat intranet/publico/info.txt
para todos
$ sudo -u carla cat intranet/privado/plan.txt
cat: intranet/privado/plan.txt: Permission denied
$ sudo -u ana sh -c 'echo "añadido por ana" >> intranet/privado/plan.txt'
$ cat intranet/privado/plan.txt
solo equipo
añadido por ana
```

`publico` → `755/644` (todos leen). `privado` → grupo `equipo` , `2770` (equipo entra/escribe, SGID para heredar el grupo) y los ficheros `660` ; **otros nada**.

• Reto 13.6 · Examen: ¿cuál es el primer paso?

Un servicio falla con "Permission denied" al abrir `/srv/datos/conf.ini` *. ¿Cuál es el mejor* ***primer*** *comando de diagnóstico?* a) `chmod -R 777 /srv` b) `namei -l /srv/datos/conf.ini` c) `reboot` d) `chown root /srv/datos/conf.ini`

💡 **Solución**

b) `namei -l` enseña los permisos de **todos los tramos**: ¡lo más probable es que falte la `x` en una carpeta! (`a)` es el parche peligroso; `d)` rompería el acceso del servicio.)

• Reto 13.7 · Examen: dos errores

Un usuario intenta `chown ana f` *(siendo dueño de* `f` *) y recibe* `Operation not permitted` *. Otro intenta leer un fichero* `-rw-------` *ajeno y recibe* `Permission denied` *. ¿Qué diferencia hay entre ambos mensajes?*

💡 **Solución**

`Operation not permitted` (EPERM): la **acción** está reservada (solo `root` cambia el dueño); da igual los permisos del fichero. `Permission denied` (EACCES): falta un **permiso** (`r`) para esa operación.

Capítulo 14 · 🏆 El jefe final: misiones, auditoría y mini-examen LPIC-1

> 🎯 **Objetivo:** juntar **todo** lo aprendido en problemas completos, con **autocomprobación** (✔/✘ medidos de verdad con distintos usuarios), una **auditoría de seguridad** con `find`, **velocidad** de conversión y un **mini-examen tipo LPIC-1** de 30 preguntas.
>
> 📘 **LPIC-1:** 104.5 · 110.1 · 107.1 · 103.3.
>
> 🧪 **Antes de empezar:** `cd ~/lab-permisos`. Si algo quedó revuelto de capítulos anteriores, **vuelve a ejecutar el script del laboratorio**.

14.1 · El método del experto

1. 📝 **Traduce el enunciado a una tabla**: *quién* debe poder hacer *qué* sobre *qué*.
2. 🧱 **Elige la herramienta de cada fila**: `chmod` (permisos), `chgrp` / `chown` (propiedad), **SGID** (grupo heredado), **sticky** (borrado), `umask` (permisos de lo nuevo), `chattr` (blindaje).
3. 🔨 **Construye de fuera hacia dentro**: primero la carpeta, luego su grupo, luego sus permisos, luego los bits especiales.
4. 🧪 **Comprueba como cada usuario** (`sudo -u …`), **los éxitos y los fallos**: *"que no pueda"* también se prueba.
5. 🧹 **Simplifica**: permisos **mínimos**, nada de `777`.

14.2 · 🕵️ Misión 1: "La intranet de la empresa"

Eres el administrador. Hay 3 personas: `ana` y `beto` (grupo `equipo`) y `carla` y `beto` (grupo `soporte`). Tienes que montar en `intranet-final/` cinco carpetas con **estas reglas**:

Carpeta	Regla
`equipo/`	Solo el grupo `equipo` puede **entrar, leer, crear y editar**. Lo que cree cualquiera **hereda el grupo** `equipo`. Los demás, **nada**.
`soporte/`	Lo mismo para el grupo `soporte`.
`publico/`	**Todos leen**, solo tú escribes.
`buzon/`	El grupo `soporte` puede **depositar** ficheros pero **no listar** la carpeta, y **ninguno borra lo de otro**. Los demás, nada.
`tmp/`	**Todos** crean; **cada uno borra solo lo suyo**.

Primero, nuestro **comprobador** (pégalo: son dos funciones que dicen ✔ o ✘):

```
si() { d=$1; shift; if "$@" >/dev/null 2>&1; then echo "  ✔ $d"; else echo "  ✘ $d   ← NO CUMPLE"; fi; }
no() { d=$1; shift; if "$@" >/dev/null 2>&1; then echo "  ✘ $d   ← NO CUMPLE"; else echo "  ✔ $d"; fi; }
```

> 🧠 **Inténtalo tú primero.** Cuando termines, pega la solución y compara.

> 💡 **Ver solución (la construcción)**

```
$ mkdir intranet-final
$ cd intranet-final
$ mkdir equipo soporte publico buzon tmp
# equipo/ y soporte/: grupo + SGID + acceso solo para el grupo
$ sudo chgrp equipo equipo
$ sudo chmod 2770 equipo
$ sudo chgrp soporte soporte
$ sudo chmod 2770 soporte
# publico/: todos leen
$ chmod 755 publico
$ echo "bienvenidos a la intranet" > publico/leeme.txt
$ chmod 644 publico/leeme.txt
# buzon/: soporte deposita (sin listar) + sticky
$ sudo chgrp soporte buzon
$ chmod 1730 buzon
# tmp/: público con sticky
$ chmod 1777 tmp
# la carpeta raíz, abierta para pasar pero sin escritura ajena
$ chmod 755 .
$ ls -l
total 20
drwx-wx--T 2 alumno soporte 4096 Oct 16 17:09 buzon
drwxrws--- 2 alumno equipo  4096 Oct 16 17:09 equipo
drwxr-xr-x 2 alumno alumno  4096 Oct 16 17:09 publico
drwxrws--- 2 alumno soporte 4096 Oct 16 17:09 soporte
drwxrwxrwt 2 alumno alumno  4096 Oct 16 17:09 tmp
```

Y ahora **el examen de la misión** (cada línea se prueba **como el usuario que corresponde**):

```
cd ~/lab-permisos/intranet-final
echo "── equipo/"
si "ana (equipo) crea en equipo/"                      sudo -u ana sh -c 'umask 002; touch equipo/de-ana.txt'
si "beto (equipo) edita lo que creó ana"              sudo -u beto sh -c 'echo x >> equipo/de-ana.txt'
si "lo creado en equipo/ hereda el grupo 'equipo'"     test "$(stat -c %G equipo/de-ana.txt)" = equipo
no "carla (otros) NO puede listar equipo/"            sudo -u carla ls equipo
echo "── soporte/"
si "carla (soporte) crea en soporte/"                  sudo -u carla sh -c 'umask 002; touch soporte/de-
carla.txt'
si "beto (soporte) edita lo que creó carla"           sudo -u beto sh -c 'echo x >> soporte/de-carla.txt'
si "lo creado en soporte/ hereda el grupo 'soporte'"   test "$(stat -c %G soporte/de-carla.txt)" = soporte
no "ana (otros) NO puede entrar en soporte/"          sudo -u ana ls soporte
echo "── publico/"
si "ana lee publico/leeme.txt"                         sudo -u ana cat publico/leeme.txt
no "ana NO puede crear en publico/"                    sudo -u ana touch publico/x
no "carla NO puede borrar en publico/"                 sudo -u carla rm -f publico/leeme.txt
echo "── buzon/"
si "carla deposita en buzon/"                          sudo -u carla touch buzon/de-carla
si "beto (soporte) deposita en buzon/"                 sudo -u beto touch buzon/de-beto
no "carla NO puede listar buzon/"                      sudo -u carla ls buzon
no "beto NO puede borrar lo de carla (sticky)"         sudo -u beto rm -f buzon/de-carla
no "ana (otros) NO puede ni depositar en buzon/"       sudo -u ana touch buzon/de-ana
echo "── tmp/"
si "ana crea en tmp/"                                  sudo -u ana touch tmp/de-ana
no "beto NO puede borrar lo de ana en tmp/"            sudo -u beto rm -f tmp/de-ana
si "ana SÍ puede borrar lo suyo"                       sudo -u ana rm -f tmp/de-ana
cd ~/lab-permisos
```

Resultado:

```
── equipo/
  ✔ ana (equipo) crea en equipo/
  ✔ beto (equipo) edita lo que creó ana
  ✔ lo creado en equipo/ hereda el grupo 'equipo'
  ✔ carla (otros) NO puede listar equipo/
── soporte/
  ✔ carla (soporte) crea en soporte/
  ✔ beto (soporte) edita lo que creó carla
  ✔ lo creado en soporte/ hereda el grupo 'soporte'
  ✔ ana (otros) NO puede entrar en soporte/
── publico/
  ✔ ana lee publico/leeme.txt
  ✔ ana NO puede crear en publico/
  ✔ carla NO puede borrar en publico/
── buzon/
  ✔ carla deposita en buzon/
  ✔ beto (soporte) deposita en buzon/
  ✔ carla NO puede listar buzon/
  ✔ beto NO puede borrar lo de carla (sticky)
  ✔ ana (otros) NO puede ni depositar en buzon/
── tmp/
  ✔ ana crea en tmp/
  ✔ beto NO puede borrar lo de ana en tmp/
  ✔ ana SÍ puede borrar lo suyo
```

Todo en ✔: **misión cumplida**. Fíjate en qué herramienta resuelve cada fila: **SGID** (grupo heredado) en `equipo` y `soporte`; **sticky +** `-wx` en el buzón; **sticky +** `rwx` en `tmp`; `755` en `publico`.

```
$ ls -l intranet-final
total 20
drwx-wx--T 2 alumno soporte 4096 Oct 16 17:09 buzon
drwxrws--- 2 alumno equipo  4096 Oct 16 17:09 equipo
drwxr-xr-x 2 alumno alumno  4096 Oct 16 17:09 publico
drwxrws--- 2 alumno soporte 4096 Oct 16 17:09 soporte
drwxrwxrwt 2 alumno alumno  4096 Oct 16 17:09 tmp
```

14.3 · 🔍 Misión 2: auditoría de seguridad

Heredas un servidor con una carpeta `auditoria2/` **llena de malas prácticas**. Montamos el escenario (ejecuta el bloque tal cual):

```
$ mkdir -p auditoria2/app auditoria2/logs auditoria2/claves auditoria2/tmp
$ printf '#!/bin/bash\necho servidor\n' > auditoria2/app/servidor.sh
$ chmod 4755 auditoria2/app/servidor.sh
$ cp /usr/bin/id auditoria2/app/herramienta
$ sudo chown root auditoria2/app/herramienta
$ sudo chmod 4755 auditoria2/app/herramienta
$ echo "pass=1234" > auditoria2/app/config.ini
$ chmod 644 auditoria2/app/config.ini
$ touch auditoria2/app/huerfano
$ sudo chown 4242:4242 auditoria2/app/huerfano
$ echo "evento" > auditoria2/logs/app.log
$ chmod 666 auditoria2/logs/app.log
$ echo "CLAVE-PRIVADA" > auditoria2/claves/id_ed25519
$ chmod 644 auditoria2/claves/id_ed25519
$ echo "clave-publica" > auditoria2/claves/id_ed25519.pub
$ chmod 644 auditoria2/claves/id_ed25519.pub
$ chmod 777 auditoria2/tmp
$ find auditoria2 -exec stat -c '%a %U:%G %n' {} + | sort -k3
775 alumno:alumno auditoria2
775 alumno:alumno auditoria2/app
644 alumno:alumno auditoria2/app/config.ini
4755 root:alumno auditoria2/app/herramienta
664 UNKNOWN:UNKNOWN auditoria2/app/huerfano
4755 alumno:alumno auditoria2/app/servidor.sh
775 alumno:alumno auditoria2/claves
644 alumno:alumno auditoria2/claves/id_ed25519
644 alumno:alumno auditoria2/claves/id_ed25519.pub
775 alumno:alumno auditoria2/logs
666 alumno:alumno auditoria2/logs/app.log
777 alumno:alumno auditoria2/tmp
```

Tu informe debe encontrar **seis tipos de problemas**. Intenta cada búsqueda **antes** de mirar la solución.

🟡 Hallazgo 14.1 · Ficheros con SUID o SGID

💡 **Solución**

```
$ find auditoria2 -type f -perm /6000
auditoria2/app/servidor.sh
auditoria2/app/herramienta
$ find auditoria2 -type f -perm -4000 -user root
auditoria2/app/herramienta
```

Dos SUID: `servidor.sh` (¡un **script** con SUID, que Linux ignora y solo es un riesgo!) y `herramienta` (un **binario SUID de** `root`, el más grave).

🟡 Hallazgo 14.2 · Ficheros escribibles por cualquiera

💡 **Solución**

```
$ find auditoria2 -type f -perm -0002
auditoria2/logs/app.log
```

`app.log` (`666`): cualquier usuario puede manipular el registro.

🟡 Hallazgo 14.3 · Carpetas públicas sin *sticky*

💡 **Solución**

```
$ find auditoria2 -type d -perm -0002 ! -perm -1000
auditoria2/tmp
```

`tmp` (`777` sin sticky): cualquiera borra lo de los demás.

🟡 Hallazgo 14.4 · Ficheros huérfanos

💡 **Solución**

```
$ find auditoria2 -nouser -o -nogroup
auditoria2/app/huerfano
```

`huerfano` pertenece al UID/GID `4242`, que no existe.

🔴 Hallazgo 14.5 · Claves privadas legibles por otros

(Pista: `-name 'id_*'`, que **no** acabe en `.pub`, con permisos para grupo u otros: `-perm /077`.)

💡 **Solución**

```
$ find auditoria2 -type f -name 'id_*' ! -name '*.pub' -perm /077
auditoria2/claves/id_ed25519
```

`id_ed25519` en `644`: `ssh` la rechazaría y cualquier usuario del sistema puede **copiarla**.

🔴 Hallazgo 14.6 · Secretos legibles por todos

(Pista: ficheros `*.ini` con lectura para otros: `-perm -0004`.)

💡 **Solución**

```
$ find auditoria2 -type f -name '*.ini' -perm -0004
auditoria2/app/config.ini
```

`config.ini` guarda `pass=1234` y es legible por todos (`644`).

• Corrección 14.7 · Dejarlo todo en orden

Arregla **cada** hallazgo con el cambio **mínimo**. Después, repite las búsquedas: **no debe salir nada**.

💡 **Solución**

```
$ chmod u-s auditoria2/app/servidor.sh
$ sudo chmod u-s auditoria2/app/herramienta
$ chmod 640 auditoria2/logs/app.log
$ chmod 1777 auditoria2/tmp
$ sudo chown root:root auditoria2/app/huerfano
$ chmod 600 auditoria2/claves/id_ed25519
$ chmod 600 auditoria2/app/config.ini
$ echo "--- repetimos la auditoría (todo vacío = bien) ---"
--- repetimos la auditoría (todo vacío = bien) ---
$ find auditoria2 -type f -perm /6000
$ find auditoria2 -type f -perm -0002
$ find auditoria2 -type d -perm -0002 ! -perm -1000
$ find auditoria2 -nouser -o -nogroup
$ find auditoria2 -type f -name 'id_*' ! -name '*.pub' -perm /077
$ find auditoria2 -type f -name '*.ini' -perm -0004
$ echo "(fin)"
(fin)
```

Cada arreglo es mínimo y **el más estricto que no rompe nada**: quitar el `s`, quitar la escritura a otros, añadir sticky, dar el fichero a `root`, `600` para secretos.

14.4 · 🧩 Misión 3: "Reconstruye este listado, solo con letras"

Te pasan este listado y debes dejar **exactamente** esos permisos, **sin usar números**:

```
-rwxr-x---  informe.sh
-rw-r-----  datos.csv
-r--r--r--  leeme.txt
-rwx--x--x  utilidad
-rw-rw-r--  notas.txt
---x------  oculto
```

💡 **Solución**

```
$ mkdir reconstruir
$ cd reconstruir
$ touch informe.sh datos.csv leeme.txt utilidad notas.txt oculto
$ chmod u=rwx,g=rx,o=       informe.sh
$ chmod u=rw,g=r,o=         datos.csv
$ chmod a=r                 leeme.txt
$ chmod u=rwx,go=x          utilidad
$ chmod ug=rw,o=r           notas.txt
$ chmod u=x,go=             oculto
$ cat > esperado.txt <<'EOF'
---x------ oculto
-r--r--r-- leeme.txt
-rw-r----- datos.csv
-rw-rw-r-- notas.txt
-rwx--x--x utilidad
-rwxr-x--- informe.sh
EOF
$ stat -c '%A %n' informe.sh datos.csv leeme.txt utilidad notas.txt oculto | sort > obtenido.txt
$ sort esperado.txt | diff - obtenido.txt && echo "✅ IGUAL: listado reproducido"
✅ IGUAL: listado reproducido
$ cd ..
```

(`diff` sin salida + el ✅ significa que son **idénticos**.)

14.5 · ⚡ Misión 4: velocidad mixta

Cronométrate. Objetivo: **12 respuestas en menos de 1 minuto** (≈ 5 s cada una). Y para repetir sin fin con preguntas nuevas cada vez: `herramientas/entrenador-conversion.html` (¡prueba el *sprint* de 60 s!).

• Serie 14.8 · A

Serie A

1. `rwxr-x---` → ¿octal?
2. `2750` → ¿letras?
3. `umask 027` → ¿fichero y carpeta nuevos?
4. `111101000` → ¿octal y letras?
5. `rwsr-xr-x` → ¿octal?
6. `420` (decimal) → ¿octal?
7. `1777` → ¿letras?
8. `umask 022` → ¿fichero y carpeta nuevos?
9. `rw-rw-r--` → ¿octal?
10. `640` → ¿letras?
11. `umask 077` → ¿fichero y carpeta nuevos?
12. `rwxrwsr-x` → ¿octal?

💡 **Respuestas**

1. `rwxr-x---` → **750**
2. `2750` → `rwxr-s---`
3. `umask 027` → ficheros **640** · carpetas **750**
4. `111101000` → **750** (`rwxr-x---`)

5. `rwsr-xr-x` → **4755**
6. `420` → **644** (`rw-r--r--`)
7. `1777` → `rwxrwxrwt`
8. `umask 022` → ficheros **644** · carpetas **755**
9. `rw-rw-r--` → **664**
10. `640` → `rw-r-----`
11. `umask 077` → ficheros **600** · carpetas **700**
12. `rwxrwsr-x` → **2775**

- **Serie 14.9 · B**

Serie B

1. `751` → ¿letras?
2. `r--r-----` → ¿octal?
3. `umask 002` → ¿fichero y carpeta nuevos?
4. `488` (decimal) → ¿octal?
5. `4711` → ¿letras?
6. `110100000` → ¿octal y letras?
7. `rwx--s--x` → ¿octal?
8. `umask 037` → ¿fichero y carpeta nuevos?
9. `2770` → ¿letras?
10. `rw-r-----` → ¿octal?
11. `umask 033` → ¿fichero y carpeta nuevos?
12. `0600` → ¿letras?

💡 **Respuestas**

1. `751` → `rwxr-x--x`
2. `r--r-----` → **440**
3. `umask 002` → ficheros **664** · carpetas **775**
4. `488` → **750** (`rwxr-x---`)
5. `4711` → `rws--x--x`
6. `110100000` → **640** (`rw-r-----`)
7. `rwx--s--x` → **2711**
8. `umask 037` → ficheros **640** · carpetas **740**
9. `2770` → `rwxrws---`
10. `rw-r-----` → **640**
11. `umask 033` → ficheros **644** · carpetas **744**
12. `0600` → `rw-------`

14.6 · 📝 Mini-examen tipo LPIC-1 (30 preguntas)

Reglas: sin ordenador (aunque tienes laboratorio para comprobar después), **un minuto por pregunta**. Anota tus respuestas y compáralas con la **clave** de abajo. Hay preguntas de **una** respuesta y de **dos o tres** (se indica).

P1. ¿Qué número octal equivale a `-rwxr-x---`? a) 750 b) 705 c) 570 d) 755

P2. ¿Qué comando muestra los permisos de la **carpeta** `/tmp` (y no los de su contenido)? a) `ls -l /tmp` b) `ls -ld /tmp` c) `ls -a /tmp` d) `stat -f /tmp`

P3. Con `umask 022`, ¿qué permisos tiene un **fichero nuevo** creado con `touch`? a) `rw-rw-rw-` b) `rwxr-xr-x` c) `rw-r--r--` d) `rw-------`

P4. ¿Quién puede cambiar el **propietario** de un fichero con `chown`? a) El propietario b) Cualquiera con permiso de escritura c) Solo `root` d) Cualquier miembro del grupo

P5. En `-rwsr-xr-x`, ¿qué significa la `s`? a) El fichero es un socket b) Se ejecuta con los privilegios de su **dueño** c) Es un fichero compartido d) Es sticky

P6. ¿Qué permiso **mínimo** sobre un directorio permite hacer `cd` en él? a) `r` b) `w` c) `x` d) `rwx`

P7. ¿Qué hace `chmod g+s carpeta`? a) Da el SUID b) Los ficheros nuevos **heredan el grupo** de la carpeta c) Impide borrar ficheros ajenos d) Hace la carpeta ejecutable

P8. (**Elige 2**) Un fichero tiene `-rw-r-----` con dueño `root` y grupo `ventas`. `juan` pertenece a `ventas`, `luis` no. ¿Qué es cierto? a) `juan` puede leerlo b) `luis` puede leerlo c) `juan` puede escribirlo d) `luis` no puede leerlo e) `juan` puede ejecutarlo

P9. ¿Qué fichero contiene las contraseñas **cifradas** de los usuarios? a) `/etc/passwd` b) `/etc/shadow` c) `/etc/group` d) `/etc/login.defs`

P10. ¿Qué `chmod` simbólico equivale a `chmod 644 f`? a) `u=rw,go=r` b) `u=rwx,go=r` c) `a=r` d) `u=r,go=rw`

P11. ¿Para qué sirve el bit *sticky* en `/tmp`? a) Para que los ficheros no se borren nunca b) Para que solo el dueño de un fichero (o `root`) pueda borrarlo c) Para ejecutar como `root` d) Para heredar el grupo

P12. (**Elige 2**) ¿Qué comandos pueden cambiar el **grupo** propietario de un fichero? a) `chgrp` b) `chmod g=` c) `chown :grupo` d) `usermod`

P13. ¿Qué comando busca **todos** los ficheros con el bit SUID en el sistema? a) `find / -perm 4000` b) `find / -perm -4000` c) `find / -type s` d) `ls -l / | grep s`

P14. Un directorio tiene `dr--------`. Su dueño ejecuta `ls -l carpeta`. ¿Qué ocurre? a) Muestra todo con detalles b) Muestra los nombres pero da errores de acceso al pedir detalles c) No muestra nada y da `Permission denied` en todo d) Muestra solo la carpeta vacía

P15. `root` ejecuta `cat fichero` donde `fichero` tiene `---------- (000)`. ¿Qué pasa? a) `Permission denied` b) Muestra el contenido c) `Operation not permitted` d) Pide contraseña

P16. ¿Cuál es el problema de `chmod -R 755 .` sobre un árbol con muchos ficheros de texto? a) Ninguno b) Los ficheros de texto pasan a ser **ejecutables** c) Se borran los permisos d) Solo afecta a las carpetas

P17. ¿Qué hace `usermod -G ventas juan` **sin** `-a`? a) Añade `ventas` a los grupos de `juan` b) **Sustituye** todos sus grupos secundarios por `ventas` c) Cambia su grupo principal d) Da error

P18. ¿Qué octal equivale a `rw-rw-r--`? a) 666 b) 664 c) 646 d) 764

P19. Un usuario ejecuta `umask 077` y luego `mkdir secreto`. ¿Qué permisos tiene `secreto`? a) `drwxr-xr-x` b) `drwx------` c) `d---------` d) `drwxrwxrwx`

P20. Un script tiene permisos `-r-x------` (dueño `maria`). `maria` ejecuta `./script`. ¿Qué ocurre? a) Se ejecuta b) `Permission denied`: necesita `w` c) `command not found` d) Se ejecuta solo con `sudo`

P21. ¿Qué permisos muestra `ls -l` para un **enlace simbólico**? a) Los del destino b) Siempre `lrwxrwxrwx` c) `l---------` d) Los del directorio

P22. ¿Qué significa la `S` **mayúscula** en `-rwSr--r--`? a) SUID con `x` para el dueño b) SUID **sin** `x` para el dueño c) SGID d) Es un error de `ls`

P23. Quieres dar `x` a **todas las carpetas** y solo a los ficheros que ya fueran ejecutables, con `-R`. ¿Qué usas? a) `chmod -R +x` b) `chmod -R a+X` c) `chmod -R 755` d) `chmod -R +s`

P24. (**Elige 2**) En el algoritmo de permisos, ¿qué es **verdad**? a) Se mira el trozo del **dueño** si eres el dueño, aunque el del grupo sea más permisivo b) Se suman los permisos de dueño y grupo c) `root` se salta (casi) todos los permisos d) "Otros" se comprueba siempre primero

P25. ¿Qué permisos necesitas en una **carpeta** para **borrar** un fichero que contiene (sin sticky)? a) `r` en el fichero b) `w` en el fichero c) `w` y `x` en la **carpeta** d) `r` y `x` en la carpeta

P26. ¿Cuál es el octal de `chmod u=rwx,g=rx,o= f`? a) 755 b) 750 c) 740 d) 705

P27. Un fichero tiene `-rwsr-xr-x` y haces `chmod 755 f`. ¿Qué ocurre con el SUID? a) Se conserva b) Se **elimina** c) Pasa a SGID d) Da error

P28. `touch nuevo` falla con `Permission denied` en una carpeta `dr-xr-xr-x` de tu propiedad. ¿Por qué? a) Falta la `x` b) Falta la `r` c) Falta la `w` en la carpeta d) El fichero debe existir antes

P29. (**Elige 2**) ¿Qué es cierto del SGID en un **directorio**? a) Los ficheros nuevos heredan el **grupo** del directorio b) Los subdirectorios nuevos heredan también el SGID c) Los ficheros nuevos se ejecutan como el dueño d) Impide borrar ficheros ajenos

P30. ¿Qué comando lista los **ficheros normales escribibles por cualquiera** en `/var`? a) `find /var -type f -perm -0002` b) `find /var -type f -perm 0002` c) `find /var -perm /0777` d) `ls -lR /var | grep w`

✅ Clave de respuestas (con explicación)

💡 **Clave de respuestas**

P	Resp.	Por qué
1	**a**	`rwx r-x ---` → 7·5·0
2	**b**	`-d` = la ficha de la carpeta misma
3	**c**	666 & ~022 = 644
4	**c**	solo `root` cambia el dueño
5	**b**	SUID
6	**c**	`x` = entrar
7	**b**	SGID en carpeta = grupo heredado
8	**a, d**	`juan` (grupo) → `r--`; `luis` (otros) → `---`
9	**b**	`/etc/shadow`
10	**a**	`u=rw` (6), `go=r` (4,4)
11	**b**	sticky
12	**a, c**	`chgrp g f` ≡ `chown :g f`
13	**b**	`-perm -4000` (todos esos bits)
14	**b**	`r` sin `x`: ves nombres, no detalles
15	**b**	`root` se salta rwx para leer
16	**b**	todos los ficheros reciben `x`
17	**b**	sin `-a` **sustituye**
18	**b**	6·6·4
19	**b**	777 & ~077 = 700
20	**a**	`r-x` del dueño: se ejecuta (necesita `r` y `x`, ¡y los tiene!)
21	**b**	siempre `lrwxrwxrwx`
22	**b**	`S` = bit sin la `x` de debajo
23	**b**	`a+X` (o `u=rwX,go=rX`)
24	**a, c**	primer trozo que coincide; root aparte
25	**c**	borrar manda la carpeta
26	**b**	7·5·0
27	**b**	en un **fichero**, `chmod 755` quita SUID/SGID
28	**c**	crear exige `w` (y `x`) en la carpeta
29	**a, b**	grupo y SGID heredados
30	**a**	`-perm -0002` (con `-`) = "tiene la `w` de otros"

> **Tu puntuación:** 27-30 → ¡a por el examen! · 22-26 → casi listo, repasa los fallos · menos de 22 → vuelve a los capítulos 4, 8 y 10.

14.7 · 🎓 Y ahora, ¿qué?

Has recorrido el 100 % de lo que pide el objetivo **104.5** de LPIC-1 y algo más. Para seguir:

Si quieres…	Mira…
Dar permisos a un usuario concreto sin cambiar grupos	**ACL**: `man setfacl`, `man acl` (capítulo 12)
Entender el **MAC** de Ubuntu	AppArmor: `man apparmor`, `aa-status`
Dominar `sudo` y sus reglas	`man sudoers`, `visudo` (el fichero `/etc/sudoers` es `440`)
Practicar buscar y filtrar con `find` / `grep`	la guía de regex y `grep`
Repasar de un vistazo	la chuleta de una página

> 🧹 **Cuando termines**, deja tu sistema como estaba: `bash ~/Guias-y-Herramientas/guias/permisos-linux-lpic1/laboratorio/preparar-laboratorio.sh --limpiar` borra `~/lab-permisos`, los usuarios `ana`, `beto`, `carla` y los grupos `equipo`, `soporte`, y devuelve a tu carpeta personal su permiso original.

📄 Chuleta: permisos en Linux (LPIC-1)

> Todo en una página. Imprímela, pégala en la pared, pero **no te limites a leerla**: ¡practica los ejercicios! 🧮 Calculadora visual: `herramientas/calculadora-chmod.html`.

1 · Leer `ls -l` (la receta de 5 segundos)

```
-rwxrwxr-x  1  alumno  alumno  1015  Oct  9 16:16  cerve.sh
|└┘└┘└┘  |    |       |      |         |          └ nombre
| |  |  |   |    |       |      |         └ fecha de modificación (hora si < 6 meses, si no año)
| |  |  |   |    |       |      └ tamaño en bytes
| |  |  |   |    |       └ grupo propietario
| |  |  |   |    └ usuario propietario
| |  |  |   └ nº de enlaces (en carpetas: 2 + subcarpetas)
| |  |  └ OTROS (o)      ┐
| |  └ GRUPO (g)         ├ 3 trozos de 3, siempre en orden r w x; la raya = "no"
| └ DUEÑO (u)            ┘
└ tipo: - fichero · d carpeta · l enlace · c b dispositivo · p tubería · s socket
```

2 · r, w, x: fichero frente a carpeta

	Fichero	Carpeta
`r` (4)	leer el contenido	**listar nombres** (`ls`)
`w` (2)	modificar el contenido	**crear / borrar / renombrar dentro** (necesita `x`)
`x` (1)	ejecutarlo (un **script** necesita `r` + `x`)	**entrar y acceder por nombre** (`cd`, `ls -l`)

- **Borrar** un fichero → manda la **carpeta** (`w` + `x`), no el fichero.
- Hace falta `x` en **todas** las carpetas del camino (`namei -l ruta`).

3 · ¿Quién decide? (el algoritmo)

`root` → pasa (salvo ejecutar sin ninguna `x`) · **dueño** → solo su trozo · **grupo** (principal o secundario) → solo su trozo · **otros**. **El primero que coincide gana; no se suma.** (Un dueño con `---` no puede leer, aunque "otros" sí.)

4 · Octal: r=4 · w=2 · x=1

Cifra	Binario (`r w x`)	Letras	Se suma	Significa
0	`000`	`---`	0	nada
1	`001`	`--x`	1	ejecutar
2	`010`	`-w-`	2	escribir
3	`011`	`-wx`	2 + 1	escribir + ejecutar
4	`100`	`r--`	4	leer
5	`101`	`r-x`	4 + 1	leer + ejecutar
6	`110`	`rw-`	4 + 2	leer + escribir
7	`111`	`rwx`	4 + 2 + 1	todo (leer + escribir + ejecutar)

`rwxr-xr-x` → `7·5·5` = **755** · cifras: **dueño · grupo · otros** · 4.ª cifra (si existe): **especiales** `4`=SUID `2`=SGID `1`=sticky.

5 · Los modos que debes reconocer

Octal	Letras	Para qué se usa
`000`	`---------`	nada para nadie (carpeta/fichero "cerrado")
`400`	`r--------`	solo lectura, solo el dueño (claves privadas muy protegidas)
`440`	`r--r-----`	solo lectura para dueño y grupo (`/etc/sudoers` : `-r--r-----`)
`444`	`r--r--r--`	solo lectura para todos
`600`	`rw-------`	**privado**: lee y escribe solo el dueño (clave SSH privada, `secreto.txt`)
`640`	`rw-r-----`	el dueño edita, el grupo lee (`/etc/shadow` , logs)
`644`	`rw-r--r--`	**el clásico de los ficheros**: todos leen, solo el dueño escribe (`/etc/passwd`)
`660`	`rw-rw----`	dueño y grupo editan (trabajo en equipo)
`664`	`rw-rw-r--`	como 660 pero todos leen (el que dan `umask 002` y Ubuntu)
`666`	`rw-rw-rw-`	todos leen y escriben (¡evítalo!)
`700`	`rwx------`	**privado**: script o carpeta solo del dueño (`~/.ssh`)
`711`	`rwx--x--x`	el dueño todo; los demás solo "atraviesan" (`--x`)
`750`	`rwxr-x---`	dueño todo; grupo lee y entra (**HOME en Ubuntu 24.04**)
`751`	`rwxr-x--x`	como 750 pero otros pueden ejecutar (`saludos.sh` de tu captura)
`755`	`rwxr-xr-x`	**el clásico de programas y carpetas**: todos leen/entran, solo el dueño escribe
`770`	`rwxrwx---`	dueño y grupo todo; otros nada
`775`	`rwxrwxr-x`	dueño y grupo todo; otros leen/ejecutan (`cerve.sh` y `salario.sh` de tu captura)
`777`	`rwxrwxrwx`	todo para todos (¡casi siempre un error!)
`1777`	`rwxrwxrwt`	**sticky**: carpeta pública donde cada uno borra lo suyo (`/tmp`)
`1730`	`rwx-wx--T`	buzón: el grupo deposita, nadie lista (`/var/spool/cron/crontabs`)
`2755`	`rwxr-sr-x`	**SGID** en programa: corre con el grupo del dueño (`/usr/bin/crontab`)
`2775`	`rwxrwsr-x`	**SGID** en carpeta de equipo: lo nuevo hereda el grupo
`4755`	`rwsr-xr-x`	**SUID**: corre con los privilegios del dueño (`/usr/bin/passwd`)

6 · Conversión rápida

De → A	Cómo
letras → octal	corta de 3 en 3; suma 4·r + 2·w + 1·x de cada trozo
octal → letras	cada cifra → su trozo (7 `rwx` · 6 `rw-` · 5 `r-x` · 4 `r--` · 0 `---`)
letras → binario	letra = 1, raya = 0 (`rwxr-xr-x` = `111 101 101`)
binario → octal	agrupa de 3 en 3
octal → decimal	`echo $(( 8#755 ))` = 493 · decimal → octal: `printf '%o\n' 493`
ver el octal real	`stat -c '%a %A %n' *`
herramienta	`permisos 755` · `permisos -rwsr-xr-x` · `permisos 644 g+w` · `permisos -u 027`

7 · `chmod`

```
chmod [ugoa]  [+ - =]  [rwxXst | ugo]   f          chmod u+x,g-w,o= f        chmod 640 f        chmod -R
u=rwX,go=rX d
        │        │            └─ X: x solo si es carpeta o ya era ejecutable
        │        └─ + añade · - quita · = fija EXACTAMENTE (borra el resto)
        └─ u dueño · g grupo · o otros · a todos       g=u copia los de u
```

- **Solo el dueño** (o root) hace `chmod`. `chmod -R` con números pone `x` a todos los ficheros → usa `X` o `find -type d/-type f`.
- ⚠️ **Sin "quién"** (`chmod -w f`, `chmod +x f`) **respeta la** `umask` → escribe siempre `a-w`, `go-w`…
- Un `chmod` numérico quita SUID/SGID de **ficheros**, pero **no** el SGID de **carpetas** (usa `g-s` o `00755`).
- Si **no eres del grupo** del fichero, tu `chmod g+s` **no pone** el SGID (sin avisar): usa `sudo`.

8 · `umask`: `permisos = base & ~umask` (base 666 ficheros · 777 carpetas)

umask	Ficheros nuevos (666 & ~umask)	Carpetas nuevas (777 & ~umask)	Uso típico
0000	666 rw-rw-rw-	777 rwxrwxrwx	nada se quita (¡peligroso!)
0002	664 rw-rw-r--	775 rwxrwxr-x	**Ubuntu** para usuarios normales (grupo escribe)
0007	660 rw-rw----	770 rwxrwx---	equipo cerrado: otros no ven nada, el grupo trabaja
0022	644 rw-r--r--	755 rwxr-xr-x	**clásica**: `root`, Debian, casi todo (grupo y otros solo leen)
0027	640 rw-r-----	750 rwxr-x---	servidores: grupo lee, otros nada
0037	640 rw-r-----	740 rwxr-----	grupo solo lee (sin `x`), otros nada
0077	600 rw-------	700 rwx------	**privado**: solo el dueño
0222	444 r--r--r--	555 r-xr-xr-x	ficheros y carpetas "de solo lectura" al nacer

Ficheros: calcula como carpeta (`7 − cifra`) y **quita la** `x` del resultado. **No es una resta** (`umask 033` → ficheros 644, no 633). Dónde: `umask` (shell) · `~/.bashrc` · `/etc/login.defs` + `pam_umask` · `UMask=` en systemd. Ubuntu: `0002` si tu grupo se llama como tú. `sudo` fuerza ≥ `022`. Obedecen la umask: `touch`, `>`, `mkdir`, `cp`. **No:** `mv`, `cp -p`, `mkdir -m`, `install -m`, `chmod`.

9 · `chown` y `chgrp`

Comando	Quién puede
`chown ana f` · `chown ana:equipo f` · `chown :equipo f` · `chown ana: f` (su grupo de login)	**solo** `root` el dueño
`chgrp equipo f`	`root`, o el dueño **si es miembro** de `equipo`
`-R` recursivo · `--from=X` · `--reference=OTRO` · `-v` `-c`	`chown`/`chgrp` **borra SUID/SGID** de un fichero: primero `chown`, luego `chmod`

`usermod -aG grupo usuario` (¡con **-a**!) y **volver a iniciar sesión** (o `newgrp`/`sg`). `sudo cp` deja ficheros de `root` → `sudo chown "$USER": f`.

10 · Especiales

Bit	Octal	Letra	En un fichero	En una carpeta
SUID	4	`s` (x del dueño)	corre con los privilegios del **dueño** (`passwd`). **Ignorado en scripts**	—
SGID	2	`s` (x del grupo)	corre con el **grupo** del fichero (`crontab`)	lo nuevo **hereda el grupo** (y el SGID)

Bit	Octal	Letra	En un fichero	En una carpeta
Sticky	1	`t` (x de otros)	—	solo el dueño del fichero/carpeta o root **borra** (`/tmp` = 1777)

`S` / `T` mayúscula = el bit **sin** `x` debajo. Poner: `chmod u+s` · `g+s` · `+t` · `4755` `2775` `1777`. Auditar: `find / -perm -4000 -type f` · `-perm -2000` · `-perm /6000` (SUID **o** SGID) · `find / -xdev -type d -perm -0002 ! -perm -1000` (públicas sin sticky) · `find / -nouser -o -nogroup`. `-perm 644` exacto · `-perm -644` **todos** esos bits · `-perm /644` **alguno**.

11 · Receta de carpeta de equipo

```
sudo chgrp equipo carpeta && sudo chmod 2770 carpeta      # SGID: lo nuevo hereda el grupo
umask 002                                                 # los miembros: lo nuevo sale rw-rw----/664
```

12 · Diagnóstico de "Permission denied"

`Permission denied` = falta un **permiso** · `Operation not permitted` = acción **reservada** (chown sin root, chmod sin ser dueño, sticky, inmutable). **1** leer el mensaje · **2** `id` · **3** `namei -l ruta` · **4** `ls -ld`, `stat` · **5** `sudo diagnostica -u usuario ruta` · **6** `lsattr`, `findmnt -T`, `getfacl`, `getcap`, AppArmor · **7** arreglar con el cambio **mínimo** (nunca `777`). No es de permisos: `command not found` (PATH) · `Is a directory` · `bad interpreter` / `required file not found` (shebang o `\r`) · `Exec format error` · `Read-only file system`.

13 · Más allá de `rwx`

`chattr +i f` inmutable (ni root) · `chattr +a f` solo-añadir · `lsattr` · `getfacl` / `setfacl -m u:ana:rw f` (ACL: `+` en `ls -l`) · `setcap` / `getcap` · montajes `ro` `noexec` `nosuid` · MAC: AppArmor (Ubuntu), SELinux.

14 · Ficheros clave

`/etc/passwd` 644 · `/etc/shadow` 640 root:shadow · `/etc/sudoers` 440 · `~/.ssh` 700 y clave privada 600 · `/tmp` 1777 · `/var/mail` 2775 root:mail · `/home/usuario` 750 · `lrwxrwxrwx` (enlace: mandan los permisos del destino).